

```
=====
File: frontend\src\admin\AdminDashboard.tsx
=====
```

```

    profession: string;
    experience: number;
    isVerified: boolean;
    hourlyRate: number;
  }
}

interface Booking {
  _id: string;
  customerId: {
    firstName: string;
    lastName: string;
    email: string;
  };
  serviceProviderId: {
    userId: {
      firstName: string;
      lastName: string;
      email: string;
    };
  };
  serviceType: string;
  status: string;
  totalAmount: number;
  scheduledDate: string;
}

const AdminDashboard = () => {
  const [stats, setStats] = useState<Stats | null>(null);
  const [users, setUsers] = useState<User[]>([]);
  const [providers, setProviders] = useState<Provider[]>([]);
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [reviews, setReviews] = useState<Review[]>([]);
  const [loading, setLoading] = useState(true);
  const [activeTab, setActiveTab] = useState<'overview' | 'users' | 'providers' | 'bookings' | 'profile' | 'reviews'>('overview');
  const [searchTerm, setSearchTerm] = useState('');

  // Profile Logic
  const { user: authUser, updateUser, logout } = useAuthStore();
  const navigate = useNavigate();
  const [profileLoading, setProfileLoading] = useState(false);
  const [profileData, setProfileData] = useState({
    firstName: authUser?.firstName || '',
    lastName: authUser?.lastName || '',
    email: authUser?.email || '',
    phone: authUser?.phone || '',
    profilePhoto: authUser?.profilePhoto || ''
  });

  const handleProfileUpdate = async (e: React.FormEvent) => {
    e.preventDefault();
    setProfileLoading(true);
    try {
      const response = await api.put('/auth/profile', profileData);
      updateUser(response.data.user);
      toast.success('Profile updated successfully');
    } catch (error: any) {
      toast.error(error.response?.data?.message || 'Something went wrong');
    } finally {
      setProfileLoading(false);
    }
  };

  const handleLogout = () => {
    logout();
    navigate('/');
  };

  useEffect(() => {
    fetchData();
  }, [activeTab]);
}

```

```

const fetchData = async () => {
  try {
    if (activeTab === 'overview') {
      const statsRes = await api.get('/admin/stats');
      setStats(statsRes.data.stats);
    } else if (activeTab === 'users') {
      const usersRes = await api.get('/admin/users');
      setUsers(usersRes.data);
    } else if (activeTab === 'providers') {
      const providersRes = await api.get('/admin/providers');
      setProviders(providersRes.data);
    } else if (activeTab === 'bookings') {
      const bookingsRes = await api.get('/admin/bookings');
      setBookings(bookingsRes.data);
    } else if (activeTab === 'reviews') {
      const reviewsRes = await api.get('/admin/reviews');
      setReviews(reviewsRes.data);
    }
    setLoading(false);
  } catch (error: any) {
    setLoading(false);
  }
};

const toggleUserStatus = async (userId: string) => {
  try {
    await api.post(`/admin/users/${userId}/toggle-active`);
    toast.success('User status updated');
    fetchData();
  } catch (error) {}
};

const verifyProvider = async (providerId: string) => {
  try {
    await api.post(`/admin/providers/${providerId}/verify`);
    toast.success('Provider verified');
    fetchData();
  } catch (error) {}
};

const deleteUser = async (userId: string) => {
  if (!window.confirm('Are you sure you want to permanently delete this user?')) return;
  try {
    await api.delete(`/admin/users/${userId}`);
    toast.success('User deleted permanently');
    fetchData();
  } catch (error) { toast.error('Failed to delete user'); }
};

const removeProvider = async (providerId: string) => {
  if (!window.confirm('Are you sure you want to remove this provider? They will be downgraded to a customer.')) return;
  try {
    await api.delete(`/admin/providers/${providerId}`);
    toast.success('Provider removed');
    fetchData();
  } catch (error) { toast.error('Failed to remove provider'); }
};

const updateBookingStatus = async (bookingId: string, status: string) => {
  try {
    await api.put(`/admin/bookings/${bookingId}/status`, { status });
    toast.success(`Booking marked as ${status}`);
    fetchData();
  } catch (error) { toast.error('Failed to update booking'); }
};

const deleteReview = async (reviewId: string) => {
  if (!window.confirm('Are you sure you want to remove this review?')) return;
  try {
    await api.delete(`/admin/reviews/${reviewId}`);
  }
};

```

```

        toast.success('Review removed');ð
        fetchData();ð
    } catch (error) { toast.error('Failed to remove review'); }ð
};ð
ð
const filteredData = () => {ð
    const term = searchTerm.toLowerCase();ð
    if (activeTab === 'users') {ð
        return users.filter(u => u.firstName.toLowerCase().includes(term) ||
u.email.toLowerCase().includes(term));ð
    }ð
    if (activeTab === 'providers') {ð
        return providers.filter(p =>
p.userId.firstName.toLowerCase().includes(term) ||
p.profession.toLowerCase().includes(term));ð
    }ð
    if (activeTab === 'bookings') {ð
        return bookings.filter(b => b.serviceType.toLowerCase().includes(term)
|| b.customerId.firstName.toLowerCase().includes(term));ð
    }ð
    return [];ð
};ð
ð
if (loading && !stats && activeTab === 'overview') {ð
    return (ð
        <div className="flex items-center justify-center min-h-screen bg-
gray-50">ð
            <div className="animate-spin rounded-full h-12 w-12 border-t-2
border-b-2 border-indigo-600"></div>ð
            </div>ð
        );ð
    }ð
ð
    return (ð
        <div className="min-h-screen bg-neutral-50 p-4 md:p-8 font-sans transition-
colors duration-300">ð
            <div className="max-w-7xl mx-auto">ð
                { /* Header */ }ð
                <header className="flex flex-col md:flex-row md:items-center justify-
between mb-10 gap-6">ð
                    <div className="space-y-1">ð
                        <h1 className="text-4xl font-extrabold text-neutral-900
tracking-tight">System Control</h1>ð
                        <p className="text-neutral-500 font-medium">Platform
Management & Analytics Overview</p>ð
                    </div>ð
                    <nav className="inline-flex bg-white/80 backdrop-blur-md p-1.5
rounded-2xl shadow-sm border border-neutral-200/60 sticky top-4 z-10 transition-all
hover:shadow-md flex-wrap">ð
                        {[ 'overview', 'users', 'providers', 'bookings', 'reviews',
'profile' ] as const}.map((tab) => (ð
                            <buttonð
                                key={tab}ð
                                onClick={() => { setActiveTab(tab);
setLoading(true); }}ð
                                className={ `px-6 py-2.5 rounded-xl text-sm font-bold
transition-all duration-300 ${activeTab === tabð
                                    ? 'bg-neutral-900 text-white shadow-lg scale-105'ð
                                    : 'text-neutral-500 hover:text-neutral-900
hover:bg-neutral-100'ð
                                } ` }ð
                            >ð
                                {tab.charAt(0).toUpperCase() + tab.slice(1)}ð
                            </button>ð
                        )))ð
                    </nav>ð
                </div>ð
                { /* Overview Tab */ }ð
                {activeTab === 'overview' && (ð
                    <div className="animate-in fade-in slide-in-from-bottom-4

```

```

duration-500 space-y-10">ð
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-
cols-4 gap-6">ð
        {[ð
          { label: 'Total Revenue', value:
stats?.totalRevenue ? ` $${stats.totalRevenue.toLocaleString()}` : ' 10', icon:
DollarSign, color: 'text-amber-600', bg: 'bg-amber-100/50' },ð
          { label: 'Total Base', value: stats?.totalUsers ||
0, icon: Users, color: 'text-blue-600', bg: 'bg-blue-100/50' },ð
          { label: 'Total Ops', value: stats?.totalBookings ||
0, icon: Calendar, color: 'text-violet-600', bg: 'bg-violet-100/50' },ð
          { label: 'Verification Queue', value:
stats?.pendingVerifications || 0, icon: Clock, color: 'text-orange-600', bg: 'bg-
orange-100/50' },ð
        ].map((stat, i) => (ð
          <div key={i} className="group bg-white p-7
rounded-3xl border border-neutral-200/60 hover:border-neutral-900 transition-all
duration-300 hover:shadow-xl hover:-translate-y-1">ð
            <div className="flex items-center justify-
between mb-4">ð
              <div className={` ${stat.bg} ${stat.color}
p-3 rounded-2xl transition-transform group-hover:rotate-6`} >ð
                <stat.icon size={26} />ð
              </div>ð
              <span className="text-neutral-400 font-bold
group-hover:text-neutral-900 transition-colors">+{Math.floor(Math.random() * 20)}%</
span>ð
            </div>ð
            <h3 className="text-sm font-bold text-
neutral-500 uppercase tracking-widest mb-1">{stat.label}</h3>ð
            <p className="text-4xl font-black text-
neutral-900">{stat.value}</p>ð
          </div>ð
        )))ð
      </div>ð
ð
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">ð
        <div className="lg:col-span-2 bg-neutral-900 rounded-
[2.5rem] p-10 text-white relative overflow-hidden group shadow-2xl">ð
          <div className="absolute top-0 right-0 w-64 h-64 bg-
indigo-500/20 blur-[100px] rounded-full -mr-20 -mt-20"></div>ð
          <div className="relative z-10 flex flex-col h-full">ð
            <div className="flex items-center justify-
between mb-10">ð
              <h2 className="text-2xl font-black flex
items-center gap-3">ð
                <BarChart3 className="text-indigo-400"
size={28} />ð
                Platform Momentumð
              </h2>ð
              <select className="bg-white/10 border-none
rounded-xl text-sm px-4 py-2 focus:ring-0">ð
                <option>Last 30 Days</option>ð
              </select>ð
            </div>ð
            <div className="flex-1 flex items-end gap-3 min-
h-[250px]">ð
              {[45, 78, 56, 92, 64, 85, 72, 59, 95, 81,
68, 88].map((h, i) => (ð
                <div key={i} className="flex-1 group/bar
relative">ð
                  <divð
                    className="w-full bg-white/20
rounded-xl transition-all duration-500 group-hover/bar:bg-indigo-400 group-hover/
bar:scale-105"ð
                    style={{ height: `${h}%` }}ð
                  </div>ð
                  <div className="absolute -top-10
left-1/2 -translate-x-1/2 bg-white text-neutral-900 px-2 py-1 rounded-lg text-[10px]
font-black opacity-0 group-hover/bar:opacity-100 transition-opacity">ð
                    {h}%ð
                  </div>ð
                )

```

```

        </div>ð
    )))ð
    </div>ð
    </div>ð
    </div>ð
ð
    <div className="space-y-6">ð
        <div className="bg-white rounded-[2.5rem] p-8 border
border-neutral-200/60 shadow-lg">ð
            <h3 className="text-xl font-black text-
neutral-900 mb-6 tracking-tight">System Integrity</h3>ð
            <div className="space-y-5">ð
                {[ð
                    { l: 'Server Latency', v: '24ms', c:
'text-emerald-500' },ð
                    { l: 'Auth Uptime', v: '99.9%', c: 'text-
blue-500' },ð
                    { l: 'Payment Gate', v: 'Active', c:
'text-indigo-500' },ð
                    { l: 'Db Health', v: 'Optimal', c: 'text-
emerald-500' }ð
                ]}.map((item, i) => (ð
                    <div key={i} className="flex items-
center justify-between p-4 bg-neutral-50 rounded-2xl hover:bg-neutral-100 transition-
colors">ð
                        <span className="text-neutral-500
font-bold">{item.l}</span>ð
                        <span className={`font-black
${item.c}`}>{item.v}</span>ð
                    </div>ð
                )))ð
            </div>ð
        </div>ð
ð
    <div className="bg-gradient-to-br from-indigo-600 to-
indigo-800 rounded-[2.5rem] p-8 text-white shadow-xl relative overflow-hidden group">ð
        <ShieldCheck className="absolute bottom-[-20px]
right-[-20px] text-white/10 w-40 h-40 group-hover:scale-110 transition-transform
duration-700" />ð
        <h3 className="text-xl font-black mb-2">Security
Hub</h3>ð
        <p className="text-white/70 text-sm mb-6 leading-
relaxed">System-wide security protocol and access log monitoring.</p>ð
        <button className="bg-white text-indigo-900 px-6
py-3 rounded-2xl font-black text-sm hover:shadow-lg transition-all active:scale-95">ð
            Enter Secure Modeð
        </button>ð
    </div>ð
</div>ð
</div>ð
    ))ð
ð
    {/* Profile Tab */}ð
    {activeTab === 'profile' && (ð
        <div className="bg-white rounded-[2.5rem] border border-
neutral-200/60 shadow-2xl overflow-hidden animate-in fade-in slide-in-from-bottom-8
duration-500">ð
            <div className="p-10 border-b border-neutral-100 flex flex-
col md:flex-row md:items-center justify-between gap-6 bg-neutral-50/50">ð
                <div className="space-y-1">ð
                    <h2 className="text-3xl font-black text-neutral-900
tracking-tight capitalize">Admin Profile</h2>ð
                    <p className="text-neutral-500 font-medium">Manage
your personal information and credentials</p>ð
                </div>ð
                <button onClick={handleLogout} className="flex items-
center gap-2 px-6 py-3 bg-rose-50 text-rose-600 rounded-xl font-bold hover:bg-
rose-100 transition-colors">ð
                    <LogOut size={20} />ð
                    Sign Outð
                </button>ð
            </div>ð
        </div>ð
    )}ð

```

```

        </div>ð
        <div className="p-10">ð
            <form onSubmit={handleProfileUpdate} className="space-
y-8 max-w-3xl">ð
                { /* Profile Photo Section */ }ð
                <div className="flex flex-col sm:flex-row items-
center gap-6 pb-8 border-b border-neutral-100">ð
                    <div className="relative group cursor-pointer">ð
                        {profileData.profilePhoto ? (ð
                            <imgð
                                src={profileData.profilePhoto}ð
                                alt="Profile"ð
                                className="h-28 w-28 rounded-full
object-cover border-4 border-white shadow-xl"ð
                                />ð
                            ) : (ð
                                <div className="h-28 w-28 rounded-full
bg-indigo-100 flex items-center justify-center border-4 border-white shadow-xl">ð
                                    <UserIcon className="h-12 w-12 text-
indigo-600" />ð
                                </div>ð
                            ) ) }ð
                        <div className="absolute inset-0 rounded-
full bg-black/40 flex items-center justify-center opacity-0 group-hover:opacity-100
transition-opacity">ð
                            <Camera className="h-8 w-8 text-white" />ð
                        </div>ð
                    </div>ð
                    <div className="flex-1 w-full text-center
sm:text-left space-y-2">ð
                        <h3 className="font-bold text-
neutral-900">Avatar URL</h3>ð
                        <inputð
                            type="text"ð
                            placeholder="Paste image URL here..."ð
                            className="w-full px-4 py-3 bg-
neutral-50 border border-neutral-200 rounded-xl focus:ring-2 focus:ring-indigo-500
focus:outline-none transition-all"ð
                                value={profileData.profilePhoto}ð
                                onChange={(e) =>
setProfileData({ ...profileData, profilePhoto: e.target.value })}ð
                                />ð
                            <p className="text-xs font-medium text-
neutral-400">Provide a direct link to an image to use as your avatar</p>ð
                        </div>ð
                    </div>ð
                </div>ð
                { /* Form Fields */ }ð
                <div className="grid grid-cols-1 md:grid-cols-2
gap-8">ð
                    <div className="space-y-2">ð
                        <label className="text-sm font-bold text-
neutral-700">First Name</label>ð
                        <div className="relative">ð
                            <UserIcon className="absolute left-4
top-1/2 -translate-y-1/2 text-neutral-400" size={20} />ð
                            <inputð
                                type="text"ð
                                className="w-full pl-12 pr-4 py-3 bg-
neutral-50 border border-neutral-200 rounded-xl focus:ring-2 focus:ring-indigo-500
focus:outline-none transition-all font-medium"ð
                                    value={profileData.firstName}ð
                                    onChange={(e) =>
setProfileData({ ...profileData, firstName: e.target.value })}ð
                                    requiredð
                                />ð
                            </div>ð
                        </div>ð
                    </div>ð
                    <div className="space-y-2">ð
                        <label className="text-sm font-bold text-
neutral-700">Last Name</label>ð

```

```

        <div className="relative">ð
            <UserIcon className="absolute left-4
top-1/2 -translate-y-1/2 text-neutral-400" size={20} />ð
            <inputð
                type="text"ð
                className="w-full pl-12 pr-4 py-3 bg-
neutral-50 border border-neutral-200 rounded-xl
focus:ring-2 focus:ring-indigo-500
focus:outline-none transition-all font-medium"ð
                value={profileData.lastName}ð
                onChange={(e) =>
setProfileData({ ...profileData, lastName: e.target.value })}ð
                requiredð
            />ð
        </div>ð
    </div>ð
    ð
        <div className="space-y-2">ð
            <label className="text-sm font-bold text-
neutral-700">Contact Email</label>ð
            <div className="relative">ð
                <Mail className="absolute left-4 top-1/2
-translate-y-1/2 text-neutral-400" size={20} />ð
                <inputð
                    type="email"ð
                    className="w-full pl-12 pr-4 py-3 bg-
neutral-100 border border-neutral-200 rounded-xl text-neutral-500 font-medium cursor-
not-allowed"ð
                    value={profileData.email}ð
                    disabledð
                    title="Email cannot be changed"ð
                />ð
            </div>ð
        </div>ð
    ð
        <div className="space-y-2">ð
            <label className="text-sm font-bold text-
neutral-700">Phone Number</label>ð
            <div className="relative">ð
                <Phone className="absolute left-4
top-1/2 -translate-y-1/2 text-neutral-400" size={20} />ð
                <inputð
                    type="text"ð
                    className="w-full pl-12 pr-4 py-3 bg-
neutral-50 border border-neutral-200 rounded-xl focus:ring-2 focus:ring-indigo-500
focus:outline-none transition-all font-medium"ð
                    value={profileData.phone}ð
                    onChange={(e) =>
setProfileData({ ...profileData, phone: e.target.value })}ð
                    requiredð
                />ð
            </div>ð
        </div>ð
    ð
        <div className="pt-6">ð
            <buttonð
                type="submit"ð
                disabled={profileLoading}ð
                className="w-full sm:w-auto flex items-
center justify-center gap-2 bg-neutral-900 text-white px-8 py-4 rounded-xl font-
black text-sm hover:bg-indigo-600 focus:ring-4 focus:ring-indigo-500/20
active:scale-95 transition-all disabled:opacity-50"ð
            >ð
                <Save size={20} />ð
                {profileLoading ? 'Synchronizing...' : 'Save
Changes'}ð
            </button>ð
        </div>ð
    </form>ð
</div>ð
</div>ð
))ð

```



```

ð
        { /* Data Tabs */ }ð
        { activeTab !== 'overview' && activeTab !== 'profile' && (ð
            <div className="bg-white rounded-[2.5rem] border border-
neutral-200/60 shadow-2xl overflow-hidden animate-in fade-in slide-in-from-bottom-8
duration-500">ð
                <div className="p-10 border-b border-neutral-100 flex flex-
col md:flex-row md:items-center justify-between gap-6 bg-neutral-50/50">ð
                    <div className="space-y-1">ð
                        <h2 className="text-3xl font-black text-neutral-900
tracking-tight capitalize">{activeTab} Database</h2>ð
                        <p className="text-neutral-500 font-medium">Manage
and monitor live {activeTab} information</p>ð
                    </div>ð
                    <div className="flex items-center gap-3">ð
                        <div className="relative group">ð
                            <Search className="absolute left-4 top-1/2 -
translate-y-1/2 text-neutral-400 group-hover:text-neutral-900 transition-colors"
size={20} />ð
                            <inputð
                                type="text"ð
                                placeholder={`Filter by name, email or
type...`}ð
                                value={searchTerm}ð
                                onChange={(e) =>
setSearchTerm(e.target.value)}ð
                                className="pl-12 pr-6 py-4 bg-white border
border-neutral-200/80 rounded-[1.5rem] focus:outline-none focus:ring-4 focus:ring-
neutral-900/5 focus:border-neutral-900 w-full md:w-80 font-medium transition-all"ð
                            />ð
                        </div>ð
                        <button className="p-4 bg-white border border-
neutral-200/80 rounded-[1.5rem] hover:bg-neutral-100 transition-colors shadow-sm">ð
                            <Filter size={20} className="text-neutral-600" />ð
                        </button>ð
                    </div>ð
                </div>ð
            </div>ð
            <div className="overflow-x-auto">ð
                <table className="w-full">ð
                    { activeTab === 'users' && (ð
                        <>ð
                            <thead className="bg-neutral-50">ð
                                <tr>ð
                                    <th className="px-10 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Principal</th>ð
                                    <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Identity</th>ð
                                    <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Lifecycle</th>ð
                                    <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Timeline</th>ð
                                    <th className="px-10 py-6 text-right
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Ops</th>ð
                                </tr>ð
                            </thead>ð
                            <tbody className="divide-y divide-
neutral-100">ð
                                {filteredData().map((u: any) => (ð
                                    <tr key={u._id} className="group
hover:bg-neutral-50/80 transition-all duration-300">ð
                                        <td className="px-10 py-7">ð
                                            <div className="flex items-
center gap-5">ð
                                                <div className="h-14
w-14 rounded-[1.2rem] bg-indigo-100 text-indigo-700 flex items-center justify-center
font-black text-xl shadow-inner group-hover:scale-110 transition-transform">ð
                                                    {u.firstName[0]}ð
                                                </div>ð
                                                <div>ð
                                                    <p className="font-
black text-neutral-900 text-lg uppercase tracking-tight">{u.firstName} {u.lastName}</
p>ð

```

```

        <p class=
neutral-400 font-bold">{u.email}</p>
        </div>
        </div>
        </td>
        <td class=
px-6 py-7">
            <span class=
px-4
py-1.5 rounded-full text-[10px] font-black uppercase tracking-widest ${u.role ===
'admin' ? 'bg-indigo-900 text-white' :
'bg-neutral-100 text-
neutral-600'
            </span>
            </td>
            <td class=
px-6 py-7">
                <div class=
center gap-3">
                    <div class=
rounded-full ${u.isActive ? 'bg-emerald-500 shadow-[0_0_12px_rgba(16,185,129,0.5)]
animate-pulse' : 'bg-rose-500'}">
                        <span class=
sm font-black uppercase tracking-tighter ${u.isActive ? 'text-emerald-700' : 'text-
rose-700'}">
                            {u.isActive ?
'Active Mode' : 'Suspended'}
                        </span>
                    </div>
                </td>
                <td class=
px-6 py-7 text-
neutral-400 font-bold text-sm">
                    Date(u.createdAt).toLocaleDateString('en-US', { day: 'numeric', month: 'short',
year: 'numeric' })
                </td>
                <td class=
px-10 py-7 text-
right">
                    <div class=
center justify-end gap-2">
                        <button onClick=
toggleUserStatus(u._id)} class=
p-4 rounded-2xl transition-all ${u.isActive ?
'text-rose-600 hover:bg-rose-100 hover:scale-110' : 'text-emerald-600 hover:bg-
emerald-100 hover:scale-110'}" title=
{u.isActive ? 'Suspend User' : 'Activate User'}
                        >
                            <UserX size=
{22}
className=
{u.isActive ? 'rotate-0' : 'rotate-180 transition-transform
duration-500'} />
                        </button>
                        <button onClick=
deleteUser(u._id)} class=
p-4 rounded-2xl text-red-600 hover:bg-red-100
hover:scale-110 transition-all" title=
Delete Permanently">
                            <Trash2 size=
{22} />
                        </button>
                    </div>
                </td>
            </tr>
        </tbody>
    </table>
    </div>
    <div>
        {activeTab === 'providers' && (
            <thead class=
bg-neutral-50">
                <tr>
                    <th class=
px-10 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]>Specialist</th>
                    <th class=
px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]>Profession</th>
                    <th class=
px-6 py-6 text-left

```

```

text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Experience</th>ð
      <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Certificate</th>ð
      <th className="px-10 py-6 text-right
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Ops</th>ð
    </tr>ð
  </thead>ð
  <tbody className="divide-y divide-
neutral-100">ð
    {filteredData().map((p: any) => (ð
      <tr key={p._id} className="group
hover:bg-neutral-50/80 transition-colors">ð
      <td className="px-10 py-7">ð
        <div className="flex items-
center gap-5">ð
          <div className="h-14
w-14 rounded-[1.2rem] bg-emerald-100 text-emerald-700 flex items-center justify-
center font-black group-hover:bg-emerald-200 transition-colors">ð
            {p.userId.firstName[0]}ð
          </div>ð
          <div>ð
            <p className="font-
black text-neutral-900 uppercase tracking-tight">{p.userId.firstName}
{p.userId.lastName}</p>ð
            <p className="text-
neutral-400 font-bold text-sm">{p.userId.email}</p>ð
          </div>ð
        </div>ð
      </td>ð
      <td className="px-6 py-7 font-
black text-neutral-700 uppercase italic tracking-tighter">{p.profession}</td>ð
      <td className="px-6 py-7 text-
neutral-900 font-black">{p.experience} Years</td>ð
      <td className="px-6 py-7">ð
        {p.isVerified ? (ð
          <div className="flex
items-center gap-2 text-emerald-600 font-black text-xs uppercase bg-emerald-50
border border-emerald-100 px-3 py-1.5 rounded-xl w-fit">ð
            <ShieldCheck
size={14} /> Officialð
          </div>ð
        ) : (ð
          <div className="flex
items-center gap-2 text-rose-600 font-black text-xs uppercase bg-rose-50 border
border-rose-100 px-3 py-1.5 rounded-xl w-fit">ð
            <Clock size={14} />
Pendingð
          </div>ð
        )}ð
      </td>ð
      <td className="px-10 py-7 text-
right">ð
        <div className="flex items-
center justify-end gap-3">ð
          {!p.isVerified && (ð
            <button onClick={()
=> verifyProvider(p._id)} className="bg-neutral-900 text-white px-6 py-3 rounded-2xl
font-black text-[10px] uppercase tracking-widest hover:bg-emerald-600 transition-all
hover:shadow-lg active:scale-95" title="Verify Provider">ð
              Authorizedð
            </button>ð
          )}ð
          <button onClick={() =>
removeProvider(p._id)} className="p-3 bg-red-50 text-red-600 hover:bg-red-600
hover:text-white rounded-2xl transition-all shadow-sm" title="Remove Provider
Listing">ð
            <Trash2 size={18} />ð
          </button>ð
        </div>ð
      </td>ð
    </tr>ð
  </tbody>ð
</table>ð

```

```

    )))
  </tbody>
</>
))
{activeTab === 'bookings' && (
  <>
    <thead className="bg-neutral-50">
      <tr>
        <th className="px-10 py-6 text-left text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Trans ID</th>
        <th className="px-6 py-6 text-left text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Stakeholders</th>
        <th className="px-6 py-6 text-left text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Status</th>
        <th className="px-6 py-6 text-left text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Volume</th>
        <th className="px-10 py-6 text-right text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Link</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-neutral-100">
      {filteredData().map((b: any) => (
        <tr key={b._id} className="group hover:bg-neutral-50/80 transition-colors">
          <td className="px-10 py-7">
            <p className="font-mono text-xs font-black text-neutral-400">#SC-{b._id.slice(-6).toUpperCase()}</p>
            <p className="font-black text-neutral-900 mt-1 uppercase text-sm tracking-tight">{b.serviceType}</p>
          </td>
          <td className="px-6 py-7">
            <div className="flex items-center gap-3">
              <div className="flex -space-x-3">
                <div className="h-10 w-10 rounded-full border-2 border-white bg-indigo-500 text-white flex items-center justify-center text-[10px] font-black uppercase ring-2 ring-neutral-50">{b.customerId?.firstName?.[0]}</div>
                <div className="h-10 w-10 rounded-full border-2 border-white bg-emerald-500 text-white flex items-center justify-center text-[10px] font-black uppercase ring-2 ring-neutral-50">{b.serviceProviderId?.userId?.firstName?.[0]}</div>
              </div>
              <span className="text-neutral-500 font-bold text-sm tracking-tighter">Client x Specialist</span>
            </div>
          </td>
          <td className="px-6 py-7">
            <select
              value={b.status}
              onChange={(e) => updateBookingStatus(b._id, e.target.value)}
              className={`px-4 py-2 rounded-xl text-[10px] font-black uppercase tracking-widest cursor-pointer outline-none border border-transparent hover:border-neutral-200 transition-all ${b.status === 'completed' ? 'bg-emerald-50 text-emerald-700' : b.status === 'pending' ? 'bg-amber-50 text-amber-700' : b.status === 'cancelled' ? 'bg-rose-50 text-rose-700' : 'bg-indigo-50 text-indigo-700'}`>
              <option value="pending">Pending</option>
              <option value="confirmed">Confirmed</option>
            </select>
          </td>
        </tr>
      ))}
    </tbody>
  </>
)

```

```

value="in_progress">In Progress</option>ð
value="completed">Completed</option>ð
value="cancelled">Cancelled</option>ð
</select>ð
</td>ð
<td className="px-6 py-7">ð
  <p className="text-xl font-
black text-neutral-900"> ¹{b.totalAmount}</p>ð
  <p className="text-[10px]
text-neutral-400 font-black uppercase tracking-tight">{new
Date(b.scheduledDate).toLocaleDateString()}</p>ð
</td>ð
<td className="px-10 py-7 text-
right">ð
  <button className="p-4
rounded-2xl bg-neutral-100 text-neutral-600 hover:bg-neutral-900 hover:text-white
transition-all shadow-sm">ð
    <ExternalLink size={18} /
>ð
  </button>ð
</td>ð
</tr>ð
  )}}ð
</tbody>ð
</>ð
)}ð
ð
  {activeTab === 'reviews' && (ð
    <>ð
      <thead className="bg-neutral-50">ð
        <tr>ð
          <th className="px-10 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Context</th>ð
          <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Rating</th>ð
          <th className="px-6 py-6 text-left
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Content</th>ð
          <th className="px-10 py-6 text-right
text-xs font-black text-neutral-400 uppercase tracking-[0.2em]">Ops</th>ð
        </tr>ð
      </thead>ð
      <tbody className="divide-y divide-
neutral-100">ð
        {reviews.map((r: any) => (ð
          <tr key={r._id} className="group
hover:bg-neutral-50/80 transition-colors">ð
            <td className="px-10 py-7">ð
              <p className="font-black
text-neutral-900 uppercase text-sm tracking-tight">{r.customerId?.firstName} !'
{r.serviceProviderId?.userId?.firstName}</p>ð
              <p className="text-[10px]
text-neutral-400 font-black uppercase tracking-tight mt-1">{new
Date(r.createdAt).toLocaleDateString()}</p>ð
            </td>ð
            <td className="px-6 py-7">ð
              <div className="flex items-
center gap-1">ð
                {[...Array(5)].map((_,
i) => (ð
                  <Star key={i}
size={16} className={i < r.rating ? "text-amber-500 fill-amber-500" : "text-
neutral-200"} />ð
                )}}ð
              </div>ð
            </td>ð
            <td className="px-6 py-7">ð
              <p className="text-sm font-
medium text-neutral-600 line-clamp-2 max-w-md">{r.comment}</p>ð
            </td>ð
          )
        )
      </tbody>
    </>
  )
}

```



```

    firstName: string;ð
    lastName: string;ð
  };ð
  serviceProviderId: {ð
    businessName?: string;ð
    userId: {ð
      firstName: string;ð
      lastName: string;ð
    };ð
  };ð
};ð
}ð
ð
const BookingDetailsPage = () => {ð
  const { id } = useParams();ð
  const { user } = useAuthStore();ð
  const [booking, setBooking] = useState<Booking | null>(null);ð
  const [loading, setLoading] = useState(true);ð
ð
  useEffect(() => {ð
    if (id) {ð
      fetchBooking();ð
    }ð
  }, [id]);ð
ð
  const fetchBooking = async () => {ð
    try {ð
      const response = await api.get(`/bookings/${id}`);ð
      setBooking(response.data);ð
    } catch (error) {ð
      toast.error('Failed to load booking');ð
    } finally {ð
      setLoading(false);ð
    }ð
  };ð
ð
  const updateStatus = async (status: string) => {ð
    try {ð
      await api.patch(`/bookings/${id}/status`, { status });ð
      toast.success('Status updated');ð
      fetchBooking();ð
    } catch (error: any) {ð
      toast.error(error.response?.data?.message || 'Failed to update status');ð
    }ð
  };ð
ð
  if (loading) {ð
    return (ð
      <div className="min-h-screen bg-gray-50">ð
        <Navbar />ð
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">ð
          <p className="text-center text-gray-600">Loading...</p>ð
        </div>ð
      </div>ð
    );ð
  }ð
ð
  if (!booking) {ð
    return (ð
      <div className="min-h-screen bg-gray-50">ð
        <Navbar />ð
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">ð
          <p className="text-center text-gray-600">Booking not found</p>ð
        </div>ð
      </div>ð
    );ð
  }ð
ð
  const isCustomer = user?.role === 'customer';ð
  const isProvider = user?.role === 'service_provider';ð
  const providerName = booking.serviceProviderId?.businessName ||ð
    `${booking.serviceProviderId?.userId?.firstName}ð
    ${booking.serviceProviderId?.userId?.lastName}`;ð

```

```

    return (
      <div className="min-h-screen bg-gray-50">
        <Navbar />
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
          { /* Status Header & Progress Tracker */ }
          <div className="bg-white rounded-2xl shadow-sm border border-gray-100 p-6 mb-8">
            <div className="flex flex-col md:flex-row md:items-center justify-between mb-8">
              <div>
                <p className="text-sm font-medium text-gray-500 mb-1">Booking Reference</p>
                <h1 className="text-xl font-mono font-bold text-gray-900">#{booking._id.slice(-8).toUpperCase()}</h1>
              </div>
              <div className="mt-4 md:mt-0">
                <span className={`px-4 py-2 rounded-full text-sm font-bold shadow-sm ${booking.status === 'completed' ? 'bg-green-500 text-white' : booking.status === 'confirmed' ? 'bg-blue-500 text-white' : booking.status === 'in_progress' ? 'bg-amber-500 text-white' : booking.status === 'cancelled' ? 'bg-red-500 text-white' : 'bg-gray-400 text-white'}`}>
                  {booking.status.replace('_', ' ').toUpperCase()}
                </span>
              </div>
            </div>
          </div>
          <div className="relative">
            <div className="overflow-hidden h-2 mb-4 text-xs flex rounded bg-gray-100">
              <div>
                style={{
                  width:
                    booking.status === 'pending' ? '25%' :
                    booking.status === 'confirmed' ? '50%' :
                    booking.status === 'in_progress' ? '75%' :
                    booking.status === 'completed' ? '100%' : '0%'
                }}
                className={`shadow-none flex flex-col text-center whitespace-nowrap text-white justify-center transition-all duration-500 ${booking.status === 'cancelled' ? 'bg-red-500 w-full' : 'bg-primary-600'}`}
              </div>
            </div>
            <div className="flex text-xs text-gray-500 justify-between font-medium">
              <span className={booking.status === 'pending' ? 'text-primary-600 font-bold' : ''}>Requested</span>
              <span className={booking.status === 'confirmed' ? 'text-primary-600 font-bold' : ''}>Confirmed</span>
              <span className={booking.status === 'in_progress' ? 'text-primary-600 font-bold' : ''}>In Progress</span>
              <span className={booking.status === 'completed' ? 'text-primary-600 font-bold' : ''}>Completed</span>
            </div>
          </div>
        </div>
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
          { /* Main Details */ }
          <div className="lg:col-span-2 space-y-8">
            <div className="bg-white rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
              <div className="bg-gray-50 px-6 py-4 border-b border-gray-100">
                <h2 className="text-lg font-bold text-gray-900">Service Details</h2>
              </div>
              <div className="p-6 space-y-6">
                <div>
                  <h3 className="text-sm font-semibold text-gray-400 uppercase tracking-wider mb-2">Service Requested</h3>
                  <p className="text-xl font-bold text-gray-900">

```



```

capitalize">{booking.serviceType}</p>
    </div>
    <div>
        <h3 className="text-sm font-semibold text-gray-400 uppercase tracking-wider mb-2">Task Description</h3>
        <p className="text-gray-700 leading-relaxed text-lg bg-gray-50 p-4 rounded-xl border border-gray-100 italic">
            "{booking.description}"
        </p>
    </div>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div className="flex items-center p-4 bg-gray-50 rounded-xl">
            <Calendar className="h-6 w-6 text-primary-600 mr-4" />
            <div>
                <p className="text-xs text-gray-500">Scheduled Date</p>
                <p className="font-bold text-gray-900">{new Date(booking.scheduledDate).toLocaleDateString(
                    undefined, { weekday: 'long', year: 'numeric', month: 'long', day: 'numeric' })}</p>
            </div>
        </div>
        <div className="flex items-center p-4 bg-gray-50 rounded-xl">
            <Clock className="h-6 w-6 text-primary-600 mr-4" />
            <div>
                <p className="text-xs text-gray-500">Preferred Time</p>
                <p className="font-bold text-gray-900">{booking.scheduledTime}</p>
            </div>
        </div>
    </div>
    <div className="flex items-start p-4 bg-gray-50 rounded-xl">
        <MapPin className="h-6 w-6 text-primary-600 mr-4 mt-1" />
        <div>
            <p className="text-xs text-gray-500">Service Location</p>
            <p className="font-bold text-gray-900 leading-snug">
                {booking.location.address}<br />
                {booking.location.city}, {booking.location.state}
                {booking.location.zipCode}
            </p>
        </div>
    </div>
    </div>
    <div>
        <div className="bg-white rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
            <div className="bg-gray-50 px-6 py-4 border-b border-gray-100 flex items-center justify-between">
                <h2 className="text-lg font-bold text-gray-900">Communication Center</h2>
                <span className="text-xs text-green-500 font-bold flex items-center">
                    <div className="h-2 w-2 rounded-full bg-green-500 mr-2 animate-pulse"></div>
                    Live Chat
                </span>
            </div>
            <ChatWindow bookingId={id!} />
        </div>
    </div>
    <div>
        <div className="space-y-8">
            <div>
                <div className="bg-white rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
                    <div className="bg-primary-600 px-6 py-4">
                        <h2 className="text-lg font-bold text-white">Price Summary</h2>
                    </div>
                    <div className="p-6 space-y-4">

```

```

        <div className="flex justify-between text-gray-600">ð
            <span>Base Rate</span>ð
            <span className="font-medium"> 1{(booking.totalAmount /
(booking.isEmergency ? 1.5 : 1)).toFixed(2)}</span>ð
        </div>ð
        <div className="flex justify-between text-gray-600">ð
            <span>Estimated Time</span>ð
            <span className="font-medium">{booking.estimatedHours} hrs</span>ð
        </div>ð
        {booking.isEmergency && (ð
            <div className="flex justify-between text-red-500 font-bold text-
sm">ð
                <span>Emergency Surcharge (50%)</span>ð
                <span>Included</span>ð
            </div>ð
        )}ð
        <div className="border-t border-gray-100 pt-4 mt-4">ð
            <div className="flex justify-between items-end">ð
                <span className="text-gray-900 font-bold">Grand Total</span>ð
                <span className="text-3xl font-black text-
primary-600"> 1{booking.totalAmount.toFixed(2)}</span>ð
            </div>ð
        </div>ð
    ð
    <div className="pt-4">ð
        <div className={`w-full py-2 rounded-lg text-center font-bold text-
xs uppercase tracking-widest ${booking.paymentStatus === 'paid' ? 'bg-green-100 text-
green-700' : 'bg-amber-100 text-amber-700'}ð
            `}>ð
            {booking.paymentStatus === 'paid' ? 'Invoice Paid' : 'Payment
Required'}ð
        </div>ð
    </div>ð
    ð
    {isCustomer && booking.paymentStatus === 'pending' &&
booking.status !== 'cancelled' && (ð
        <div className="pt-4">ð
            <PaymentButton bookingId={booking._id}
amount={booking.totalAmount} onSuccess={fetchBooking} />ð
        </div>ð
    )}ð
    </div>ð
</div>ð
ð
    {/* Provider Card */}ð
    <div className="bg-white rounded-2xl shadow-sm border border-gray-100
p-6">ð
        <h3 className="text-sm font-semibold text-gray-400 uppercase tracking-
wider mb-4">ð
            {isCustomer ? 'Your Service Expert' : 'Client Profile'}ð
        </h3>ð
        <div className="flex items-center mb-6">ð
            <div className="h-14 w-14 bg-primary-100 rounded-full flex items-
center justify-center text-primary-600 font-black text-xl mr-4 shadow-inner">ð
                {(isCustomer ? providerName :
booking.customerId.firstName).charAt(0).toUpperCase())}ð
            </div>ð
            <div>ð
                <h4 className="font-bold text-gray-900 text-lg leading-tight">ð
                    {isCustomer ? providerName : `${booking.customerId.firstName}
${booking.customerId.lastName}`}ð
                </h4>ð
                <p className="text-sm text-gray-500">Service Professional</p>ð
            </div>ð
        </div>ð
    ð
    <div className="space-y-3">ð
        {isProvider && (ð
            <>ð
                <div className="flex items-center text-sm text-gray-600
hover:text-primary-600 transition-colors cursor-pointer">ð
                    <Mail className="h-4 w-4 mr-3" />ð

```

```

                {(booking.customerId as any).email}}ð
            </div>ð
            <div className="flex items-center text-sm text-gray-600
hover:text-primary-600 transition-colors cursor-pointer">ð
                <Phone className="h-4 w-4 mr-3" />ð
                {(booking.customerId as any).phone}ð
            </div>ð
        </>ð
    )}ð
    <button className="w-full mt-4 py-3 border border-gray-200 rounded-
xl text-sm font-bold text-gray-700 hover:bg-gray-50 transition-all active:scale-95">ð
        View Full Profileð
    </button>ð
</div>ð
</div>ð
ð
    {/* Cancellation Action */}ð
    {isCustomer && booking.status === 'pending' && (ð
        <buttonð
            onClick={() => updateStatus('cancelled')}ð
            className="w-full flex items-center justify-center py-4 px-4
rounded-2xl font-bold bg-red-50 text-red-600 hover:bg-red-100 transition-all group"ð
            >ð
                <XCircle className="h-5 w-5 mr-2 group-hover:rotate-90 transition-
transform" />ð
                Cancel Requestð
            </button>ð
        )}ð
    ð
    {/* Provider Actions */}ð
    {isProvider && booking.status === 'pending' && (ð
        <buttonð
            onClick={() => updateStatus('confirmed')}ð
            className="w-full flex items-center justify-center py-4 px-4 mb-2
rounded-2xl font-bold bg-blue-50 text-blue-600 hover:bg-blue-100 transition-all"ð
            >ð
                Accept Booking & Confirmð
            </button>ð
        )}ð
    {isProvider && booking.status === 'confirmed' && (ð
        <buttonð
            onClick={() => updateStatus('in_progress')}ð
            className="w-full flex items-center justify-center py-4 px-4 mb-2
rounded-2xl font-bold bg-amber-50 text-amber-600 hover:bg-amber-100 transition-all"ð
            >ð
                Mark as In Progressð
            </button>ð
        )}ð
    {isProvider && booking.status === 'in_progress' && (ð
        <buttonð
            onClick={() => updateStatus('completed')}ð
            className="w-full flex items-center justify-center py-4 px-4 mb-2
rounded-2xl font-bold bg-green-50 text-green-600 hover:bg-green-100 transition-all"ð
            >ð
                Complete Jobð
            </button>ð
        )}ð
    </div>ð
</div>ð
</div>ð
</div>ð
    );ð
};ð
ð
export default BookingDetailsPage;ð
ð

```

```

=====
File: frontend\src\admin\BrowseProvidersPage.tsx
=====

```

```

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import Navbar from '../components/Navbar';
import { Search, Star, MapPin, Shield, Filter, ArrowRight } from 'lucide-react';
import api from '../api/api';
import toast from 'react-hot-toast';

interface ServiceProvider {
  _id: string;
  businessName?: string;
  serviceTypes: string[];
  hourlyRate: number;
  location: {
    city: string;
    state: string;
  };
  rating: number;
  totalReviews: number;
  isVerified: boolean;
  userId: {
    firstName: string;
    lastName: string;
  };
}

const BrowseProvidersPage = () => {
  const [providers, setProviders] = useState<ServiceProvider[]>([]);
  const [loading, setLoading] = useState(true);
  const [searchTerm, setSearchTerm] = useState('');
  const [serviceType, setServiceType] = useState('');
  const [minRating, setMinRating] = useState('');

  useEffect(() => {
    fetchProviders();
  }, [serviceType, minRating]);

  const fetchProviders = async () => {
    try {
      setLoading(true);
      const params: any = {};
      if (serviceType) params.serviceType = serviceType;
      if (minRating) params.minRating = minRating;

      const response = await api.get('/service-providers', { params });
      setProviders(response.data);
    } catch (error) {
      toast.error('Failed to load service providers');
    } finally {
      setLoading(false);
    }
  };

  const filteredProviders = providers.filter(provider => {
    const name = provider.businessName || `${provider.userId.firstName} ${provider.userId.lastName}`;
    return name.toLowerCase().includes(searchTerm.toLowerCase()) ||
      provider.serviceTypes.some(type =>
        type.toLowerCase().includes(searchTerm.toLowerCase())) ||
      provider.location.city.toLowerCase().includes(searchTerm.toLowerCase());
  });

  return (
    <div className="min-h-screen bg-dark-900 text-white relative">
      <div className="absolute top-0 right-0 w-[600px] h-[600px] bg-primary-600/10 rounded-full blur-[120px] pointer-events-none" />
      <Navbar />
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12 relative animate-fade-in">
        <div className="text-center mb-12">

```

```

    <h1 className="text-4xl font-bold font-heading mb-4">ð
      Find <span className="text-gradient">Elite Professionals</span>ð
    </h1>ð
    <p className="text-dark-300 max-w-2xl mx-auto">ð
      Connect with verified experts for your next project.ð
      Quality assured, satisfaction guaranteed.ð
    </p>ð
  </div>ð

ð
  { /* Filters */ }ð
  <div className="glass-card rounded-2xl border-white/5 p-6 mb-12 shadow-xl
shadow-black/20">ð
    <div className="flex items-center space-x-2 mb-6 text-primary-400">ð
      <Filter className="h-5 w-5" />ð
      <h2 className="font-semibold text-lg">Refine Search</h2>ð
    </div>ð

ð
    <div className="grid grid-cols-1 md:grid-cols-4 gap-4">ð
      <div className="relative group">ð
        <Search className="absolute left-3 top-1/2 transform -translate-y-1/2
h-5 w-5 text-dark-400 group-focus-within:text-primary-500 transition-colors" />ð
        <inputð
          type="text"ð
          placeholder="Search by name, service, or city..."ð
          value={searchTerm}ð
          onChange={(e) => setSearchTerm(e.target.value)}ð
          className="w-full pl-10 pr-4 py-3 bg-dark-800 border border-dark-600
rounded-xl text-white placeholder-dark-400 focus:outline-none focus:border-
primary-500 focus:ring-1 focus:ring-primary-500 transition-all"ð
        />ð
      </div>ð

ð
      <selectð
        value={serviceType}ð
        onChange={(e) => setServiceType(e.target.value)}ð
        className="px-4 py-3 bg-dark-800 border border-dark-600 rounded-xl
text-white focus:outline-none focus:border-primary-500 focus:ring-1 focus:ring-
primary-500 transition-all appearance-none cursor-pointer hover:bg-dark-700"ð
      >ð
        <option value="">All Services</option>ð
        <option value="plumber">Plumber</option>ð
        <option value="electrician">Electrician</option>ð
        <option value="mechanic">Mechanic</option>ð
        <option value="carpenter">Carpenter</option>ð
      </select>ð

ð
      <selectð
        value={minRating}ð
        onChange={(e) => setMinRating(e.target.value)}ð
        className="px-4 py-3 bg-dark-800 border border-dark-600 rounded-xl
text-white focus:outline-none focus:border-primary-500 focus:ring-1 focus:ring-
primary-500 transition-all appearance-none cursor-pointer hover:bg-dark-700"ð
      >ð
        <option value="">All Ratings</option>ð
        <option value="4">4+ Stars</option>ð
        <option value="3">3+ Stars</option>ð
      </select>ð

ð
      <buttonð
        onClick={() => {ð
          setServiceType('');ð
          setMinRating('');ð
          setSearchTerm('');ð
        }}ð
        className="px-4 py-3 text-dark-300 border border-dark-600 rounded-xl
hover:bg-dark-700 hover:text-white transition-colors font-medium"ð
      >ð
        Clear Filtersð
      </button>ð
    </div>ð
  </div>ð

ð

```

```

    { /* Providers List */ }
    { loading ? (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6
animate-pulse">
        {[1, 2, 3, 4, 5, 6].map(i => (
          <div key={i} className="h-80 bg-dark-800 rounded-2xl"></div>
        ))}
      </div>
    ) : filteredProviders.length === 0 ? (
      <div className="text-center py-20 bg-dark-800/30 rounded-3xl border border-
white/5">
        <Search className="h-16 w-16 text-dark-600 mx-auto mb-4" />
        <p className="text-dark-300 text-lg">No providers found matching your
criteria.</p>
        <button
          onClick={() => {
            setServiceType('');
            setMinRating('');
            setSearchTerm('');
          }}
          className="mt-4 text-primary-400 hover:text-primary-300 font-medium"
        >
          Clear filters and try again
        </button>
      </div>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6
animate-slide-up">
        {filteredProviders.map((provider) => {
          const name = provider.businessName || `${provider.userId.firstName}
${provider.userId.lastName}`;
          return (
            <Link
              key={provider._id}
              to={` /providers/${provider._id}`}
              className="group bg-dark-800/50 backdrop-blur-sm border border-
white/5 rounded-2xl p-6 hover:bg-dark-800 hover:border-primary-500/50
hover:shadow-2xl hover:shadow-primary-500/10 transition-all duration-300 hover:-
translate-y-1"
            >
              <div className="flex items-start justify-between mb-6">
                <div>
                  <h3 className="text-xl font-bold text-white group-hover:text-
primary-400 transition-colors mb-1">{name}</h3>
                  <div className="flex items-center space-x-2 text-dark-400">
                    <MapPin className="h-4 w-4" />
                    <span className="text-sm">
                      {provider.location.city}, {provider.location.state}
                    </span>
                  </div>
                </div>
                {provider.isVerified && (
                  <div className="bg-primary-500/10 p-2 rounded-lg"
title="Verified Professional">
                    <Shield className="h-5 w-5 text-primary-400" />
                  </div>
                )}
              </div>
              <div className="mb-6">
                <div className="flex flex-wrap gap-2">
                  {provider.serviceTypes.slice(0, 3).map((type) => (
                    <span
                      key={type}
                      className="px-3 py-1 bg-dark-700/50 border border-white/5
text-dark-300 text-xs font-medium rounded-lg capitalize group-hover:border-
primary-500/30 transition-colors"
                    >
                      {type}
                    </span>
                  ))}
                  {provider.serviceTypes.length > 3 && (

```

```

        <span className="px-3 py-1 bg-dark-700/50 border border-
white/5 text-dark-400 text-xs font-medium rounded-lg">ð
        +{provider.serviceTypes.length - 3} moreð
        </span>ð
    ))ð
    </div>ð
</div>ð
ð
    <div className="flex items-center justify-between pt-4 border-t
border-white/5">ð
        <div className="flex items-center space-x-1.5">ð
            <Star className="h-5 w-5 text-yellow-500 fill-yellow-500" />ð
            <span className="font-bold text-white">ð
                {provider.rating.toFixed(1)}ð
            </span>ð
            <span className="text-sm text-dark-400">ð
                ({provider.totalReviews})ð
            </span>ð
        </div>ð
ð
        <div className="flex items-center gap-3">ð
            <span className="text-lg font-bold text-white">ð
                {provider.hourlyRate}<span className="text-xs text-dark-400
font-normal">/hr</span>ð
            </span>ð
        </div>ð
    </div>ð
ð
    <div className="mt-4 flex items-center text-primary-400 text-sm
font-medium opacity-0 group-hover:opacity-100 transition-opacity transform translate-
y-2 group-hover:translate-y-0">ð
        View Profile <ArrowRight className="ml-1 h-4 w-4" />ð
    </div>ð
</Link>ð
    );ð
    ))ð
</div>ð
    ))ð
</div>ð
</div>ð
    );ð
};ð
ð
export default BrowseProvidersPage;ð
ð

```

=====
File: frontend\src\admin\ChangePasswordPage.tsx
=====

```

import React, { useState } from 'react';ð
import { useAuthStore } from '../store/authStore';ð
import { useNavigate, Navigate } from 'react-router-dom';ð
import { Key, Save, ArrowLeft, Lock } from 'lucide-react';ð
import toast from 'react-hot-toast';ð
import Navbar from '../components/Navbar';ð
ð
const ChangePasswordPage = () => {ð
    const { user, token } = useAuthStore();ð
    const navigate = useNavigate();ð
    const [loading, setLoading] = useState(false);ð
    const [formData, setFormData] = useState({ð
        currentPassword: '',ð
        newPassword: '',ð
        confirmPassword: '',ð
    });ð
ð
    if (!user) {ð
        return <Navigate to="/login" />;ð
    }ð

```

```

    const handleSubmit = async (e: React.FormEvent) => {
      e.preventDefault();

      if (formData.newPassword !== formData.confirmPassword) {
        return toast.error('Passwords do not match');
      }

      setLoading(true);

      try {
        const response = await fetch('http://localhost:5000/api/auth/change-
password', {
          method: 'PUT',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${token}`,
          },
          body: JSON.stringify({
            currentPassword: formData.currentPassword,
            newPassword: formData.newPassword,
          }),
        });

        const data = await response.json();

        if (response.ok) {
          toast.success('Password changed successfully');
          navigate('/profile');
        } else {
          toast.error(data.message || 'Change password failed');
        }
      } catch (error) {
        toast.error('Something went wrong');
      } finally {
        setLoading(false);
      }
    };

    return (
      <div className="min-h-screen bg-gray-50">
        <Navbar />
        <div className="max-w-2xl mx-auto px-4 py-12">
          <button
            onClick={() => navigate(-1)}
            className="flex items-center text-gray-600 hover:text-
primary-600 mb-8 transition-colors">
            <ArrowLeft className="h-4 w-4 mr-2" />
            Back
          </button>

          <div className="bg-white rounded-2xl shadow-sm border border-
gray-100 overflow-hidden">
            <div className="p-8">
              <h1 className="text-2xl font-bold text-gray-900 mb-6">Change
Password</h1>

              <form onSubmit={handleSubmit} className="space-y-6">
                <div>
                  <label className="block text-sm font-medium text-
gray-700 mb-1">Current Password</label>
                  <div className="relative">
                    <Lock className="absolute left-3 top-1/2 -
translate-y-1/2 h-4 w-4 text-gray-400" />
                    <input
                      type="password"
                      className="w-full pl-10 pr-4 py-2 border
border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"
                      value={formData.currentPassword}
                      onChange={(e) => setFormData({ ...formData,
currentPassword: e.target.value })}

```



```

                                requiredð
                                />ð
                            </div>ð
                        </div>ð
ð
                            <div>ð
                                <label className="block text-sm font-medium text-
gray-700 mb-1">New Password</label>ð
                                <div className="relative">ð
                                    <Key className="absolute left-3 top-1/2 -
translate-y-1/2 h-4 w-4 text-gray-400" />ð
                                    <inputð
                                        type="password"ð
                                        className="w-full pl-10 pr-4 py-2 border
border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"ð
                                        value={formData.newPassword}ð
                                        onChange={(e) => setFormData({ ...formData,
newPassword: e.target.value })}}ð
                                            requiredð
                                            minLength={6}ð
                                        />ð
                                    </div>ð
                                </div>ð
ð
                            <div>ð
                                <label className="block text-sm font-medium text-
gray-700 mb-1">Confirm New Password</label>ð
                                <div className="relative">ð
                                    <Key className="absolute left-3 top-1/2 -
translate-y-1/2 h-4 w-4 text-gray-400" />ð
                                    <inputð
                                        type="password"ð
                                        className="w-full pl-10 pr-4 py-2 border
border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"ð
                                        value={formData.confirmPassword}ð
                                        onChange={(e) => setFormData({ ...formData,
confirmPassword: e.target.value })}}ð
                                            requiredð
                                            minLength={6}ð
                                        />ð
                                    </div>ð
                                </div>ð
ð
                            <div className="flex justify-end pt-6 border-t border-
gray-50">ð
                                <buttonð
                                    type="submit"ð
                                    disabled={loading}ð
                                    className="flex items-center bg-primary-600 text-
white px-6 py-2 rounded-lg font-medium hover:bg-primary-700 focus:ring-4 focus:ring-
primary-100 transition-all disabled:opacity-50"ð
                                >ð
                                    <Save className="h-4 w-4 mr-2" />ð
                                    {loading ? 'Changing...' : 'Update Password'}ð
                                </button>ð
                            </div>ð
                        </form>ð
                    </div>ð
                </div>ð
            </div>ð
        </div>ð
    );ð
};ð
ð
export default ChangePasswordPage;ð

```

```

=====
File: frontend\src\admin\CreateBookingPage.tsx
=====

```

```

import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import Navbar from '../components/Navbar';
import api from '../api/api';
import toast from 'react-hot-toast';
import { Calendar, Clock, MapPin, AlertCircle } from 'lucide-react';

interface ServiceProvider {
  _id: string;
  businessName?: string;
  hourlyRate: number;
  serviceTypes: string[];
  userId: {
    firstName: string;
    lastName: string;
  };
};

const CreateBookingPage = () => {
  const { providerId } = useParams();
  const navigate = useNavigate();
  const [provider, setProvider] = useState<ServiceProvider | null>(null);
  const [formData, setFormData] = useState({
    serviceType: '',
    description: '',
    address: '',
    city: '',
    state: '',
    zipCode: '',
    scheduledDate: '',
    scheduledTime: '',
    estimatedHours: 2,
    isEmergency: false,
  });
  const [loading, setLoading] = useState(true);
  const [submitting, setSubmitting] = useState(false);

  useEffect(() => {
    if (providerId) {
      fetchProvider();
    }
  }, [providerId]);

  const fetchProvider = async () => {
    try {
      const response = await api.get(`/service-providers/${providerId}`);
      setProvider(response.data);
      if (response.data.serviceTypes.length > 0) {
        setFormData(prev => ({
          ...prev,
          serviceType: response.data.serviceTypes[0]
        }));
      }
    } catch (error) {
      toast.error('Failed to load provider');
      navigate('/providers');
    } finally {
      setLoading(false);
    }
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitting(true);

    try {
      // Get user's location if available
      let coordinates = { lat: 0, lng: 0 };
      if (navigator.geolocation) {
        const position = await new Promise<GeolocationPosition>((resolve, reject) =>
          navigator.geolocation.getCurrentPosition(resolve, reject));

```

```

    });
    coordinates = {
      lat: position.coords.latitude,
      lng: position.coords.longitude,
    };
  }
}

const bookingData = {
  serviceProviderId: providerId,
  ...formData,
  location: {
    ...formData,
    coordinates,
  },
  scheduledDate: new Date(formData.scheduledDate),
};

const response = await api.post('/bookings', bookingData);
toast.success('Booking created successfully!');
navigate(`/bookings/${response.data._id}`);
} catch (error: any) {
  toast.error(error.response?.data?.message || 'Failed to create booking');
} finally {
  setSubmitting(false);
}
};

if (loading) {
  return (
    <div className="min-h-screen bg-gray-50">
      <Navbar />
      <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <p className="text-center text-gray-600">Loading...</p>
      </div>
    </div>
  );
}

if (!provider) {
  return null;
}

const name = provider.businessName || `${provider.userId.firstName} ${provider.userId.lastName}`;
const totalAmount = provider.hourlyRate * formData.estimatedHours * (formData.isEmergency ? 1.5 : 1);

return (
  <div className="min-h-screen bg-gray-50">
    <Navbar />
    <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
      <h1 className="text-3xl font-bold text-gray-900 mb-6">Book Service</h1>

      <div className="bg-white rounded-lg shadow-md p-6 mb-6">
        <h2 className="text-xl font-semibold text-gray-900 mb-2">{name}</h2>
        <p className="text-gray-600 mb-4">Rate:  {provider.hourlyRate}/hour</p>
      </div>

      <form onSubmit={handleSubmit} className="bg-white rounded-lg shadow-md p-8">
        <div className="space-y-6">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-2">Service Type
          </label>
          <select
            required
            value={formData.serviceType}
            onChange={(e) => setFormData({ ...formData, serviceType:
e.target.value })}
            className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-primary-500"
          >

```

```

        <option value="">Select service type</option>
        {provider.serviceTypes.map((type) => (
          <option key={type} value={type}>
            {type.charAt(0).toUpperCase() + type.slice(1)}
          </option>
        ))}
      </select>
    </div>

    <div>
      <label className="block text-sm font-medium text-gray-700 mb-2">
        Description
      </label>
      <textarea
        required
        rows={4}
        value={formData.description}
        onChange={(e) => setFormData({ ...formData, description:
e.target.value })}
        placeholder="Describe the service you need..."
        className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
      />
    </div>

    <div className="grid grid-cols-2 gap-4">
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-2">
          <Calendar className="h-4 w-4 inline mr-1" />
          Date
        </label>
        <input
          required
          type="date"
          min={new Date().toISOString().split('T')[0]}
          value={formData.scheduledDate}
          onChange={(e) => setFormData({ ...formData, scheduledDate:
e.target.value })}
          className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
        />
      </div>
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-2">
          <Clock className="h-4 w-4 inline mr-1" />
          Time
        </label>
        <input
          required
          type="time"
          value={formData.scheduledTime}
          onChange={(e) => setFormData({ ...formData, scheduledTime:
e.target.value })}
          className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
        />
      </div>
    </div>

    <div>
      <label className="block text-sm font-medium text-gray-700 mb-2">
        <MapPin className="h-4 w-4 inline mr-1" />
        Service Address
      </label>
      <div className="grid grid-cols-1 gap-4">
        <input
          required
          type="text"
          placeholder="Street address"
          value={formData.address}
          onChange={(e) => setFormData({ ...formData, address:
e.target.value })}

```

```

        className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500">
      />
      <div className="grid grid-cols-3 gap-4">
        <input
          required
          type="text"
          placeholder="City"
          value={formData.city}
          onChange={(e) => setFormData({ ...formData, city:
e.target.value })}
          className="px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500">
        />
        <input
          required
          type="text"
          placeholder="State"
          value={formData.state}
          onChange={(e) => setFormData({ ...formData, state:
e.target.value })}
          className="px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500">
        />
        <input
          required
          type="text"
          placeholder="ZIP"
          value={formData.zipCode}
          onChange={(e) => setFormData({ ...formData, zipCode:
e.target.value })}
          className="px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500">
        />
      </div>
    </div>
  </div>
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-2">
      Estimated Hours
    </label>
    <input
      required
      type="number"
      min="0.5"
      step="0.5"
      value={formData.estimatedHours}
      onChange={(e) => setFormData({ ...formData, estimatedHours:
parseFloat(e.target.value) })}
      className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500">
    />
  </div>
  <div className="flex items-center">
    <input
      type="checkbox"
      id="emergency"
      checked={formData.isEmergency}
      onChange={(e) => setFormData({ ...formData, isEmergency:
e.target.checked })}
      className="h-4 w-4 text-primary-600 focus:ring-primary-500 border-
gray-300 rounded">
    />
    <label htmlFor="emergency" className="ml-2 flex items-center text-sm
text-gray-700">
      <AlertCircle className="h-4 w-4 mr-1 text-red-500" />
      Emergency service (50% surcharge)
    </label>
  </div>

```

```

        <div className="bg-primary-50 p-4 rounded-lg">ð
          <div className="flex justify-between items-center">ð
            <span className="text-lg font-semibold text-gray-900">Total Amount:</
span>ð
            <span className="text-2xl font-bold text-primary-600">ð
              1{totalAmount.toFixed(2)}ð
            </span>ð
          </div>ð
          {formData.isEmergency && (ð
            <p className="text-sm text-gray-600 mt-2">ð
              *Emergency service includes 50% surchargeð
            </p>ð
          )}ð
        </div>ð
      ð
      <buttonð
        type="submit"ð
        disabled={submitting}ð
        className="w-full bg-primary-600 text-white py-3 rounded-lg font-
semibold hover:bg-primary-700 transition-colors disabled:opacity-50"ð
      >ð
        {submitting ? 'Creating Booking...' : 'Create Booking'}ð
      </button>ð
    </div>ð
  </form>ð
</div>ð
</div>ð
);ð
};ð
ð
export default CreateBookingPage;ð
ð

```

```

=====
File: frontend\src\admin\CustomerDashboardPage.tsx
=====

```

```

import { useEffect, useState } from 'react';ð
import { Link } from 'react-router-dom';ð
import Navbar from '../components/Navbar';ð
import { useAuthStore } from '../store/authStore';ð
import { Wrench, Plus, Clock, Star, Calendar, ChevronRight, Zap, Droplet, Truck,
Paintbrush, Hammer, Search } from 'lucide-react';ð
import api from '../api/api';ð
ð
interface Booking {ð
  _id: string;ð
  serviceType: string;ð
  status: string;ð
  scheduledDate: string;ð
  totalAmount: number;ð
  serviceProviderId: {ð
    businessName?: string;ð
    userId: {ð
      firstName: string;ð
      lastName: string;ð
    };ð
  };ð
};ð
}ð
ð
const CustomerDashboardPage = () => {ð
  const { user } = useAuthStore();ð
  const [bookings, setBookings] = useState<Booking[]>([]);ð
  const [loading, setLoading] = useState(true);ð
  ð
  useEffect(() => {ð
    if (user) {ð
      fetchBookings();ð
    }ð
  }, [user]);ð

```

```

    }
    const fetchBookings = async () => {
      try {
        const response = await api.get('/bookings/my-bookings');
        setBookings(response.data.slice(0, 5));
      } catch (error) {
        console.error('Failed to load bookings', error);
        // toast.error('Failed to load bookings');
      } finally {
        setLoading(false);
      }
    };
  }

  const categories = [
    { name: 'Plumber', icon: Droplet, color: 'text-blue-400', bg: 'bg-blue-500/20' },
    { name: 'Electrician', icon: Zap, color: 'text-yellow-400', bg: 'bg-yellow-500/20' },
    { name: 'Mechanic', icon: Wrench, color: 'text-red-400', bg: 'bg-red-500/20' },
    { name: 'Carpenter', icon: Hammer, color: 'text-orange-400', bg: 'bg-orange-500/20' },
    { name: 'Painter', icon: Paintbrush, color: 'text-purple-400', bg: 'bg-purple-500/20' },
    { name: 'Movers', icon: Truck, color: 'text-green-400', bg: 'bg-green-500/20' },
  ];

  return (
    <div className="min-h-screen bg-dark-900 text-white relative overflow-hidden">
      <div className="absolute top-0 right-0 w-[500px] h-[500px] bg-primary-600/10 rounded-full blur-[100px] pointer-events-none" />
      <Navbar />
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12 relative">
        <div className="flex flex-col md:flex-row justify-between items-end mb-12 animate-fade-in">
          <div>
            <h1 className="text-4xl font-bold font-heading mb-2">
              Hello, <span className="text-gradient">{user?.firstName}</span>
            </h1>
            <p className="text-dark-300">Ready to find the perfect professional?</p>
          </div>
          <Link
            to="/providers"
            className="mt-4 md:mt-0 px-6 py-3 bg-primary-600 hover:bg-primary-500 text-white rounded-xl font-bold transition-all shadow-lg shadow-primary-500/20 flex items-center">
            <Search className="h-5 w-5 mr-2" />
            Browse All Providers
          </Link>
        </div>
        <div>
          <div className="mb-12 animate-slide-up">
            <h2 className="text-xl font-bold font-heading mb-6 flex items-center">
              <Wrench className="h-5 w-5 mr-2 text-primary-400" />
              Quick Services
            </h2>
            <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-4">
              {categories.map((cat, index) => (
                <Link
                  key={index}
                  to={`\providers?category=${cat.name.toLowerCase()}`}
                  className="glass-card p-6 rounded-2xl border-white/5 hover:border-primary-500/30 hover:bg-dark-800 transition-all duration-300 group text-center flex flex-col items-center">
                    <div className={`${p-4 rounded-full mb-3 ${cat.bg} group-hover:scale-110 transition-transform`} >
                      <cat.icon className={`${h-6 w-6 ${cat.color}}` />
                    </div>
                    <span className="font-semibold text-dark-200 group-hover:text-white"

```

```

transition-colors">{cat.name}</span>ð
    </Link>ð
  )}ð
</div>ð
</div>ð
ð
  <div className="grid grid-cols-1 lg:grid-cols-3 gap-8 mb-12">ð
    { /* Active Bookings Section */ }ð
    <div className="lg:col-span-2 glass-card rounded-2xl border-white/5 p-8
animate-fade-in delay-100">ð
      <div className="flex items-center justify-between mb-8">ð
        <h2 className="text-2xl font-bold font-heading flex items-center">ð
          <Calendar className="h-6 w-6 mr-2 text-primary-400" />ð
          Active Bookingsð
        </h2>ð
        <Link to="/my-bookings" className="text-sm text-primary-400 hover:text-
primary-300 font-medium hover:underline">ð
          View Historyð
        </Link>ð
      </div>ð
ð
    {loading ? (ð
      <div className="animate-pulse space-y-4">ð
        {[1, 2].map(i => (ð
          <div key={i} className="h-24 bg-dark-800 rounded-xl"></div>ð
        )))}ð
      </div>ð
    ) : bookings.length === 0 ? (ð
      <div className="text-center py-12 bg-dark-800/50 rounded-xl border
border-white/5 border-dashed">ð
        <Calendar className="h-12 w-12 text-dark-500 mx-auto mb-4" />ð
        <p className="text-dark-300 mb-6">No active bookings</p>ð
        <Linkð
          to="/providers"ð
          className="inline-flex items-center text-primary-400 hover:text-
primary-300 font-bold">ð
          >ð
          <Plus className="h-4 w-4 mr-1" />ð
          Book a Serviceð
        </Link>ð
      </div>ð
    ) : (ð
      <div className="space-y-4">ð
        {bookings.map((booking) => (ð
          <Linkð
            key={booking._id}ð
            to={`/${bookings/${booking._id}}`ð
            className="flex justify-between items-center p-5 bg-dark-800/50
border border-white/5 rounded-xl hover:border-primary-500/30 hover:bg-dark-800
transition-all group">ð
            >ð
              <div className="flex items-center gap-4">ð
                <div className="h-12 w-12 rounded-full bg-dark-700 flex items-
center justify-center text-dark-300 font-bold border border-white/5">ð
                  {booking.serviceType.charAt(0)}ð
                </div>ð
                <div>ð
                  <h3 className="font-bold text-white capitalize group-
hover:text-primary-400 transition-colors">ð
                    {booking.serviceType}ð
                  </h3>ð
                  <p className="text-sm text-dark-400">ð
                    with {booking.serviceProviderId?.businessName ||ð
                      `${booking.serviceProviderId?.userId?.firstName}ð
${booking.serviceProviderId?.userId?.lastName}`}ð
                  </p>ð
                </div>ð
              </div>ð
            </div>ð
          </div>ð
        </div>ð
      </div>ð
    ) : (ð
      <div className="text-right">ð
        <span className={`inline-block px-3 py-1 text-xs font-bold
rounded-full mb-1 ${booking.status === 'completed' ? 'bg-green-500/20 text-
green-400' :ð

```



```

const HomePage = () => {
  const services = [
    { icon: Wrench, name: 'Plumbing', desc: 'Expert pipe repairs & installation', color: 'from-blue-500 to-cyan-400' },
    { icon: Zap, name: 'Electrical', desc: 'Certified electricians', color: 'from-yellow-500 to-amber-400' },
    { icon: Car, name: 'Mechanic', desc: 'Auto maintenance & diagnostics', color: 'from-green-500 to-emerald-400' },
    { icon: Hammer, name: 'Carpentry', desc: 'Custom woodwork & repairs', color: 'from-orange-500 to-red-400' }
  ];

  const features = [
    {
      icon: Shield,
      title: 'Verified Professionals',
      description: 'Strict background checks ensure your safety and peace of mind.'
    },
    {
      icon: Clock,
      title: 'Instant Booking',
      description: 'Book services in as little as 60 seconds with our optimized flow.'
    },
    {
      icon: Star,
      title: 'Premium Quality',
      description: 'Top-rated providers delivering excellence in every job.'
    },
    {
      icon: MessageCircle,
      title: '24/7 Support',
      description: 'Our dedicated team is always here to help you anytime.'
    }
  ];

  return (
    <div className="min-h-screen bg-dark-900 text-white overflow-hidden">
      <Navbar />

      {/* Hero Section */}
      <section className="relative pt-32 pb-20 px-4 sm:px-6 lg:px-8">
        <div className="absolute top-0 left-1/2 -translate-x-1/2 w-full h-[500px] bg-gradient-to-b from-primary-500/20 to-transparent blur-3xl -z-10 pointer-events-none" />

        <div className="max-w-7xl mx-auto text-center animate-fade-in">
          <div className="inline-flex items-center px-4 py-2 rounded-full glass-card border-primary-500/30 mb-8 animate-slide-up">
            <span className="flex h-2 w-2 rounded-full bg-primary-400 mr-2 animate-pulse"></span>
            <span className="text-sm font-medium text-primary-300">#1 Trusted Service Marketplace</span>
          </div>

          <h1 className="text-6xl md:text-7xl font-bold tracking-tight mb-8 font-heading">
            Mastery at Your <br />
            <span className="text-gradient">Fingertips</span>
          </h1>

          <p className="text-xl text-dark-300 mb-10 max-w-2xl mx-auto leading-relaxed">
            Experience the new standard in home services. From quick fixes to major renovations, connect with elite professionals instantly.
          </p>

          <div className="flex flex-col sm:flex-row justify-center items-center gap-4">
            <Link
              to="/register"
              className="group bg-gradient-to-r from-primary-600 to-primary-500 text-

```

```

white px-8 py-4 rounded-xl font-bold shadow-lg shadow-primary-500/25 hover:shadow-
primary-500/40 hover:scale-105 transition-all duration-300 flex items-center"ð
>ð
  Get Startedð
  <ArrowRight className="ml-2 h-5 w-5 group-hover:translate-x-1
transition-transform" />ð
</Link>ð
<Linkð
  to="/providers"ð
  className="px-8 py-4 rounded-xl font-bold text-white glass-card
hover:bg-white/10 transition-all duration-300"ð
>ð
  Browse Servicesð
</Link>ð
</div>ð
</div>ð
</section>ð
ð
  { /* Services Section */ }ð
  <section className="py-20 relative">ð
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">ð
      <div className="text-center mb-16">ð
        <h2 className="text-3xl md:text-4xl font-bold font-heading mb-4">Elite
Services</h2>ð
        <p className="text-dark-400">Curated for excellence, delivered with
precision.</p>ð
      </div>ð
    </div>ð
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">ð
      {services.map((service) => {ð
        const Icon = service.icon;ð
        return (ð
          <Linkð
            key={service.name}ð
            to="/providers"ð
            className="group glass-card p-8 rounded-2xl hover:border-
primary-500/50 transition-all duration-300 hover:transform hover:-translate-y-1"ð
          >ð
            <div className={`w-14 h-14 rounded-xl bg-gradient-to-br
${service.color} p-0.5 mb-6 group-hover:shadow-lg group-hover:shadow-primary-500/20
transition-all`} >ð
              <div className="w-full h-full bg-dark-900 rounded-[10px] flex
items-center justify-center">ð
                <Icon className="h-7 w-7 text-white" />ð
              </div>ð
            </div>ð
            <h3 className="text-xl font-bold mb-2 group-hover:text-primary-400
transition-colors">{service.name}</h3>ð
            <p className="text-sm text-dark-400">{service.desc}</p>ð
          </Link>ð
        );ð
      })}ð
    </div>ð
  </div>ð
</section>ð
ð
  { /* Features Section */ }ð
  <section className="py-20 bg-dark-800/50">ð
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">ð
      <div className="grid lg:grid-cols-2 gap-16 items-center">ð
        <div>ð
          <h2 className="text-4xl font-bold font-heading mb-6">ð
            Why Top Tier Clients <br />ð
            <span className="text-gradient">Choose Us</span>ð
          </h2>ð
          <p className="text-dark-300 mb-8 text-lg">ð
            We don't just connect you; we ensure a seamless, premium experience
from start to finish.ð
          </p>ð
        </div>ð
        <div className="space-y-6">ð
          {features.map((feature) => (ð

```

```

        <div key={feature.title} className="flex gap-4">
          <div className="flex-shrink-0 mt-1">
            <CheckCircle className="h-6 w-6 text-primary-500" />
          </div>
          <div>
            <h3 className="font-bold text-white mb-1">{feature.title}</h3>
            <p className="text-dark-400 text-sm">{feature.description}</p>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div className="relative">
    <div className="absolute inset-0 bg-gradient-to-r from-primary-500 to-secondary-500 rounded-3xl blur-2xl opacity-20 transform rotate-3"></div>
    <div className="glass-card p-8 rounded-3xl relative border-none bg-dark-800">
      <div className="grid gap-6">
        { /* Mock UI Elements for Visual Interest */ }
        <div className="flex items-center gap-4 p-4 bg-dark-900/50 rounded-xl">
          <div className="h-10 w-10 rounded-full bg-primary-500/20 flex items-center justify-center text-primary-400">
            <Star className="h-5 w-5 fill-current" />
          </div>
          <div>
            <div className="h-2 w-24 bg-dark-700 rounded mb-2"></div>
            <div className="h-2 w-16 bg-dark-700 rounded"></div>
          </div>
          <div className="ml-auto text-primary-400 font-bold">5.0</div>
        </div>
        <div className="flex items-center gap-4 p-4 bg-dark-900/50 rounded-xl opacity-75">
          <div className="h-10 w-10 rounded-full bg-secondary-500/20 flex items-center justify-center text-secondary-400">
            <Shield className="h-5 w-5" />
          </div>
          <div>
            <div className="h-2 w-32 bg-dark-700 rounded mb-2"></div>
            <div className="h-2 w-20 bg-dark-700 rounded"></div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>

{ /* CTA Section */ }
<section className="py-24 px-4">
  <div className="max-w-5xl mx-auto relative">
    <div className="absolute inset-0 bg-gradient-to-r from-primary-600 to-secondary-600 rounded-3xl blur-xl opacity-30"></div>
    <div className="relative glass-card rounded-3xl p-12 text-center border-primary-500/20 overflow-hidden">
      <div className="absolute top-0 right-0 p-12 opacity-10 transform translate-x-1/2 -translate-y-1/2">
        <Zap className="h-64 w-64 text-white" />
      </div>
      <h2 className="text-4xl md:text-5xl font-bold mb-6 font-heading">Ready to Upgrade?</h2>
      <p className="text-xl text-dark-200 mb-10 max-w-2xl mx-auto">
        Join the exclusive network of top-rated professionals and discerning clients.
      </p>
      <div className="flex flex-col sm:flex-row justify-center gap-4">
        <Link
          to="/register"
          className="bg-white text-dark-900 px-10 py-4 rounded-xl font-bold

```

```

hover:bg-gray-100 transition-colors shadow-lg"ð
    >ð
        Join Nowð
    </Link>ð
</div>ð
</div>ð
</div>ð
</section>ð
ð
    <Chatbot />ð
</div>ð
);ð
};ð
ð
export default HomePage;ð
ð

```

```

=====
File: frontend\src\admin\LoginPage.tsx
=====

```

```

import { useState } from 'react';ð
import { Link, useNavigate } from 'react-router-dom';ð
import { useAuthStore } from '../store/authStore';ð
import api from '../api/api';ð
import toast from 'react-hot-toast';ð
import Navbar from '../components/Navbar';ð
import { Logo } from '../components/Logo';ð
import { ArrowLeft, Mail, Lock, Eye, EyeOff, ShieldCheck, User } from 'lucide-react';ð
ð
const LoginPage = () => {ð
    const [formData, setFormData] = useState({ð
        identifier: '',ð
        password: '',ð
    });ð
    const [loading, setLoading] = useState(false);ð
    const [showPassword, setShowPassword] = useState(false);ð
    const [isAdmin, setIsAdmin] = useState(false);ð
    const { setAuth } = useAuthStore();ð
    const navigate = useNavigate();ð
ð
    const handleSubmit = async (e: React.FormEvent) => {ð
        e.preventDefault();ð
ð
        // Validation for identifier (Email or 10-digit phone)ð
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;ð
        const phoneRegex = /^[0-9]{10}$/;ð
ð
        if (!emailRegex.test(formData.identifier) && !
phoneRegex.test(formData.identifier)) {ð
            toast.error('Please enter a valid email or 10-digit phone number');ð
            return;ð
        }ð
ð
        setLoading(true);ð
ð
        try {ð
            const response = await api.post('/auth/login', formData);ð
            const user = response.data.user;ð
            ð
            // Verification if trying to login as admin but user doesn't have the roleð
            if (isAdmin && user.role !== 'admin') {ð
                toast.error('This account does not have admin privileges');ð
                setLoading(false);ð
                return;ð
            }ð
ð
            setAuth(user, response.data.token);ð
            ð
            let roleName = 'User';ð

```

```

        if (user.role === 'admin') roleName = 'Administrator';
        else if (user.role === 'service_provider') roleName = 'Service Provider';
        else roleName = 'Customer';
    }
    toast.success(`Login successful! Signed in as ${roleName}`);
    if (user.role === 'admin') {
        navigate('/admin/dashboard');
    } else if (user.role === 'service_provider') {
        navigate('/provider/dashboard');
    } else {
        navigate('/dashboard');
    }
    catch (error: any) {
        toast.error(error.response?.data?.message || 'Login failed');
    } finally {
        setLoading(false);
    }
};
}
return (
    <div className="min-h-screen bg-dark-900 text-white relative overflow-hidden">
        { /* Dynamic Background */ }
        <div className={`absolute inset-0 bg-cover bg-center opacity-10 pointer-events-none transition-all duration-700 ${
            isAdmin ? "bg-[url('https://images.unsplash.com/photo-1550751827-4bd374c3f58b?ixlib=rb-1.2.1&auto=format&fit=crop&w=1920&q=80')]" : "bg-[url('https://images.unsplash.com/photo-1581578731117-104f2a8d23e9?ixlib=rb-1.2.1&auto=format&fit=crop&w=1920&q=80')]"
        }` />
        <div className="absolute top-0 left-0 w-full h-full pointer-events-none overflow-hidden">
            <div className={`absolute -top-1/4 -right-1/4 w-1/2 h-1/2 rounded-full blur-[120px] transition-colors duration-700 ${isAdmin ? 'bg-secondary-500/20' : 'bg-primary-500/20'}` />
            <div className={`absolute -bottom-1/4 -left-1/4 w-1/2 h-1/2 rounded-full blur-[120px] transition-colors duration-700 ${isAdmin ? 'bg-indigo-500/10' : 'bg-amber-500/10'}` />
        </div>
        <Navbar />
        <div className="relative flex items-center justify-center min-h-[calc(100vh-80px)] px-4 sm:px-6 lg:px-8 py-12">
            <div className={`max-w-md w-full glass-card p-8 rounded-2xl animate-fade-in border-t-2 transition-all duration-500 ${isAdmin ? 'border-t-secondary-500 shadow-secondary-500/10' : 'border-t-primary-500 shadow-primary-500/10'}` />
                <div className="text-center mb-6">
                    <Link to="/" className="inline-flex items-center text-dark-400 hover:text-white mb-6 transition-colors group">
                        <ArrowLeft className="h-4 w-4 mr-2 transform group-hover:-translate-x-1 transition-transform" />
                        Back to Home
                    </Link>
                </div>
                <div className="flex justify-center mb-4">
                    <div className={`p-4 rounded-full transition-all duration-500 transform hover:scale-110 ${isAdmin ? 'bg-secondary-500/20 text-secondary-400' : 'bg-primary-500/20 text-primary-500'}` />
                        {isAdmin ? <ShieldCheck className="h-10 w-10" /> : <Logo className="h-10 w-10" />}
                    </div>
                </div>
                <h2 className={`text-3xl font-bold font-heading mb-2 transition-colors duration-500 ${isAdmin ? 'text-secondary-400' : 'text-white'}` />
                    {isAdmin ? 'Admin Console' : 'Welcome Back'}
                </h2>
                <p className="text-dark-300">
                    {isAdmin ? 'Access the management dashboard' : 'Sign in to manage your services'}
                </p>
            </div>
        </div>
    </div>
);

```

```

    </p>ð
  </div>ð
ð
    { /* Role Toggle */ }ð
    <div className="flex p-1 bg-dark-800/50 rounded-xl mb-8 border border-
dark-600/50 relative overflow-hidden">ð
      <div ð
        className={ `absolute top-1 bottom-1 w-[calc(50%-4px)] rounded-lg
transition-all duration-300 ease-out ${ð
          isAdmin ? 'left-[calc(50%+2px)] bg-secondary-600 shadow-lg shadow-
secondary-900/40' : 'left-1 bg-primary-600 shadow-lg shadow-primary-900/40'ð
        } ` }ð
      </div>ð
      <buttonð
        type="button"ð
        onClick={() => setIsAdmin(false)}ð
        className={ `flex-1 flex items-center justify-center py-2.5 px-4
rounded-lg text-sm font-semibold transition-colors duration-300 relative z-10 ${ð
          !isAdmin ? 'text-white' : 'text-dark-400 hover:text-dark-200'ð
        } ` }ð
      >ð
        <User className={ `h-4 w-4 mr-2 transition-transform duration-300 ${!
isAdmin ? 'scale-110' : 'scale-100' } ` } />ð
        Userð
      </button>ð
      <buttonð
        type="button"ð
        onClick={() => setIsAdmin(true)}ð
        className={ `flex-1 flex items-center justify-center py-2.5 px-4
rounded-lg text-sm font-semibold transition-colors duration-300 relative z-10 ${ð
          isAdmin ? 'text-white' : 'text-dark-400 hover:text-dark-200'ð
        } ` }ð
      >ð
        <ShieldCheck className={ `h-4 w-4 mr-2 transition-transform
duration-300 ${isAdmin ? 'scale-110' : 'scale-100' } ` } />ð
        Adminð
      </button>ð
    </div>ð
ð
    <form className="space-y-6" onSubmit={handleSubmit}>ð
      <div className="space-y-4">ð
        <div>ð
          <label htmlFor="identifier" className="block text-sm font-medium
text-dark-200 mb-1.5 ml-1">ð
            Email or Phone Numberð
          </label>ð
          <div className="relative group">ð
            <div className="absolute inset-y-0 left-0 pl-3.5 flex items-center
pointer-events-none">ð
              <Mail className={ `h-4.5 w-4.5 transition-colors duration-300
${isAdmin ? 'text-secondary-500' : 'text-dark-400 group-focus-within:text-
primary-500' } ` } />ð
            </div>ð
            <inputð
              id="identifier"ð
              name="identifier"ð
              type="text"ð
              requiredð
              value={formData.identifier}ð
              onChange={(e) => setFormData({ ...formData, identifier:
e.target.value }) }ð
              className={ `block w-full pl-11 pr-3 py-3.5 border rounded-xl
leading-5 bg-dark-800/80 text-white placeholder-dark-500 focus:outline-none
focus:ring-2 transition-all duration-300 ${ð
                isAdmin ð
                  ? 'border-dark-600 focus:border-secondary-500 focus:ring-
secondary-500/20' ð
                  : 'border-dark-600 focus:border-primary-500 focus:ring-
primary-500/20'ð
              } ` }ð
              placeholder="Enter email or phone"ð
            </div>ð
          </div>ð
        </div>ð
      </div>ð
    </form>ð
  </div>ð
ð

```

```

        </div>ð
    </div>ð
ð
    <div>ð
        <label htmlFor="password" className="block text-sm font-medium text-
dark-200 mb-1.5 ml-1">ð
            Passwordð
        </label>ð
        <div className="relative group">ð
            <div className="absolute inset-y-0 left-0 pl-3.5 flex items-center
pointer-events-none">ð
                <Lock className={`h-4.5 w-4.5 transition-colors duration-300
${isAdmin ? 'text-secondary-500' : 'text-dark-400 group-focus-within:text-
primary-500'}`}>ð
            </div>ð
            <inputð
                id="password"ð
                name="password"ð
                type={showPassword ? 'text' : 'password'}ð
                requiredð
                value={formData.password}ð
                onChange={(e) => setFormData({ ...formData, password:
e.target.value })}ð
                className={`block w-full pl-11 pr-11 py-3.5 border rounded-xl
leading-5 bg-dark-800/80 text-white placeholder-dark-500 focus:outline-none
focus:ring-2 transition-all duration-300 ${ð
                    isAdmin ð
                        ? 'border-dark-600 focus:border-secondary-500 focus:ring-
secondary-500/20' ð
                        : 'border-dark-600 focus:border-primary-500 focus:ring-
primary-500/20'ð
                    }`}>ð
                placeholder="Enter your password"ð
            </div>ð
            <buttonð
                type="button"ð
                onClick={() => setShowPassword(!showPassword)}ð
                className="absolute inset-y-0 right-0 pr-3.5 flex items-center
text-dark-400 hover:text-white transition-colors">ð
                >ð
                {showPassword ? (ð
                    <EyeOff className="h-4.5 w-4.5" />ð
                ) : (ð
                    <Eye className="h-4.5 w-4.5" />ð
                )}ð
            </button>ð
        </div>ð
    </div>ð
</div>ð
ð
    <div className="flex items-center justify-end">ð
        <a href="#" className={`text-xs font-semibold transition-colors
duration-300 ${isAdmin ? 'text-secondary-400 hover:text-secondary-300' : 'text-
primary-400 hover:text-primary-300'}`}>ð
            Forgot password?ð
        </a>ð
    </div>ð
ð
    <buttonð
        type="submit"ð
        disabled={loading}ð
        className={`w-full flex justify-center py-3.5 px-4 rounded-xl shadow-
xl text-sm font-bold text-white transition-all transform active:scale-[0.98]
disabled:opacity-50 disabled:cursor-not-allowed ${ð
            isAdmin ð
                ? 'bg-gradient-to-r from-secondary-600 to-indigo-600 hover:from-
secondary-500 hover:to-indigo-500 shadow-secondary-500/20' ð
                : 'bg-gradient-to-r from-primary-600 to-amber-500 hover:from-
primary-500 hover:to-amber-400 shadow-primary-500/20'ð
            }`}>ð
        >ð
        {loading ? (ð

```



```

        <span className="flex items-center">ð
            <svg className="animate-spin -ml-1 mr-3 h-5 w-5 text-white"
xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">ð
                <circle className="opacity-25" cx="12" cy="12" r="10"
stroke="currentColor" strokeWidth="4"></circle>ð
                <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0
018-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3
7.938l3-2.647z"></path>ð
            </svg>ð
            Processing...ð
        </span>ð
    ) : (ð
        isAdmin ? 'Enter Admin Dashboard' : 'Sign in to Account'ð
    ))ð
</button>ð
ð
    {!isAdmin && (ð
        <div className="text-center mt-6">ð
            <p className="text-sm text-dark-400 font-medium">ð
                New to Service Connect?{' '}ð
                <Link to="/register" className="text-primary-400 hover:text-
primary-300 font-bold ml-1 transition-colors">ð
                    Join nowð
                </Link>ð
            </p>ð
        </div>ð
    ))ð
</form>ð
</div>ð
</div>ð
</div>ð
);ð
};ð
ð
export default LoginPage;ð
ð

```

```

=====
File: frontend\src\admin\MyBookingsPage.tsx
=====

```

```

import { useEffect, useState } from 'react';ð
import { Link } from 'react-router-dom';ð
import Navbar from '../components/Navbar';ð
import api from '../api/api';ð
import toast from 'react-hot-toast';ð
import { Calendar, Clock, MapPin, ArrowRight } from 'lucide-react';ð
import { useAuthStore } from '../store/authStore';ð
ð
interface Booking {ð
    _id: string;ð
    serviceType: string;ð
    status: string;ð
    scheduledDate: string;ð
    scheduledTime: string;ð
    totalAmount: number;ð
    location: {ð
        city: string;ð
        state: string;ð
    };ð
    customerId?: {ð
        firstName: string;ð
        lastName: string;ð
    };ð
    serviceProviderId: {ð
        businessName?: string;ð
        userId: {ð
            firstName: string;ð
            lastName: string;ð
        };ð
    };ð
}

```

```

    };
  }
}
const MyBookingsPage = () => {
  const { user } = useAuthStore();
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [filter, setFilter] = useState('all');

  useEffect(() => {
    fetchBookings();
  }, []);

  const fetchBookings = async () => {
    try {
      const response = await api.get('/bookings/my-bookings');
      setBookings(response.data);
    } catch (error) {
      toast.error('Failed to load bookings');
    } finally {
      setLoading(false);
    }
  };

  const filteredBookings = filter === 'all'
    ? bookings
    : bookings.filter(b => b.status === filter);

  return (
    <div className="min-h-screen bg-gray-50">
      <Navbar />
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <div className="flex justify-between items-center mb-6">
          <h1 className="text-3xl font-bold text-gray-900">My Bookings</h1>
          {user?.role === 'customer' && (
            <Link
              to="/providers"
              className="bg-primary-600 text-white px-6 py-2 rounded-lg font-semibold hover:bg-primary-700 transition-colors"
            >
              Find Providers
            </Link>
          )}
        </div>
        { /* Filters */ }
        <div className="bg-white rounded-lg shadow-md p-4 mb-6">
          <div className="flex space-x-2">
            {['all', 'pending', 'confirmed', 'in_progress', 'completed', 'cancelled'].map((status) => (
              <button
                key={status}
                onClick={() => setFilter(status)}
                className={`px-4 py-2 rounded-lg font-medium transition-colors ${
                  filter === status
                    ? 'bg-primary-600 text-white'
                    : 'bg-gray-100 text-gray-700 hover:bg-gray-200'
                }`}
              >
                {status === 'all' ? 'All' : status.replace('_', ' ').charAt(0).toUpperCase() + status.replace('_', ' ').slice(1)}
              </button>
            ))}
          </div>
          <div>
            {loading ? (
              <div className="text-center py-12">
                <p className="text-gray-600">Loading bookings...</p>
              </div>
            ) : filteredBookings.length === 0 ? (
              <div className="text-center py-12 bg-white rounded-lg shadow-md">

```

```

<p className="text-gray-600 mb-4">No bookings found</p>
{user?.role === 'customer' && (
  <Link
    to="/providers"
    className="text-primary-600 hover:text-primary-700 font-semibold"
  >
    Browse Service Providers
  </Link>
)}
</div>
) : (
  <div className="space-y-4">
    {filteredBookings.map((booking) => {
      const providerName = booking.serviceProviderId?.businessName ||
        `${booking.serviceProviderId?.userId?.firstName}
        ${booking.serviceProviderId?.userId?.lastName}`;
      const customerName = booking.customerId
        ? `${booking.customerId.firstName} ${booking.customerId.lastName}`
        : '';
      return (
        <Link
          key={booking._id}
          to={`/${bookings}/${booking._id}`}
          className="bg-white rounded-lg shadow-md hover:shadow-lg
          transition-shadow p-6 block"
        >
          <div className="flex justify-between items-start">
            <div className="flex-1">
              <div className="flex items-center space-x-3 mb-3">
                <h3 className="text-xl font-semibold text-gray-900
                capitalize">
                  {booking.serviceType}
                </h3>
                <span className={`px-3 py-1 rounded-full text-sm font-
                semibold ${
                  booking.status === 'completed' ? 'bg-green-100 text-
                  green-800' :
                  booking.status === 'confirmed' ? 'bg-blue-100 text-
                  blue-800' :
                  booking.status === 'in_progress' ? 'bg-yellow-100 text-
                  yellow-800' :
                  booking.status === 'cancelled' ? 'bg-red-100 text-
                  red-800' :
                  'bg-gray-100 text-gray-800'
                }`}>
                  {booking.status.replace('_', ' ')}
                </span>
              </div>
            </div>
            <div className="grid grid-cols-1 md:grid-cols-3 gap-4 text-sm
            text-gray-600">
              <div className="flex items-center space-x-2">
                <Calendar className="h-4 w-4" />
                <span>{new
                Date(booking.scheduledDate).toLocaleDateString()}</span>
              </div>
              <div className="flex items-center space-x-2">
                <Clock className="h-4 w-4" />
                <span>{booking.scheduledTime}</span>
              </div>
              <div className="flex items-center space-x-2">
                <MapPin className="h-4 w-4" />
                <span>{booking.location.city}, {booking.location.state}</
                span>
              </div>
            </div>
          </div>
          <p className="text-sm text-gray-600 mt-3">
            {user?.role === 'customer'
              ? `Provider: ${providerName}`
              : `Customer: ${customerName}`}
          </p>
        </Link>
      )
    })}
  </div>
)

```

```

        </p>
    </div>
    <div className="text-right ml-4">
        <p className="text-2xl font-bold text-primary-600">
            {booking.totalAmount.toFixed(2)}
        </p>
        <ArrowRight className="h-5 w-5 text-gray-400 mt-2 ml-auto" />
    </div>
</div>
</Link>
    );
    })}
</div>
    );
</div>
</div>
    );
};
export default MyBookingsPage;

```

```

=====
File: frontend\src\admin\ProfilePage.tsx
=====

```

```

import React, { useState } from 'react';
import { useAuthStore } from '../store/authStore';
import { useNavigate, Navigate } from 'react-router-dom';
import { User, Phone, Mail, Camera, Save, ArrowLeft } from 'lucide-react';
import toast from 'react-hot-toast';
import Navbar from '../components/Navbar';

const ProfilePage = () => {
    const { user, updateUser, token } = useAuthStore();
    const navigate = useNavigate();
    const [loading, setLoading] = useState(false);
    const [formData, setFormData] = useState({
        firstName: user?.firstName || '',
        lastName: user?.lastName || '',
        email: user?.email || '',
        phone: user?.phone || '',
        profilePhoto: user?.profilePhoto || ''
    });

    if (!user) {
        return <Navigate to="/login" />;
    }

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);

        try {
            const response = await fetch('http://localhost:5000/api/auth/profile', {
                method: 'PUT',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': `Bearer ${token}`
                },
                body: JSON.stringify(formData)
            });

            const data = await response.json();

            if (response.ok) {
                updateUser(data.user);
                toast.success('Profile updated successfully');
            } else {

```

```

        toast.error(data.message || 'Update failed');ð
    }ð
    } catch (error) {ð
        toast.error('Something went wrong');ð
    } finally {ð
        setLoading(false);ð
    }ð
};ð
ð
return (ð
    <div className="min-h-screen bg-gray-50">ð
        <Navbar />ð
        <div className="max-w-3xl mx-auto px-4 py-12">ð
            <buttonð
                onClick={() => navigate(-1)}ð
                className="flex items-center text-gray-600 hover:text-
primary-600 mb-8 transition-colors"ð
            >ð
                <ArrowLeft className="h-4 w-4 mr-2" />ð
                Backð
            </button>ð
ð
            <div className="bg-white rounded-2xl shadow-sm border border-
gray-100 overflow-hidden">ð
                <div className="p-8">ð
                    <h1 className="text-2xl font-bold text-gray-900
mb-6">Profile Settings</h1>ð
ð
                    <form onSubmit={handleSubmit} className="space-y-6">ð
                        /* Profile Photo Section */ð
                        <div className="flex flex-col items-center sm:flex-row
sm:items-center space-y-4 sm:space-x-6 pb-6 border-b border-gray-50">ð
                            <div className="relative group">ð
                                {formData.profilePhoto ? (ð
                                    <imgð
                                        src={formData.profilePhoto}ð
                                        alt="Profile"ð
                                        className="h-24 w-24 rounded-full object-
cover border-4 border-white shadow-md"ð
                                    >ð
                                ) : (ð
                                    <div className="h-24 w-24 rounded-full bg-
primary-100 flex items-center justify-center border-4 border-white shadow-md">ð
                                        <User className="h-10 w-10 text-
primary-600" />ð
                                    </div>ð
                                )}ð
                            <div className="absolute inset-0 rounded-full bg-
black bg-opacity-40 flex items-center justify-center opacity-0 group-
hover:opacity-100 transition-opacity cursor-pointer">ð
                                <Camera className="h-6 w-6 text-white" />ð
                            </div>ð
                        <div className="flex-1 text-center sm:text-left">ð
                            <h3 className="text-sm font-medium text-
gray-900">Profile Photo</h3>ð
                            <p className="text-xs text-gray-500 mt-1
mb-3">JPG, GIF or PNG. Max size of 800K</p>ð
                            <inputð
                                type="text"ð
                                placeholder="Paste image URL here"ð
                                className="w-full px-3 py-2 text-sm border
border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 focus:border-
transparent transition-all outline-none"ð
                                value={formData.profilePhoto}ð
                                onChange={(e) => setFormData({ ...formData,
profilePhoto: e.target.value })}ð
                            >ð
                        </div>ð
                    </div>ð
                </div>ð
            </div>ð
ð
            /* Form Fields */ð

```

```

        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">First Name</label>
                <div className="relative">
                    <User className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-400" />
                    <input
                        type="text"
                        className="w-full pl-10 pr-4 py-2 border border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"
                        value={formData.firstName}
                        onChange={(e) => setFormData({ ...formData, firstName: e.target.value })}
                        required
                    />
                </div>
            </div>
            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Last Name</label>
                <div className="relative">
                    <User className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-400" />
                    <input
                        type="text"
                        className="w-full pl-10 pr-4 py-2 border border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"
                        value={formData.lastName}
                        onChange={(e) => setFormData({ ...formData, lastName: e.target.value })}
                        required
                    />
                </div>
            </div>
            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Email Address</label>
                <div className="relative">
                    <Mail className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-400" />
                    <input
                        type="email"
                        className="w-full pl-10 pr-4 py-2 border border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none bg-gray-50"
                        value={formData.email}
                        disabled
                    />
                </div>
            </div>
            <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Phone Number</label>
                <div className="relative">
                    <Phone className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-gray-400" />
                    <input
                        type="text"
                        className="w-full pl-10 pr-4 py-2 border border-gray-200 rounded-lg focus:ring-2 focus:ring-primary-500 outline-none"
                        value={formData.phone}
                        onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
                        required
                    />
                </div>
            </div>
        </div>

```

```

    <div className="flex justify-end pt-6 border-t border-
gray-50">
        <button
            type="submit"
            disabled={loading}
            className="flex items-center bg-primary-600 text-
white px-6 py-2 rounded-lg font-medium hover:bg-primary-700 focus:ring-4 focus:ring-
primary-100 transition-all disabled:opacity-50"
            >
            <Save className="h-4 w-4 mr-2" />
            {loading ? 'Saving...' : 'Save Changes'}
        </button>
    </div>
</form>
</div>
</div>
</div>
</div>
);
};
export default ProfilePage;

```

```

=====
File: frontend\src\admin\ProviderProfilePage.tsx
=====

```

```

import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import Navbar from '../components/Navbar';
import { Star, MapPin, Shield, Clock, ArrowLeft, CheckCircle } from 'lucide-react';
import api from '../api/api';
import toast from 'react-hot-toast';
import { useAuthStore } from '../store/authStore';

interface ServiceProvider {
  _id: string;
  businessName?: string;
  serviceTypes: string[];
  hourlyRate: number;
  experience: number;
  location: {
    address: string;
    city: string;
    state: string;
    zipCode: string;
  };
  rating: number;
  totalReviews: number;
  isVerified: boolean;
  description: string;
  userId: {
    firstName: string;
    lastName: string;
    phone: string;
  };
};

interface Review {
  _id: string;
  rating: number;
  comment: string;
  customerId: {
    firstName: string;
    lastName: string;
  };
  createdAt: string;
};

```

```

const ProviderProfilePage = () => {
  const { id } = useParams();
  const { user } = useAuthStore();
  const [provider, setProvider] = useState<ServiceProvider | null>(null);
  const [reviews, setReviews] = useState<Review[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (id) {
      fetchProvider();
      fetchReviews();
    }
  }, [id]);

  const fetchProvider = async () => {
    try {
      const response = await api.get(`/service-providers/${id}`);
      setProvider(response.data);
    } catch (error) {
      toast.error('Failed to load provider');
    } finally {
      setLoading(false);
    }
  };

  const fetchReviews = async () => {
    try {
      const response = await api.get(`/reviews/provider/${id}`);
      setReviews(response.data);
    } catch (error) {
      // Reviews might not exist yet
    }
  };

  if (loading) {
    return (
      <div className="min-h-screen bg-dark-900 text-white">
        <Navbar />
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-20 flex justify-center">
          <div className="animate-spin rounded-full h-12 w-12 border-t-2 border-b-2 border-primary-500"></div>
        </div>
      </div>
    );
  }

  if (!provider) {
    return (
      <div className="min-h-screen bg-dark-900 text-white">
        <Navbar />
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-20 text-center">
          <h2 className="text-2xl font-bold mb-4">Provider not found</h2>
          <Link to="/providers" className="text-primary-400 hover:text-primary-300">
            Return to Browse
          </Link>
        </div>
      </div>
    );
  }

  const name = provider.businessName || `${provider.userId.firstName} ${provider.userId.lastName}`;

  return (
    <div className="min-h-screen bg-dark-900 text-white relative">
      <div className="absolute top-0 right-0 w-[500px] h-[500px] bg-primary-600/10 rounded-full blur-[100px] pointer-events-none" />
      <Navbar />
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 animate-fade-in relative min-h-screen">

```



```

        <Link to="/providers" className="inline-flex items-center text-dark-400
hover:text-white mb-8 transition-colors">ð
        <ArrowLeft className="h-4 w-4 mr-2" />ð
        Back to Providersð
    </Link>ð
ð
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">ð
ð
        {/* Left Column: Profile Info */}ð
        <div className="lg:col-span-2 space-y-8">ð
            <div className="glass-card rounded-2xl border-white/5 p-8 relative
overflow-hidden">ð
                <div className="absolute top-0 right-0 p-8 opacity-5">ð
                    <Shield className="h-32 w-32" />ð
                </div>ð
ð
                <div className="relative z-10">ð
                    <div className="flex flex-col md:flex-row justify-between items-
start md:items-center mb-6">ð
                        <div>ð
                            <div className="flex items-center space-x-3 mb-2">ð
                                <h1 className="text-4xl font-bold font-heading">{name}</h1>ð
                                {provider.isVerified && (ð
                                    <div title="Verified Professional">ð
                                        <Shield className="h-6 w-6 text-primary-400 fill-
primary-400/20" />ð
                                    </div>ð
                                )}ð
                            </div>ð
                            <div className="flex items-center space-x-2 text-dark-300">ð
                                <MapPin className="h-5 w-5 text-primary-500" />ð
                                <span>ð
                                    {provider.location.city}, {provider.location.state}ð
                                </span>ð
                            </div>ð
                        </div>ð
                    </div>ð
                </div>ð
            </div>ð
ð
            <div className="flex flex-wrap gap-2 mb-8">ð
                {provider.serviceTypes.map((type) => (ð
                    <spanð
                        key={type}ð
                        className="px-4 py-1.5 bg-primary-500/10 border border-
primary-500/20 text-primary-300 rounded-lg capitalize font-medium"ð
                    >ð
                        {type}ð
                    </span>ð
                ))}ð
            </div>ð
ð
            <div className="prose prose-invert max-w-none">ð
                <h3 className="text-lg font-bold text-white mb-2">About</h3>ð
                <p className="text-dark-300 leading-relaxed">{provider.description}
</p>ð
            </div>ð
        </div>ð
ð
        {/* Stats Grid */}ð
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">ð
            <div className="glass-card p-6 rounded-2xl border-white/5 text-center">ð
                <div className="flex justify-center mb-2">ð
                    <Star className="h-8 w-8 text-yellow-500 fill-yellow-500" />ð
                </div>ð
                <p className="text-3xl font-bold text-white
mb-1">{provider.rating.toFixed(1)}</p>ð
                <p className="text-sm text-dark-400">{provider.totalReviews}
reviews</p>ð
            </div>ð
            <div className="glass-card p-6 rounded-2xl border-white/5 text-center">ð
                <div className="flex justify-center mb-2">ð
                    <Clock className="h-8 w-8 text-primary-500" />ð

```

```

        </div>ð
        <p className="text-3xl font-bold text-white mb-1">{provider.experience}</p>ð
        <p className="text-sm text-dark-400">Years Experience</p>ð
    </div>ð
    <div className="glass-card p-6 rounded-2xl border-white/5 text-center">ð
        <div className="flex justify-center mb-2">ð
            <CheckCircle className="h-8 w-8 text-green-500" />ð
        </div>ð
        <p className="text-3xl font-bold text-white mb-1">100%</p>ð
        <p className="text-sm text-dark-400">Job Success</p>ð
    </div>ð
</div>ð

ð
/* Reviews Section */ð
<div className="glass-card rounded-2xl border-white/5 p-8">ð
    <h2 className="text-2xl font-bold font-heading mb-6">Client Reviews</
h2>ð

    {reviews.length > 0 ? (ð
        <div className="space-y-6">ð
            {reviews.map((review) => (ð
                <div key={review._id} className="border-b border-white/5 pb-6 last:border-0 last:pb-0">ð
                    <div className="flex items-start justify-between mb-4">ð
                        <div>ð
                            <p className="font-bold text-white">ð
                                {review.customerId.firstName}
{review.customerId.lastName}&#x27;>ð
                            </p>ð
                            <div className="flex items-center space-x-1 mt-1">ð
                                {[...Array(5)].map((_, i) => (ð
                                    <Starð
                                        key={i}&#x27;>ð
                                        className={`h-4 w-4 ${i < review.rating&#x27;>
                                            ? 'text-yellow-500 fill-yellow-500'ð
                                            : 'text-dark-600'}ð
                                        }`ð
                                    )&#x27;>ð
                                )&#x27;>ð
                            </div>ð
                        </div>ð
                        <div>ð
                            <span className="text-sm text-dark-400">ð
                                {new Date(review.createdAt).toLocaleDateString()}ð
                            </span>ð
                        </div>ð
                        <div>ð
                            {review.comment &#x27;> (ð
                                <p className="text-dark-300 italic">"{review.comment}"</p>ð
                            )&#x27;>ð
                        </div>ð
                    </div>ð
                </div>ð
            )&#x27;>ð
        </div>ð
    ) : (ð
        <p className="text-dark-400 text-center py-8">No reviews yet.</p>ð
    )&#x27;>ð
</div>ð

ð
/* Right Column: Booking Card */ð
<div className="lg:col-span-1">ð
    <div className="glass-card rounded-2xl border-white/5 p-6 sticky top-24 border-primary-500/20 shadow-2xl shadow-primary-900/20">ð
        <h3 className="text-xl font-bold text-white mb-6">Booking Summary</h3>ð
    ð
    <div className="flex items-center justify-between mb-6 pb-6 border-b border-white/10">ð
        <span className="text-dark-300">Hourly Rate</span>ð
        <span className="text-2xl font-bold text-white"> ^{provider.hourlyRate}
</span>ð
    </div>ð

    ð
    <div className="space-y-4 mb-8">ð
        <div className="flex items-center text-dark-300 text-sm">ð

```

```

        <CheckCircle className="h-4 w-4 text-green-500 mr-2" />ð
        Verified Licenseð
    </div>ð
    <div className="flex items-center text-dark-300 text-sm">ð
        <CheckCircle className="h-4 w-4 text-green-500 mr-2" />ð
        Background Checkedð
    </div>ð
    <div className="flex items-center text-dark-300 text-sm">ð
        <CheckCircle className="h-4 w-4 text-green-500 mr-2" />ð
        Satisfaction Guaranteeð
    </div>ð
</div>ð

ð
    {user && user.role === 'customer' ? (ð
        <Linkð
            to={`/${book}/${provider._id}`}ð
            className="block w-full bg-gradient-to-r from-primary-600 to-
primary-500 text-white text-center py-4 rounded-xl font-bold hover:shadow-lg
hover:shadow-primary-500/25 transition-all active:scale-95"ð
            >ð
                Book Appointmentð
            </Link>ð
        ) : !user ? (ð
            <Linkð
                to="/login"ð
                className="block w-full bg-dark-700 text-white text-center py-4
rounded-xl font-bold hover:bg-dark-600 transition-all"ð
                >ð
                    Login to Bookð
                </Link>ð
            ) : (ð
                <div className="text-center p-4 bg-dark-800 rounded-xl text-dark-300
text-sm">ð
                    Only customers can book services. Your current role is
<strong>{user.role.replace('_', ' ')}</strong>.ð
                </div>ð
            )}ð
        </div>ð
    </div>ð
</div>ð
</div>ð
</div>ð
);ð
};ð
ð
export default ProviderProfilePage;ð
ð

```

```

=====
File: frontend\src\admin\RegisterPage.tsx
=====

```

```

import { useState } from 'react';ð
import { Link, useNavigate } from 'react-router-dom';ð
import { useAuthStore } from '../store/authStore';ð
import api from '../api/api';ð
import toast from 'react-hot-toast';ð
import Navbar from '../components/Navbar';ð
import { Logo } from '../components/Logo';ð
import { ArrowLeft, User, Mail, Phone, Lock, CheckCircle, Eye, EyeOff } from 'lucide-
react';ð
ð
const RegisterPage = () => {ð
    const [formData, setFormData] = useState({ð
        email: '',ð
        password: '',ð
        firstName: '',ð
        lastName: '',ð
        phone: '',ð
        role: 'customer' as 'customer' | 'service_provider',ð
    });

```

```

    });
    const [loading, setLoading] = useState(false);
    const [showPassword, setShowPassword] = useState(false);
    const { setAuth } = useAuthStore();
    const navigate = useNavigate();

    const handleSubmit = async (e: React.FormEvent) => {
      e.preventDefault();

      // Phone number validation (10 digits)
      const phoneRegex = /^[0-9]{10}$/;
      if (!phoneRegex.test(formData.phone)) {
        toast.error('Please enter a valid 10-digit phone number');
        return;
      }

      setLoading(true);

      try {
        const response = await api.post('/auth/register', formData);
        setAuth(response.data.user, response.data.token);
        const roleName = response.data.user.role === 'service_provider' ? 'Service
Provider' : 'Customer';
        toast.success(`Registration successful! Welcome, ${roleName}`);

        if (response.data.user.role === 'service_provider') {
          navigate('/provider/setup');
        } else {
          navigate('/dashboard');
        }
      } catch (error: any) {
        toast.error(error.response?.data?.message || 'Registration failed');
      } finally {
        setLoading(false);
      }
    };

    return (
      <div className="min-h-screen bg-dark-900 text-white relative">
        <div className="absolute inset-0 bg-[url('https://images.unsplash.com/
photo-1517048676732-d65bc937f952?ixlib=rb-1.2.1&auto=format&fit=crop&w=1920&q=80')]
bg-cover bg-center opacity-10 pointer-events-none" />
        <Navbar />

        <div className="relative flex items-center justify-center min-h-
[calc(100vh-80px)] px-4 sm:px-6 lg:px-8 py-12">
          <div className="max-w-md w-full glass-card p-8 rounded-2xl animate-fade-in">
            <div className="text-center mb-10">
              <Link to="/" className="inline-flex items-center text-primary-400
hover:text-primary-300 mb-6 transition-colors">
                <ArrowLeft className="h-4 w-4 mr-2" />
                Back to Home
              </Link>
              <div className="flex justify-center mb-4">
                <div className="bg-primary-500/20 p-4 rounded-full">
                  <Logo className="h-10 w-10 text-primary-500" />
                </div>
              </div>
              <h2 className="text-3xl font-bold font-heading">
                Create Account
              </h2>
              <p className="mt-2 text-dark-300">
                Join our community today
              </p>
            </div>

            <form className="space-y-6" onSubmit={handleSubmit}>
              <div className="grid grid-cols-2 gap-4">
                <div>
                  <label htmlFor="firstName" className="block text-sm font-medium text-
dark-200 mb-1">
                    First Name

```

```

        </label>ð
        <div className="relative">ð
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none">ð
                <User className="h-5 w-5 text-dark-400" />ð
            </div>ð
            <inputð
                id="firstName"ð
                name="firstName"ð
                type="text"ð
                requiredð
                value={formData.firstName}ð
                onChange={(e) => setFormData({ ...formData, firstName:
e.target.value })}ð
                className="block w-full pl-10 pr-3 py-3 border border-dark-600
rounded-xl leading-5 bg-dark-800 text-white placeholder-dark-400 focus:outline-none
focus:border-primary-500 focus:ring-1 focus:ring-primary-500 sm:text-sm transition-
colors"ð
            />ð
        </div>ð
    </div>ð
    <div>ð
        <label htmlFor="lastName" className="block text-sm font-medium text-
dark-200 mb-1">ð
            Last Nameð
        </label>ð
        <div className="relative">ð
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none">ð
            </div>ð
            <inputð
                id="lastName"ð
                name="lastName"ð
                type="text"ð
                requiredð
                value={formData.lastName}ð
                onChange={(e) => setFormData({ ...formData, lastName:
e.target.value })}ð
                className="block w-full px-3 py-3 border border-dark-600 rounded-
xl leading-5 bg-dark-800 text-white placeholder-dark-400 focus:outline-none
focus:border-primary-500 focus:ring-1 focus:ring-primary-500 sm:text-sm transition-
colors"ð
            />ð
        </div>ð
    </div>ð
ð
    <div>ð
        <label htmlFor="email" className="block text-sm font-medium text-
dark-200 mb-1">ð
            Email addressð
        </label>ð
        <div className="relative">ð
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none">ð
                <Mail className="h-5 w-5 text-dark-400" />ð
            </div>ð
            <inputð
                id="email"ð
                name="email"ð
                type="email"ð
                requiredð
                value={formData.email}ð
                onChange={(e) => setFormData({ ...formData, email:
e.target.value })}ð
                className="block w-full pl-10 pr-3 py-3 border border-dark-600
rounded-xl leading-5 bg-dark-800 text-white placeholder-dark-400 focus:outline-none
focus:border-primary-500 focus:ring-1 focus:ring-primary-500 sm:text-sm transition-
colors"ð
            />ð
        </div>ð
    </div>ð

```

```

ð
    <div>ð
        <label htmlFor="phone" className="block text-sm font-medium text-
dark-200 mb-1">ð
            Phone Numberð
        </label>ð
        <div className="relative">ð
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none">ð
                <Phone className="h-5 w-5 text-dark-400" />ð
            </div>ð
            <inputð
                id="phone"ð
                name="phone"ð
                type="tel"ð
                requiredð
                maxLength={10}ð
                pattern="[0-9]{10}"ð
                value={formData.phone}ð
                onChange={(e) => setFormData({ ...formData, phone:
e.target.value.replace(/\D/g, '') })}ð
                className="block w-full pl-10 pr-3 py-3 border border-dark-600
rounded-xl leading-5 bg-dark-800 text-white placeholder-dark-400 focus:outline-none
focus:border-primary-500 focus:ring-1 focus:ring-primary-500 sm:text-sm transition-
colors"ð
                    placeholder="Enter 10-digit phone number"ð
                />ð
            </div>ð
        </div>ð
ð
    <div>ð
        <label htmlFor="password" className="block text-sm font-medium text-
dark-200 mb-1">ð
            Passwordð
        </label>ð
        <div className="relative">ð
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center
pointer-events-none">ð
                <Lock className="h-5 w-5 text-dark-400" />ð
            </div>ð
            <inputð
                id="password"ð
                name="password"ð
                type={showPassword ? 'text' : 'password'}ð
                requiredð
                minLength={6}ð
                value={formData.password}ð
                onChange={(e) => setFormData({ ...formData, password:
e.target.value })}ð
                className="block w-full pl-10 pr-10 py-3 border border-dark-600
rounded-xl leading-5 bg-dark-800 text-white placeholder-dark-400 focus:outline-none
focus:border-primary-500 focus:ring-1 focus:ring-primary-500 sm:text-sm transition-
colors"ð
                    />ð
            <buttonð
                type="button"ð
                onClick={() => setShowPassword(!showPassword)}ð
                className="absolute inset-y-0 right-0 pr-3 flex items-center text-
dark-400 hover:text-dark-200 transition-colors"ð
                    >ð
                        {showPassword ? (ð
                            <EyeOff className="h-5 w-5" />ð
                        ) : (ð
                            <Eye className="h-5 w-5" />ð
                        )}ð
                    </button>ð
            </div>ð
        </div>ð
ð
    <div>ð
        <label className="block text-sm font-medium text-dark-200 mb-3">ð
            I am a:ð

```

```

        </label>ð
        <div className="grid grid-cols-2 gap-4">ð
            <label className={`ð
                relative flex flex-col items-center p-4 cursor-pointer rounded-xl
border-2 transition-allð
                ${formData.role === 'customer'ð
                    ? 'border-primary-500 bg-primary-500/10'ð
                    : 'border-dark-600 bg-dark-800 hover:border-dark-500'}ð
                `}>ð
                <inputð
                    type="radio"ð
                    name="role"ð
                    value="customer"ð
                    checked={formData.role === 'customer'}ð
                    onChange={(e) => setFormData({ ...formData, role: e.target.value
as 'customer' | 'service_provider' })}ð
                    className="sr-only"ð
                />ð
                <span className="font-semibold">Customer</span>ð
                {formData.role === 'customer' && (ð
                    <CheckCircle className="absolute top-2 right-2 h-4 w-4 text-
primary-500" />ð
                )}ð
            </label>ð
        ð

        <label className={`ð
            relative flex flex-col items-center p-4 cursor-pointer rounded-xl
border-2 transition-allð
            ${formData.role === 'service_provider'ð
                ? 'border-primary-500 bg-primary-500/10'ð
                : 'border-dark-600 bg-dark-800 hover:border-dark-500'}ð
            `}>ð
            <inputð
                type="radio"ð
                name="role"ð
                value="service_provider"ð
                checked={formData.role === 'service_provider'}ð
                onChange={(e) => setFormData({ ...formData, role: e.target.value
as 'customer' | 'service_provider' })}ð
                className="sr-only"ð
            />ð
            <span className="font-semibold text-center">Provider</span>ð
            {formData.role === 'service_provider' && (ð
                <CheckCircle className="absolute top-2 right-2 h-4 w-4 text-
primary-500" />ð
            )}ð
        </label>ð
    </div>ð
ð

    <buttonð
        type="submit"ð
        disabled={loading}ð
        className="w-full flex justify-center py-3 px-4 border border-
transparent rounded-xl shadow-lg text-sm font-medium text-white bg-gradient-to-r
from-primary-600 to-primary-500 hover:from-primary-500 hover:to-primary-400
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-primary-500
disabled:opacity-50 transition-all transform active:scale-95"ð
    >ð
        {loading ? 'Creating account...' : 'Create account'}ð
    </button>ð
ð

    <div className="text-center mt-4">ð
        <p className="text-sm text-dark-300">ð
            Already have an account?{' '}ð
            <Link to="/login" className="font-medium text-primary-400 hover:text-
primary-300">ð
                Sign inð
            </Link>ð
        </p>ð
    </div>ð
</form>ð

```

```

        </div>ð
    </div>ð
</div>ð
);ð
};ð
ð
export default RegisterPage;ð
ð

```

```

=====
File: frontend\src\admin\ServiceProviderDashboard.tsx
=====

```

```

import { useEffect, useState } from 'react';ð
import { Link } from 'react-router-dom';ð
import Navbar from '../components/Navbar';ð
import api from '../api/api';ð
import toast from 'react-hot-toast';ð
import { Clock, DollarSign, Calendar, AlertCircle, TrendingUp, Power, CheckCircle }
from 'lucide-react';ð
ð
interface Booking {ð
  _id: string;ð
  serviceType: string;ð
  status: string;ð
  scheduledDate: string;ð
  totalAmount: number;ð
  customerId: {ð
    firstName: string;ð
    lastName: string;ð
  };ð
};ð
}ð
ð
interface ProviderStats {ð
  totalBookings: number;ð
  completedBookings: number;ð
  pendingBookings: number;ð
  totalEarnings: number;ð
}ð
ð
const ServiceProviderDashboard = () => {ð
  const [bookings, setBookings] = useState<Booking[]>([]);ð
  const [stats, setStats] = useState<ProviderStats>({ð
    totalBookings: 0,ð
    completedBookings: 0,ð
    pendingBookings: 0,ð
    totalEarnings: 0,ð
  });ð
  const [loading, setLoading] = useState(true);ð
  const [isAvailable, setIsAvailable] = useState(true); // Mock availability stateð
ð
  useEffect(() => {ð
    fetchBookings();ð
  }, []);ð
ð
  const fetchBookings = async () => {ð
    try {ð
      const response = await api.get('/bookings/my-bookings');ð
      const bookingsData = response.data;ð
      setBookings(bookingsData.slice(0, 5));ð
ð
      // Calculate statsð
      const completed = bookingsData.filter((b: Booking) => b.status ===
'completed');ð
      const pending = bookingsData.filter((b: Booking) =>ð
        ['pending', 'confirmed', 'in_progress'].includes(b.status)ð
      );ð
      const earnings = completed.reduce((sum: number, b: Booking) => sum +
b.totalAmount, 0);ð
ð

```



```

        setStats({
          totalBookings: bookingsData.length,
          completedBookings: completed.length,
          pendingBookings: pending.length,
          totalEarnings: earnings,
        });
      } catch (error) {
        // toast.error('Failed to load dashboard data');
        console.log(error);
      } finally {
        setLoading(false);
      }
    };
  },
  const toggleAvailability = () => {
    // In a real app, this would make an API call
    setIsAvailable(!isAvailable);
    toast.success(isAvailable ? 'You are now offline' : 'You are now online');
  },
  const StatCard = ({ title, value, icon: Icon, color, prefix = '' }: any) => (
    <div className="glass-card p-6 rounded-2xl border-white/5 relative overflow-hidden group hover:-translate-y-1 transition-transform duration-300">
      <div className={`absolute top-0 right-0 p-4 opacity-10 group-hover:opacity-20 transition-opacity`}>
        <Icon className={`h-20 w-20 ${color}`} />
      </div>
      <div className="relative z-10">
        <div className={`h-12 w-12 rounded-xl bg-white/5 flex items-center justify-center mb-4 border border-white/10`}>
          <Icon className={`h-6 w-6 ${color}`} />
        </div>
        <p className="text-dark-400 text-sm font-medium mb-1">{title}</p>
        <p className="text-3xl font-bold text-white tracking-tight">{prefix}{value}</p>
      </div>
    </div>
  );
  return (
    <div className="min-h-screen bg-dark-900 text-white relative overflow-hidden">
      <div className="absolute top-0 left-0 w-[500px] h-[500px] bg-secondary-600/10 rounded-full blur-[100px] pointer-events-none" />
      <Navbar />
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12 relative">
        <div className="flex flex-col md:flex-row justify-between items-start md:items-end mb-12 animate-fade-in gap-6">
          <div>
            <h1 className="text-4xl font-bold font-heading mb-2">
              Provider Dashboard
            </h1>
            <p className="text-dark-300">Manage your business and track performance.</p>
          </div>
          <div className="flex items-center gap-4 bg-dark-800/50 p-2 rounded-xl border border-white/5">
            <div className={`flex items-center px-4 py-2 rounded-lg transition-colors ${isAvailable ? 'bg-green-500/20 text-green-400' : 'text-dark-400'}`}>
              <CheckCircle className="h-4 w-4 mr-2" />
              <span className="font-bold text-sm">Online</span>
            </div>
            <button
              onClick={toggleAvailability}
              className={`p-2 rounded-lg transition-all ${isAvailable ? 'bg-red-500/10 hover:bg-red-500/20 text-red-400' : 'bg-green-500/20 text-green-400'}`}
              title={isAvailable ? "Go Offline" : "Go Online"}
            >
              <Power className="h-5 w-5" />
            </button>
          </div>
        </div>
      </div>
    </div>
  );

```

```

        </div>ð
    </div>ð
ð
    {/ * Stats Cards */}ð
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-12
animate-slide-up">ð
        <StatCardð
            title="Total Earnings"ð
            value={stats.totalEarnings.toFixed(2)}ð
            icon={DollarSign}ð
            color="text-green-400"ð
            prefix=" 1"ð
        />ð
        <StatCardð
            title="Active Jobs"ð
            value={stats.pendingBookings}ð
            icon={Clock}ð
            color="text-yellow-400"ð
        />ð
        <StatCardð
            title="Completed"ð
            value={stats.completedBookings}ð
            icon={TrendingUp}ð
            color="text-primary-400"ð
        />ð
        <StatCardð
            title="Total Bookings"ð
            value={stats.totalBookings}ð
            icon={Calendar}ð
            color="text-secondary-400"ð
        />ð
    </div>ð
ð
    {/ * Quick Actions */}ð
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mb-12">ð
        <Linkð
            to="/provider/setup"ð
            className="group glass-card p-8 rounded-2xl border-white/5 hover:border-
primary-500/30 transition-all duration-300 relative overflow-hidden"ð
        >ð
            <div className="absolute inset-0 bg-gradient-to-r from-primary-600/10 to-
transparent opacity-0 group-hover:opacity-100 transition-opacity" />ð
            <div className="relative z-10">ð
                <h3 className="text-xl font-bold text-white mb-2">Edit Profile</h3>ð
                <p className="text-dark-400">Update services, pricing, and business
details</p>ð
            </div>ð
        </Link>ð
ð
        <Linkð
            to="/my-bookings"ð
            className="group glass-card p-8 rounded-2xl border-white/5 hover:border-
secondary-500/30 transition-all duration-300 relative overflow-hidden"ð
        >ð
            <div className="absolute inset-0 bg-gradient-to-r from-secondary-600/10
to-transparent opacity-0 group-hover:opacity-100 transition-opacity" />ð
            <div className="relative z-10">ð
                <h3 className="text-xl font-bold text-white mb-2">Manage Bookings</h3>ð
                <p className="text-dark-400">View upcoming jobs and booking requests</
p>ð
            </div>ð
        </Link>ð
    </div>ð
ð
    {/ * Recent Bookings */}ð
    <div className="glass-card rounded-2xl border-white/5 p-8 animate-fade-in
delay-100">ð
        <h2 className="text-2xl font-bold font-heading mb-8">Recent Requests</h2>ð
        {loading ? (ð
            <div className="animate-pulse space-y-4">ð
                {[1, 2, 3].map(i => (ð
                    <div key={i} className="h-24 bg-dark-800 rounded-xl"></div>ð
                ))}ð
            </div>ð
        ) : null}ð
    </div>ð

```

```

    )))
  </div>
) : bookings.length === 0 ? (
  <div className="text-center py-12 bg-dark-800/50 rounded-xl border
border-white/5">
    <AlertCircle className="h-12 w-12 text-dark-500 mx-auto mb-4" />
    <p className="text-dark-300 mb-2">No active bookings yet</p>
    <p className="text-sm text-dark-500">
      Optimize your profile to attract more customers
    </p>
  </div>
) : (
  <div className="space-y-4">
    {bookings.map((booking) => (
      <Link
        key={booking._id}
        to={`/bookings/${booking._id}`}
        className="block p-5 bg-dark-800/50 border border-white/5 rounded-
xl hover:border-primary-500/30 hover:bg-dark-800 transition-all group">
        <div className="flex justify-between items-center">
          <div className="flex items-center gap-4">
            <div className="h-12 w-12 rounded-full bg-dark-700 flex items-
center justify-center text-dark-300 font-bold border border-white/5">
              {booking.serviceType.charAt(0)}
            </div>
            <div>
              <h3 className="font-bold text-white capitalize group-
hover:text-primary-400 transition-colors">
                {booking.serviceType}
              </h3>
              <p className="text-sm text-dark-300">
                Client: {booking.customerId.firstName}
                {booking.customerId.lastName}
              </p>
            </div>
          </div>
          <div className="text-right">
            <span className={`inline-block px-3 py-1 text-xs font-bold
rounded-full mb-2 ${booking.status === 'completed' ? 'bg-green-500/20 text-
green-400' :
              booking.status === 'confirmed' ? 'bg-blue-500/20 text-
blue-400' :
              booking.status === 'in_progress' ? 'bg-yellow-500/20 text-
yellow-400' :
              'bg-dark-700 text-dark-300'}>
              {booking.status.replace('_', ' ').toUpperCase()}
            </span>
            <p className="text-lg font-bold text-white">
              {booking.totalAmount.toFixed(2)}
            </p>
          </div>
        </div>
      </Link>
    )))
  </div>
))
</div>
</div>
</div>
);
};
export default ServiceProviderDashboard;

```

```

=====
File: frontend\src\admin\ServiceProviderProfileSetup.tsx
=====

```

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import Navbar from '../components/Navbar';
import api from '../api/api';
import toast from 'react-hot-toast';
import { MapPin, DollarSign, Clock, FileText } from 'lucide-react';

interface ProfileData {
  businessName: string;
  serviceTypes: string[];
  experience: number;
  hourlyRate: number;
  description: string;
  location: {
    address: string;
    city: string;
    state: string;
    zipCode: string;
    coordinates: {
      lat: number;
      lng: number;
    };
  };
  availability: {
    monday: { start: string; end: string; available: boolean };
    tuesday: { start: string; end: string; available: boolean };
    wednesday: { start: string; end: string; available: boolean };
    thursday: { start: string; end: string; available: boolean };
    friday: { start: string; end: string; available: boolean };
    saturday: { start: string; end: string; available: boolean };
    sunday: { start: string; end: string; available: boolean };
  };
}

const ServiceProviderProfileSetup = () => {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [formData, setFormData] = useState<ProfileData>({
    businessName: '',
    serviceTypes: [],
    experience: 0,
    hourlyRate: 0,
    description: '',
    location: {
      address: '',
      city: '',
      state: '',
      zipCode: '',
      coordinates: { lat: 0, lng: 0 },
    },
    availability: {
      monday: { start: '09:00', end: '17:00', available: true },
      tuesday: { start: '09:00', end: '17:00', available: true },
      wednesday: { start: '09:00', end: '17:00', available: true },
      thursday: { start: '09:00', end: '17:00', available: true },
      friday: { start: '09:00', end: '17:00', available: true },
      saturday: { start: '09:00', end: '17:00', available: true },
      sunday: { start: '09:00', end: '17:00', available: false },
    },
  });

  const serviceTypeOptions = ['plumber', 'electrician', 'mechanic', 'carpenter', 'painter', 'handyman', 'other'];

  useEffect(() => {
    fetchProfile();
  }, []);

  const fetchProfile = async () => {
    try {

```

```

    const response = await api.get('/service-providers/profile/me');
    if (response.data) {
      setFormData(response.data);
    }
  } catch (error) {
    // Profile doesn't exist yet, that's ok
  } finally {
    setLoading(false);
  }
};

const handleServiceTypeToggle = (type: string) => {
  setFormData(prev => ({
    ...prev,
    serviceTypes: prev.serviceTypes.includes(type)
      ? prev.serviceTypes.filter(t => t !== type)
      : [...prev.serviceTypes, type]
  }));
};

const handleLocationChange = (field: string, value: string) => {
  setFormData(prev => ({
    ...prev,
    location: {
      ...prev.location,
      [field]: value
    }
  }));
};

const getCurrentLocation = () => {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition((position) => {
      setFormData(prev => ({
        ...prev,
        location: {
          ...prev.location,
          coordinates: {
            lat: position.coords.latitude,
            lng: position.coords.longitude,
          }
        }
      }));
      toast.success('Location updated');
    }, () => {
      toast.error('Failed to get location');
    });
  }
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setSaving(true);

  try {
    const existing = await api.get('/service-providers/profile/me').catch(() => null);
    if (existing?.data) {
      await api.put(`/service-providers/${existing.data._id}`, formData);
      toast.success('Profile updated successfully!');
    } else {
      await api.post('/service-providers', formData);
      toast.success('Profile created successfully!');
    }
    navigate('/provider/dashboard');
  } catch (error: any) {
    toast.error(error.response?.data?.message || 'Failed to save profile');
  } finally {
    setSaving(false);
  }
};

```

```

    };
    if (loading) {
        return (
            <div className="min-h-screen bg-gray-50">
                <Navbar />
                <div className="max-w-4xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
                    <p className="text-center text-gray-600">Loading...</p>
                </div>
            </div>
        );
    }
    return (
        <div className="min-h-screen bg-gray-50">
            <Navbar />
            <div className="max-w-4xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
                <h1 className="text-3xl font-bold text-gray-900 mb-6">Service Provider
Profile</h1>
                <form onSubmit={handleSubmit} className="bg-white rounded-lg shadow-md p-8
space-y-8">
                    <div>
                        <h2 className="text-xl font-semibold text-gray-900 mb-4">Basic
Information</h2>
                        <div className="space-y-4">
                            <div>
                                <label className="block text-sm font-medium text-gray-700 mb-2">
                                    Business Name (Optional)
                                </label>
                                <input
                                    type="text"
                                    value={formData.businessName}
                                    onChange={(e) => setFormData({ ...formData, businessName:
e.target.value })}
                                    className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
                                    placeholder="Your business name"
                                />
                            </div>
                            <div>
                                <label className="block text-sm font-medium text-gray-700 mb-2">
                                    Service Types
                                </label>
                                <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
                                    {serviceTypeOptions.map((type) => (
                                        <label key={type} className="flex items-center space-x-2 cursor-
pointer">
                                            <input
                                                type="checkbox"
                                                checked={formData.serviceTypes.includes(type)}
                                                onChange={() => handleServiceTypeToggle(type)}
                                                className="rounded border-gray-300 text-primary-600
focus:ring-primary-500"
                                            />
                                            <span className="text-sm text-gray-700 capitalize">{type}</
span>
                                        </label>
                                    ))}
                                </div>
                            </div>
                        </div>
                    </div>
                <div className="grid grid-cols-2 gap-4">
                    <div>
                        <label className="block text-sm font-medium text-gray-700 mb-2">
                            <Clock className="h-4 w-4 inline mr-1" />
                            Years of Experience
                        </label>
                        <input
                            type="number"

```

```

        min="0"
        required
        value={formData.experience}
        onChange={(e) => setFormData({ ...formData, experience:
parseInt(e.target.value) })}
        className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
    />
</div>

<div>
    <label className="block text-sm font-medium text-gray-700 mb-2">
        <DollarSign className="h-4 w-4 inline mr-1" />
        Hourly Rate ( ^ )
    </label>
    <input
        type="number"
        min="0"
        step="0.01"
        required
        value={formData.hourlyRate}
        onChange={(e) => setFormData({ ...formData, hourlyRate:
parseFloat(e.target.value) })}
        className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
    />
    </div>
</div>

<div>
    <label className="block text-sm font-medium text-gray-700 mb-2">
        <FileText className="h-4 w-4 inline mr-1" />
        Description
    </label>
    <textarea
        rows={4}
        required
        value={formData.description}
        onChange={(e) => setFormData({ ...formData, description:
e.target.value })}
        placeholder="Tell customers about your services and expertise..."
        className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
    />
    </div>
</div>
</div>

{ /* Location */ }
<div>
    <h2 className="text-xl font-semibold text-gray-900 mb-4">
        <MapPin className="h-5 w-5 inline mr-1" />
        Location
    </h2>
    <div className="space-y-4">
        <input
            type="text"
            required
            placeholder="Street Address"
            value={formData.location.address}
            onChange={(e) => handleLocationChange('address', e.target.value)}
            className="w-full px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"
        />
        <div className="grid grid-cols-3 gap-4">
            <input
                type="text"
                required
                placeholder="City"
                value={formData.location.city}
                onChange={(e) => handleLocationChange('city', e.target.value)}
                className="px-4 py-2 border border-gray-300 rounded-lg

```

```

focus:outline-none focus:ring-2 focus:ring-primary-500"ð
    />ð
    <inputð
      type="text"ð
      requiredð
      placeholder="State"ð
      value={formData.location.state}ð
      onChange={(e) => handleLocationChange('state', e.target.value)}ð
      className="px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"ð
    />ð
    <inputð
      type="text"ð
      requiredð
      placeholder="ZIP Code"ð
      value={formData.location.zipCode}ð
      onChange={(e) => handleLocationChange('zipCode', e.target.value)}ð
      className="px-4 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary-500"ð
    />ð
  </div>ð
  <buttonð
    type="button"ð
    onClick={getCurrentLocation}ð
    className="text-primary-600 hover:text-primary-700 text-sm font-
medium"ð
    >ð
      Use Current Locationð
    </button>ð
  </div>ð
</div>ð
ð
  <buttonð
    type="submit"ð
    disabled={saving || formData.serviceTypes.length === 0}ð
    className="w-full bg-primary-600 text-white py-3 rounded-lg font-
semibold hover:bg-primary-700 transition-colors disabled:opacity-50"ð
    >ð
      {saving ? 'Saving...' : 'Save Profile'}ð
    </button>ð
  </form>ð
</div>ð
</div>ð
);ð
};ð
ð
export default ServiceProviderProfileSetup;ð
ð

```

```

=====
File: frontend\src\api\api.ts
=====

```

```

import axios from 'axios';ð
import { useAuthStore } from '../store/authStore';ð
import toast from 'react-hot-toast';ð
ð
const api = axios.create({ð
  baseURL: '/api',ð
  timeout: 10000, // 10 second timeoutð
});ð
ð
api.interceptors.request.use((config) => {ð
  const token = useAuthStore.getState().token;ð
  if (token) {ð
    config.headers.Authorization = `Bearer ${token}`;ð
  }ð
  return config;ð
});ð
ð

```



```

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response) {
      // Server responded with error status
    }
    // Don't show toast for 401 errors (handled by auth redirect)
    if (error.response.status === 401) {
      const { logout } = useAuthStore.getState();
      logout();
      // Redirect handled by ProtectedRoute
    } else if (error.response.status >= 500) {
      toast.error('Server error. Please try again later.');

```

```

=====
File: frontend\src\App.tsx
=====

```

```

import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { Toaster } from 'react-hot-toast';
import { useAuthStore } from '../store/authStore';
import HomePage from './admin/HomePage';
import LoginPage from './admin/LoginPage';
import RegisterPage from './admin/RegisterPage';
import CustomerDashboardPage from './admin/CustomerDashboardPage';
import BrowseProvidersPage from './admin/BrowseProvidersPage';
import ProviderProfilePage from './admin/ProviderProfilePage';
import CreateBookingPage from './admin/CreateBookingPage';
import BookingDetailsPage from './admin/BookingDetailsPage';
import MyBookingsPage from './admin/MyBookingsPage';
import ServiceProviderDashboard from './admin/ServiceProviderDashboard';
import ServiceProviderProfileSetup from './admin/ServiceProviderProfileSetup';
import ProfilePage from './admin/ProfilePage';
import ChangePasswordPage from './admin/ChangePasswordPage';
import AdminDashboard from './admin/AdminDashboard';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  const { user } = useAuthStore();

  return (
    <Router>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/login" element={user ? <Navigate to={user.role === 'admin' ? '/admin/dashboard' : user.role === 'service_provider' ? '/provider/dashboard' : '/dashboard'} /> : <LoginPage />} />
        <Route path="/register" element={user ? <Navigate to={user.role === 'admin' ? '/admin/dashboard' : user.role === 'service_provider' ? '/provider/dashboard' : '/dashboard'} /> : <RegisterPage />} />
        <Route path="/providers" element={<BrowseProvidersPage />} />
        <Route path="/providers/:id" element={<ProviderProfilePage />} />
      </Routes>
    </Router>
  );
}

```

```

    path="/dashboard"
    element={
      <ProtectedRoute requiredRole="customer">
        <CustomerDashboardPage />
      </ProtectedRoute>
    }
  />
</Route>
<Route
  path="/my-bookings"
  element={
    <ProtectedRoute>
      <MyBookingsPage />
    </ProtectedRoute>
  }
/>
<Route
  path="/book/:providerId"
  element={
    <ProtectedRoute requiredRole="customer">
      <CreateBookingPage />
    </ProtectedRoute>
  }
/>
<Route
  path="/bookings/:id"
  element={
    <ProtectedRoute>
      <BookingDetailsPage />
    </ProtectedRoute>
  }
/>
<Route
  path="/provider/dashboard"
  element={
    <ProtectedRoute requiredRole="service_provider">
      <ServiceProviderDashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/provider/setup"
  element={
    <ProtectedRoute requiredRole="service_provider">
      <ServiceProviderProfileSetup />
    </ProtectedRoute>
  }
/>
<Route
  path="/profile"
  element={
    <ProtectedRoute>
      <ProfilePage />
    </ProtectedRoute>
  }
/>
<Route
  path="/change-password"
  element={
    <ProtectedRoute>
      <ChangePasswordPage />
    </ProtectedRoute>
  }
/>
<Route
  path="/admin/dashboard"
  element={
    <ProtectedRoute requiredRole="admin">
      <AdminDashboard />
    </ProtectedRoute>
  }
/>
</Routes>

```

```

        <Toaster position="top-right" />
    </Router>
  );
}
export default App;

```

```

=====
File: frontend\src\components\Chatbot.tsx
=====

```

```

import { useState, useRef, useEffect } from 'react';
import { MessageCircle, X, Send, Sparkles } from 'lucide-react';
import api from '../api/api';

const Chatbot = () => {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState<Array<{ text: string; isBot: boolean }>>([
    { text: "Hello! I'm here to help you find skilled service providers. How can I assist you today?", isBot: true }
  ]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages, isOpen]);

  const handleSend = async () => {
    if (!input.trim() || loading) return;

    const userMessage = { text: input, isBot: false };
    setMessages(prev => [...prev, userMessage]);
    setInput('');
    setLoading(true);

    try {
      const response = await api.post('/chatbot/chat', { message: input });
      const botMessage = { text: response.data.response, isBot: true };
      setMessages(prev => [...prev, botMessage]);
    } catch (error) {
      const botMessage = { text: 'Sorry, I encountered an error. Please try again.', isBot: true };
      setMessages(prev => [...prev, botMessage]);
    } finally {
      setLoading(false);
    }
  };

  return (
    <>
      {!isOpen && (
        <button
          onClick={() => setIsOpen(true)}
          className="fixed bottom-6 right-6 bg-gradient-to-r from-primary-600 to-primary-500 text-white p-4 rounded-full shadow-lg shadow-primary-600/30 hover:shadow-primary-600/50 hover:scale-105 transition-all z-50 animate-bounce-slow group"
          >
            <MessageCircle className="h-6 w-6 group-hover:rotate-12 transition-transform" />
          </button>
        )}
      {isOpen && (

```

```

<div className="fixed bottom-6 right-6 w-96 h-[500px] glass-card rounded-2xl
shadow-2xl flex flex-col z-50 border border-white/10 animate-slide-up overflow-
hidden">
  <div className="bg-gradient-to-r from-primary-600 to-primary-500 p-4 flex
justify-between items-center">
    <div className="flex items-center space-x-2 text-white">
      <Sparkles className="h-5 w-5" />
      <h3 className="font-bold">Service Assistant</h3>
    </div>
    <button
      onClick={() => setIsOpen(false)}
      className="text-white/80 hover:text-white hover:bg-white/10 rounded-
full p-1 transition-colors">
      >
      <X className="h-5 w-5" />
    </button>
  </div>

  <div className="flex-1 overflow-y-auto p-4 space-y-4 bg-dark-900/95
scrollbar-thin scrollbar-thumb-dark-700">
    {messages.map((msg, idx) => (
      <div
        key={idx}
        className={`flex ${msg.isBot ? 'justify-start' : 'justify-end'}`}
      >
        <div
          className={`max-w-[85%] rounded-2xl p-3 text-sm ${msg.isBot
? 'bg-dark-800 text-white border border-white/5 rounded-tl-sm'
: 'bg-primary-600 text-white rounded-br-sm shadow-md shadow-
primary-900/20'
        }`}
        >
          {msg.text}
        </div>
      </div>
    ))}
    {loading && (
      <div className="flex justify-start">
        <div className="bg-dark-800 rounded-2xl p-3 border border-white/5
rounded-tl-sm flex space-x-1">
          <div className="w-2 h-2 bg-dark-400 rounded-full animate-bounce"
style={{ animationDelay: '0s' }} />
          <div className="w-2 h-2 bg-dark-400 rounded-full animate-bounce"
style={{ animationDelay: '0.2s' }} />
          <div className="w-2 h-2 bg-dark-400 rounded-full animate-bounce"
style={{ animationDelay: '0.4s' }} />
        </div>
      </div>
    )}
    <div ref={messagesEndRef} />
  </div>

  <div className="p-4 bg-dark-900 border-t border-white/5">
    <div className="relative">
      <input
        type="text"
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyPress={(e) => e.key === 'Enter' && handleSend()}
        placeholder="Ask for plumbing, wiring..."
        className="w-full bg-dark-800 border border-dark-600 rounded-xl pl-4
pr-12 py-3 text-white placeholder-dark-400 focus:outline-none focus:border-
primary-500 focus:ring-1 focus:ring-primary-500 transition-colors">
      >
      <button
        onClick={handleSend}
        disabled={!input.trim() || loading}
        className="absolute right-2 top-1/2 -translate-y-1/2 p-2 bg-
primary-600 text-white rounded-lg hover:bg-primary-500 transition-colors
disabled:opacity-50 disabled:cursor-not-allowed">
      >
      <Send className="h-4 w-4" />
    </div>
  </div>

```

```

        </button>ð
      </div>ð
    </div>ð
  </div>ð
)}ð
</>ð
);ð
};ð
ð
export default Chatbot;ð
ð

```

```

=====
File: frontend\src\components\ChatWindow.tsx
=====

```

```

import { useEffect, useState, useRef } from 'react';ð
import { io, Socket } from 'socket.io-client';ð
import { Send, MessageCircle } from 'lucide-react';ð
import api from '../api/api';ð
import { useAuthStore } from '../store/authStore';ð
ð
interface Message {ð
  bookingId: string;ð
  senderId: string;ð
  message: string;ð
  timestamp: Date;ð
}ð
ð
interface ChatWindowProps {ð
  bookingId: string;ð
}ð
ð
const ChatWindow = ({ bookingId }: ChatWindowProps) => {ð
  const { token, user } = useAuthStore();ð
  const [socket, setSocket] = useState<Socket | null>(null);ð
  const [messages, setMessages] = useState<Message[]>([]);ð
  const [input, setInput] = useState('');ð
  const messagesEndRef = useRef<HTMLDivElement>(null);ð
ð
  useEffect(() => {ð
    if (!token || !user) return;ð
ð
    // Fetch initial messagesð
    const fetchMessages = async () => {ð
      try {ð
        const response = await api.get(`/bookings/${bookingId}/messages`);ð
        const formattedMessages = response.data.map((msg: any) => ({ð
          bookingId: msg.bookingId,ð
          senderId: msg.senderId,ð
          message: msg.content,ð
          timestamp: new Date(msg.createdAt),ð
        }));ð
        setMessages(formattedMessages);ð
      } catch (error) {ð
        console.error('Failed to fetch messages:', error);ð
      }ð
    };ð
ð
    fetchMessages();ð
ð
    const newSocket = io(import.meta.env.VITE_SOCKET_URL || 'http://localhost:5000', {ð
      auth: { token },ð
    });ð
ð
    newSocket.on('connect', () => {ð
      newSocket.emit('join_booking', bookingId);ð
    });ð
ð

```

```

    newSocket.on('new_message', (message: Message) => {
      setMessages(prev => [...prev, {
        ...message,
        timestamp: new Date(message.timestamp)
      }]);
    });

    newSocket.on('connect_error', (error) => {
      console.error('Socket connection error:', error);
    });

    setSocket(newSocket);

    return () => {
      newSocket.close();
    };
  }, [bookingId, token, user]);

  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  const handleSend = () => {
    if (!input.trim() || !socket) return;

    socket.emit('send_message', {
      bookingId,
      message: input,
    });

    setInput('');
  };

  return (
    <div className="flex flex-col h-[600px] bg-white rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
      { /* Chat Header */ }
      <div className="px-6 py-4 bg-gray-50/50 border-b border-gray-100 flex items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="bg-primary-100 p-2 rounded-lg">
            <MessageCircle className="h-5 w-5 text-primary-600" />
          </div>
          <div>
            <h2 className="text-sm font-bold text-gray-900">Conversation</h2>
            <div className="flex items-center gap-1.5">
              <span className="flex h-2 w-2 rounded-full bg-green-500"></span>
              <span className="text-[10px] font-bold text-gray-400 uppercase tracking-widest">Live Portal</span>
            </div>
          </div>
        </div>
      </div>
      { /* Messages Area */ }
      <div className="flex-1 overflow-y-auto px-6 py-6 space-y-6 bg-[url('https://www.transparenttextures.com/patterns/cubes.png')] bg-fixed">
        {messages.length === 0 ? (
          <div className="h-full flex flex-col items-center justify-center text-center px-12">
            <div className="bg-gray-50 p-6 rounded-full mb-4">
              <MessageCircle className="h-12 w-12 text-gray-200" />
            </div>
            <h3 className="text-gray-900 font-bold mb-1">Secure Channel Established</h3>
            <p className="text-sm text-gray-400 leading-relaxed">
              Start your conversation with the service professional here. All messages are encrypted and logged for your safety.
            </p>
          </div>
        ) : (
          messages.map((msg, idx) => {

```

```

        const isOwn = msg.senderId === user?.id || String(msg.senderId) ===
String(user?.id);
        return (
          <div
            key={idx}
            className={`flex ${isOwn ? 'justify-end' : 'justify-start'} animate-
fade-in-up`}
            >
            <div className={`flex flex-col ${isOwn ? 'items-end' : 'items-
start'} max-w-[80%]`} >
              <div
                className={`px-4 py-3 rounded-2xl text-sm leading-relaxed shadow-
sm ${isOwn
                  ? 'bg-primary-600 text-white rounded-tr-none'
                  : 'bg-white text-gray-700 border border-gray-100 rounded-tl-
none'}`}
                >
                {msg.message}
              </div>
              <span className="text-[10px] font-medium text-gray-400 mt-1.5 px-1
uppercase tracking-tighter">
                {new Date(msg.timestamp).toLocaleTimeString([], { hour: '2-
digit', minute: '2-digit' })}
              </span>
            </div>
          </div>
        );
      })
    )
    <div ref={messagesEndRef} />
  </div>
)
{ /* Input Area */ }
<div className="p-4 bg-white border-t border-gray-100">
  <div className="relative flex items-center bg-gray-50 rounded-xl p-2 focus-
within:ring-2 focus-within:ring-primary-500/20 transition-all border border-
transparent focus-within:border-primary-100">
    <input
      type="text"
      value={input}
      onChange={(e) => setInput(e.target.value)}
      onPress={(e) => e.key === 'Enter' && handleSend()}
      placeholder="Type your message here..."
      className="flex-1 bg-transparent border-none focus:ring-0 text-sm px-3
py-2 text-gray-700 placeholder-gray-400"
    />
    <button
      onClick={handleSend}
      disabled={!input.trim()}
      className="bg-primary-600 text-white p-2.5 rounded-lg hover:bg-
primary-700 transition-all active:scale-95 disabled:opacity-40 disabled:grayscale"
    >
      <Send className="h-4 w-4" />
    </button>
  </div>
  <p className="text-[9px] text-center text-gray-400 mt-2 font-medium
uppercase tracking-widest">
    Press enter to transmit message
  </p>
</div>
);
};
export default ChatWindow;

```

=====

File: frontend\src\components\Logo.tsx

```
import React from 'react';
interface LogoProps {
  className?: string;
}
export const Logo: React.FC<LogoProps> = ({ className = "h-8 w-8" }) => {
  return (
    <svg
      viewBox="0 0 100 100"
      fill="none"
      xmlns="http://www.w3.org/2000/svg"
      className={className}
    >
      <circle cx="50" cy="50" r="45" stroke="currentColor" strokeWidth="8"
        className="text-primary-600" fill="white" />
      <path
        d="M35 35 L65 35 L65 45 L45 45 L45 55 L65 55 L65 65 L35 65 L35 55
        L55 55 L55 45 L35 45 Z"
        fill="currentColor"
        className="text-primary-600"
      />
      <circle cx="70" cy="30" r="10" fill="currentColor" className="text-
        primary-500" />
    </svg>
  );
};
export const LogoText: React.FC = () => (
  <div className="flex items-center gap-2">
    <Logo className="h-10 w-10 text-primary-600" />
    <span className="text-xl font-bold bg-clip-text text-transparent bg-gradient-
      to-r from-primary-600 to-primary-800">
      Service Connect
    </span>
  </div>
);
```

File: frontend\src\components\Navbar.tsx

```
import { useState, useRef, useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuthStore } from '../store/authStore';
import toast from 'react-hot-toast';
import { LogoText } from './Logo';
import {
  Logout,
  User as UserIcon,
  Home,
  Settings,
  ShieldCheck,
  ShieldAlert,
  UserX,
  Trash2,
  Key
} from 'lucide-react';
const Navbar = () => {
  const { user, logout } = useAuthStore();
  const navigate = useNavigate();
  const [isDropdownOpen, setIsDropdownOpen] = useState(false);
  const dropdownRef = useRef<HTMLDivElement>(null);
  const handleLogout = () => {
    logout();
```



```

    setIsDropdownOpen(false);
    navigate('/');
  };
}

useEffect(() => {
  const handleClickOutside = (event: MouseEvent) => {
    if (dropdownRef.current && !dropdownRef.current.contains(event.target as Node)) {
      setIsDropdownOpen(false);
    }
  };
  document.addEventListener('mousedown', handleClickOutside);
  return () => document.removeEventListener('mousedown', handleClickOutside);
}, []);

return (
  <nav className="bg-white shadow-md sticky top-0 z-50">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex justify-between h-16">
        <div className="flex items-center">
          <Link to="/" className="flex items-center hover:opacity-90 transition-opacity">
            <LogoText />
          </Link>
        </div>
        <div className="flex items-center space-x-4">
          <Link
            to="/"
            className="text-gray-700 hover:text-primary-600 px-3 py-2 rounded-md text-sm font-medium transition-colors"
            >
            Home
          </Link>
          <Link
            to="/providers"
            className="text-gray-700 hover:text-primary-600 px-3 py-2 rounded-md text-sm font-medium transition-colors"
            >
            Find Providers
          </Link>
          {user ? (
            <Link
              to={user.role === 'service_provider' ? '/provider/dashboard' : '/dashboard'}
              className="hidden md:flex items-center space-x-1 text-gray-700 hover:text-primary-600 px-3 py-2 rounded-md text-sm font-medium transition-colors"
              >
                <Home className="h-4 w-4" />
                <span>Dashboard</span>
              </Link>
            ) : (
              /* Profile Dropdown */
              <div className="relative" ref={dropdownRef}>
                <button
                  onClick={() => setIsDropdownOpen(!isDropdownOpen)}
                  className="flex items-center space-x-2 focus:outline-none group p-1 rounded-full hover:bg-gray-100 transition-colors"
                  >
                  {user.profilePhoto ? (
                    <img
                      src={user.profilePhoto}
                      alt="Profile"
                      className="h-8 w-8 rounded-full object-cover border-2 border-primary-500"
                    />
                  ) : (
                    <div className="h-8 w-8 rounded-full bg-primary-100 flex items-center justify-center border-2 border-primary-500">
                      <UserIcon className="h-5 w-5 text-primary-600" />
                    </div>
                  )}
                </button>
              </div>
            )}
          </div>
        </div>
      </div>
    </div>
  </nav>
);

```

```

        </div>@
    ))@
    <span className="hidden sm:block text-sm font-medium text-
gray-700 group-hover:text-primary-600 transition-colors">@
        {user.firstName}@
    </span>@
</button>@

@
    {isDropdownOpen && (@
        <div className="absolute right-0 mt-2 w-72 bg-white rounded-xl
shadow-xl py-2 border border-gray-100 ring-1 ring-black ring-opacity-5 animate-in
fade-in zoom-in duration-200 origin-top-right">@
            {/* User Info Header */}@
            <div className="px-4 py-3 border-b border-gray-50">@
                <div className="flex items-center space-x-3">@
                    {user.profilePhoto ? (@
                        <img@
                            src={user.profilePhoto}@
                            alt="Profile"@
                            className="h-12 w-12 rounded-full object-cover"@
                        />@
                    ) : (@
                        <div className="h-12 w-12 rounded-full bg-primary-50
flex items-center justify-center">@
                            <UserIcon className="h-6 w-6 text-primary-600" />@
                        </div>@
                    ))@
                    <div className="flex flex-col overflow-hidden">@
                        <span className="text-sm font-bold text-gray-900
truncate">@
                            {user.firstName} {user.lastName}@
                        </span>@
                        <span className="text-xs text-gray-500
truncate">{user.email}</span>@
                        <span className="text-xs text-gray-500
truncate">{user.phone}</span>@
                    </div>@
                </div>@
                <div className="mt-2 flex items-center space-x-1">@
                    {user.isVerified ? (@
                        <div className="flex items-center text-green-600 text-
[10px] font-bold uppercase tracking-wider bg-green-50 px-2 py-0.5 rounded-full">@
                            <ShieldCheck className="h-3 w-3 mr-1" />@
                            Verified@
                        </div>@
                    ) : (@
                        <div className="flex items-center text-amber-600 text-
[10px] font-bold uppercase tracking-wider bg-amber-50 px-2 py-0.5 rounded-full">@
                            <ShieldAlert className="h-3 w-3 mr-1" />@
                            Pending Verification@
                        </div>@
                    ))@
                </div>@
            </div>@
        </div>@
    @

    <div className="py-1">@
        {user.role === 'admin' && (@
            <Link@
                to="/admin/dashboard"@
                onClick={() => setIsDropdownOpen(false)}@
                className="flex items-center px-4 py-2 text-sm text-
indigo-600 hover:bg-indigo-50 font-bold transition-colors"@
            >@
                <ShieldCheck className="h-4 w-4 mr-3 text-indigo-500" />@
                Admin Dashboard@
            </Link>@
        )}@
        <Link@
            to="/profile"@
            onClick={() => setIsDropdownOpen(false)}@
            className="flex items-center px-4 py-2 text-sm text-
gray-700 hover:bg-gray-50 hover:text-primary-600 transition-colors"@

```

```

        >ð
        <Settings className="h-4 w-4 mr-3 text-gray-400 group-
hover:text-primary-500" />ð
        Update Profileð
        </Link>ð
        <Linkð
            to="/change-password"ð
            onClick={() => setIsDropdownOpen(false)}ð
            className="flex items-center px-4 py-2 text-sm text-
gray-700 hover:bg-gray-50 hover:text-primary-600 transition-colors"ð
        >ð
        <Key className="h-4 w-4 mr-3 text-gray-400 group-
hover:text-primary-500" />ð
        Change Passwordð
        </Link>ð
    </div>ð
ð
    <div className="border-t border-gray-50 py-1">ð
        <buttonð
            onClick={async () => {ð
                if (window.confirm('Are you sure you want to deactivate
your account?')) {ð
                    try {ð
                        const response = await fetch('http://localhost:5000/
api/auth/deactivate', {ð
                            method: 'POST',ð
                            headers: {ð
                                'Authorization': `Bearer
${useAuthStore.getState().token}`,ð
                            },ð
                        });ð
                        if (response.ok) {ð
                            toast.success('Account deactivated');ð
                            handleLogout();ð
                        } else {ð
                            const errorData = await response.json();ð
                            toast.error(errorData.message || 'Failed to
deactivate account');ð
                        }ð
                    } catch (error) {ð
                        toast.error('Failed to deactivate');ð
                    }ð
                }ð
                setIsDropdownOpen(false);ð
            }}ð
            className="flex items-center w-full px-4 py-2 text-sm text-
amber-600 hover:bg-amber-50 transition-colors"ð
        >ð
        <UserX className="h-4 w-4 mr-3" />ð
        Deactivate Accountð
        </button>ð
        <buttonð
            onClick={async () => {ð
                if (window.confirm('CRITICAL: This will permanently
delete your account details. Proceed?')) {ð
                    try {ð
                        const response = await fetch('http://localhost:5000/
api/auth/account', {ð
                            method: 'DELETE',ð
                            headers: {ð
                                'Authorization': `Bearer
${useAuthStore.getState().token}`,ð
                            },ð
                        });ð
                        if (response.ok) {ð
                            toast.success('Account deleted');ð
                            handleLogout();ð
                        } else {ð
                            const errorData = await response.json();ð
                            toast.error(errorData.message || 'Failed to delete
account');ð
                        }ð
                    }

```

```

        } catch (error) {
            toast.error('Failed to delete');
        }
    }
    setIsDropdownOpen(false);
  }}
  className="flex items-center w-full px-4 py-2 text-sm text-
red-600 hover:bg-red-50 transition-colors"
  >
    <Trash2 className="h-4 w-4 mr-3" />
    Delete Account
  </button>
</div>
)
    <div className="border-t border-gray-50 mt-1">
      <button
        onClick={handleLogout}
        className="flex items-center w-full px-4 py-2 text-sm text-
gray-700 hover:bg-gray-50 hover:text-red-600 transition-colors font-medium"
        >
        <LogOut className="h-4 w-4 mr-3 text-gray-400" />
        Logout
      </button>
    </div>
  </div>
  )}
</div>
) : (
  <div className="flex items-center space-x-2">
    <Link
      to="/login"
      className="text-gray-700 hover:text-primary-600 px-3 py-2 rounded-
md text-sm font-medium transition-colors"
      >
      Login
    </Link>
    <Link
      to="/register"
      className="bg-primary-600 text-white hover:bg-primary-700 px-4
py-2 rounded-md text-sm font-medium shadow-sm transition-all hover:shadow-md"
      >
      Sign Up
    </Link>
  </div>
  )}
</div>
</div>
</nav>
);
};
export default Navbar;

```

```

=====
File: frontend\src\components\PaymentButton.tsx
=====

```

```

import { useState } from 'react';
import { loadStripe } from '@stripe/stripe-js';
import { Elements, CardElement, useStripe, useElements } from '@stripe/react-stripe-
js';
import api from '../api/api';
import toast from 'react-hot-toast';
const stripePromise = loadStripe(import.meta.env.VITE_STRIPE_PUBLISHABLE_KEY ||
'pk_test_placeholder');

```

```

interface PaymentButtonProps {
  bookingId: string;
  amount: number;
  onSuccess: () => void;
}

const PaymentForm = ({ bookingId, amount, onSuccess }: PaymentButtonProps) => {
  const stripe = useStripe();
  const elements = useElements();
  const [processing, setProcessing] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    if (!stripe || !elements) {
      toast.error('Payment system not ready. Please try again.');
```

return;

```

    }

    setProcessing(true);

    try {
      // Create payment intent
      const { data } = await api.post('/payments/create-intent', { bookingId });
      if (!data.clientSecret) {
        throw new Error('Failed to create payment intent');
```

}

```

      const { clientSecret } = data;

      // Confirm payment
      const { error, paymentIntent } = await stripe.confirmCardPayment(clientSecret, {
        payment_method: {
          card: elements.getElement(CardElement)!,
        },
      });

      if (error) {
        toast.error(error.message || 'Payment failed');
```

return;

```

      }

      if (paymentIntent?.status === 'succeeded') {
        await api.post('/payments/confirm', { bookingId, paymentIntentId:
paymentIntent.id });
        toast.success('Payment successful!');
        onSuccess();
      } else {
        toast.error('Payment was not successful. Please try again.');
```

}

```

      } catch (error: any) {
        const errorMessage = error.response?.data?.message || error.message ||
'Payment failed';
        toast.error(errorMessage);
      } finally {
        setProcessing(false);
      }
    };

    const handleMockPayment = async (e: React.FormEvent) => {
      e.preventDefault();
      setProcessing(true);

      try {
        await api.post('/payments/mock-confirm', { bookingId });
        toast.success('Mock Payment Successful (Bypassed Stripe)');
```

onSuccess();

```

      } catch (error: any) {
        toast.error('Mock payment failed');
```

finally {

```

        setProcessing(false);
      }
    }
  }
}

```

```

    }
  };
}
return (
  <form onSubmit={handleSubmit} className="space-y-4">
    <div className="border border-gray-300 rounded-lg p-4">
      <CardElement
        options={{
          style: {
            base: {
              fontSize: '16px',
              color: '#424770',
              '::placeholder': {
                color: '#aab7c4',
              },
            },
          },
        }}
      />
    </div>
    <button
      type="submit"
      disabled={!stripe || processing}
      className="w-full bg-primary-600 text-white py-3 rounded-lg font-semibold
hover:bg-primary-700 transition-colors disabled:opacity-50"
    >
      {processing ? 'Processing Stripe...' : `Pay 1${amount.toFixed(2)}`}
    </button>
  )
  { /* Development Testing Force-Bypass */ }
  <button
    type="button"
    onClick={handleMockPayment}
    disabled={processing}
    className="w-full bg-emerald-100 text-emerald-800 border-2 border-emerald-500 py-3 rounded-lg font-bold hover:bg-emerald-200 transition-colors disabled:opacity-50 mt-4 flex items-center justify-center gap-2"
    title="Use this to simulate a successful payment without needing a configured Stripe account"
  >
    {processing ? 'Simulating...' : `Test Payment (Bypass API)`}
  </button>
</form>
);
};
}
const PaymentButton = (props: PaymentButtonProps) => {
  return (
    <Elements stripe={stripePromise}>
      <PaymentForm {...props} />
    </Elements>
  );
};
export default PaymentButton;
}

```

```

=====
File: frontend\src\components\ProtectedRoute.tsx
=====

```

```

import { Navigate } from 'react-router-dom';
import { useAuthStore } from '../store/authStore';
import { ReactNode } from 'react';
interface ProtectedRouteProps {
  children: ReactNode;
  requiredRole?: 'customer' | 'service_provider' | 'admin';
}

```

```

const ProtectedRoute = ({ children, requiredRole }: ProtectedRouteProps) => {
  const { user } = useAuthStore();
  if (!user) {
    return <Navigate to="/login" />;
  }
  if (requiredRole && user.role !== requiredRole) {
    const fallbackPath = user.role === 'admin'
      ? '/admin/dashboard'
      : user.role === 'service_provider'
        ? '/provider/dashboard'
        : '/dashboard';
    return <Navigate to={fallbackPath} />;
  }
  return <>{children}</>;
};
export default ProtectedRoute;

```

=====
File: frontend\src\index.css
=====

```

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@400;500;600;700;800&display=swap');
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
  line-height: 1.5;
  font-weight: 400;

  color-scheme: dark;
  color: rgba(255, 255, 255, 0.9);
  background-color: #0f172a; /* dark-900 */

  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

body {
  margin: 0;
  min-height: 100vh;
  background-color: #0f172a;
  background-image: 
    radial-gradient(at 0% 0%, rgba(139, 92, 246, 0.15) 0px, transparent 50%),
    radial-gradient(at 100% 0%, rgba(245, 158, 11, 0.15) 0px, transparent 50%);
  color: #f8fafc;
}

.glass-card {
  @apply bg-white/5 backdrop-blur-lg border border-white/10 shadow-xl;
}

.text-gradient {
  @apply bg-clip-text text-transparent bg-gradient-to-r from-primary-400 to-primary-600;
}

.text-gradient-secondary {
  @apply bg-clip-text text-transparent bg-gradient-to-r from-secondary-400 to-secondary-600;
}

```

```

}ð
ð
#root {ð
  width: 100%;ð
  min-height: 100vh;ð
}ð
ð

```

```

=====
File: frontend\src\main.tsx
=====

```

```

import React from 'react'ð
import ReactDOM from 'react-dom/client'ð
import App from './App.tsx'ð
import './index.css'ð
ð
ReactDOM.createRoot(document.getElementById('root')!).render(ð
  <React.StrictMode>ð
    <App />ð
  </React.StrictMode>,ð
)ð
ð

```

```

=====
File: frontend\src\store\authStore.ts
=====

```

```

import { create } from 'zustand';ð
ð
interface User {ð
  id: string;ð
  email: string;ð
  firstName: string;ð
  lastName: string;ð
  phone: string;ð
  role: 'customer' | 'service_provider' | 'admin';ð
  isVerified: boolean;ð
  profilePhoto?: string;ð
  isActive: boolean;ð
}ð
ð
interface AuthState {ð
  user: User | null;ð
  token: string | null;ð
  setAuth: (user: User, token: string) => void;ð
  updateUser: (user: Partial<User>) => void;ð
  logout: () => void;ð
}ð
ð
const loadAuthFromStorage = () => {ð
  try {ð
    const stored = localStorage.getItem('auth-storage');ð
    if (stored) {ð
      const parsed = JSON.parse(stored);ð
      return { user: parsed.user, token: parsed.token };ð
    }ð
  } catch (error) {ð
    // Ignore errorsð
  }ð
  return { user: null, token: null };ð
};ð
ð
const saveAuthToStorage = (user: User | null, token: string | null) => {ð
  try {ð
    if (user && token) {ð
      localStorage.setItem('auth-storage', JSON.stringify({ user, token }));ð
    } else {ð

```



```

        localStorage.removeItem('auth-storage');ð
    }ð
} catch (error) {ð
    // Ignore errorsð
}ð
};ð
ð
const initialState = loadAuthFromStorage();ð
ð
export const useAuthStore = create<AuthState>((set) => ({ð
    user: initialState.user,ð
    token: initialState.token,ð
    setAuth: (user, token) => {ð
        saveAuthToStorage(user, token);ð
        set({ user, token });ð
    },ð
    updateUser: (updatedFields) => {ð
        set((state) => {ð
            const newUser = state.user ? { ...state.user, ...updatedFields } : null;ð
            if (newUser && state.token) {ð
                saveAuthToStorage(newUser, state.token);ð
            }ð
            return { user: newUser };ð
        });ð
    },ð
    logout: () => {ð
        saveAuthToStorage(null, null);ð
        set({ user: null, token: null });ð
    },ð
}));ð
ð

```

```

=====
File: frontend\src\vite-env.d.ts
=====

```

```

/// <reference types="vite/client" />ð
ð
interface ImportMetaEnv {ð
    readonly VITE_STRIPE_PUBLISHABLE_KEY: string;ð
}ð
ð
interface ImportMeta {ð
    readonly env: ImportMetaEnv;ð
}ð
ð

```

```

=====
File: backend\src\index.ts
=====

```

```

import express from 'express';ð
import cors from 'cors';ð
import dotenv from 'dotenv';ð
import mongoose from 'mongoose';ð
import { createServer } from 'http';ð
import { Server } from 'socket.io';ð
import authRoutes from './routes/auth.routes.js';ð
import serviceProviderRoutes from './routes/serviceProvider.routes.js';ð
import bookingRoutes from './routes/booking.routes.js';ð
import paymentRoutes from './routes/payment.routes.js';ð
import reviewRoutes from './routes/review.routes.js';ð
import chatbotRoutes from './routes/chatbot.routes.js';ð
import adminRoutes from './routes/admin.routes.js';ð
import { initializeSocket } from './socket/socket.js';ð
import { errorHandler } from './middleware/errorHandler.middleware.js';ð
ð
dotenv.config();ð

```

```

ð
const app = express();ð
const httpServer = createServer(app);ð
const io = new Server(httpServer, {ð
  cors: {ð
    origin: process.env.CLIENT_URL || "http://localhost:5173",ð
    credentials: trueð
  }ð
});ð
ð
app.use(cors());ð
app.use(express.json());ð
app.use(express.urlencoded({ extended: true }));ð
ð
// Routesð
app.use('/api/auth', authRoutes);ð
app.use('/api/service-providers', serviceProviderRoutes);ð
app.use('/api/bookings', bookingRoutes);ð
app.use('/api/payments', paymentRoutes);ð
app.use('/api/reviews', reviewRoutes);ð
app.use('/api/chatbot', chatbotRoutes);ð
app.use('/api/admin', adminRoutes);ð
ð
// Health checkð
app.get('/api/health', (req, res) => {ð
  res.json({ status: 'ok', message: 'Service Connect API is running' });ð
});ð
ð
// Error handling middleware (must be last)ð
app.use(errorHandler as express.ErrorRequestHandler);ð
ð
// 404 handlerð
app.use((req, res) => {ð
  res.status(404).json({ message: 'Route not found' });ð
});ð
ð
// Initialize Socket.ioð
initializeSocket(io);ð
ð
// Database connectionð
mongoose.connect(process.env.MONGODB_URI || 'mongodb://localhost:27017/service-
connect')ð
  .then(() => console.log('Connected to MongoDB'))ð
  .catch((error) => console.error('MongoDB connection error:', error));ð
ð
const PORT = process.env.PORT || 5000;ð
ð
if (process.env.NODE_ENV !== 'production') {ð
  httpServer.listen(PORT, () => {ð
    console.log(`Server running on port ${PORT}`);ð
  });ð
}ð
ð
export default app;ð
ð

```

```

=====
File: backend\src\middleware\auth.middleware.ts
=====

```

```

import { Request, Response, NextFunction } from 'express';ð
import jwt from 'jsonwebtoken';ð
ð
export interface AuthRequest extends Request {ð
  userId?: string;ð
  userRole?: string;ð
  body: any; // Ensure body property existsð
}ð
ð
export const authenticate = (req: AuthRequest, res: Response, next: NextFunction) =>
{ð

```

```

    try {
        const token = req.headers.authorization?.split(' ')[1];
        if (!token) {
            return res.status(401).json({ message: 'Authentication required' });
        }
        const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
        as { userId: string; role: string };
        req.userId = decoded.userId;
        req.userRole = decoded.role;
        next();
    } catch (error) {
        res.status(401).json({ message: 'Invalid or expired token' });
    }
};
export const authorize = (...roles: string[]) => {
    return (req: AuthRequest, res: Response, next: NextFunction) => {
        if (!req.userRole || !roles.includes(req.userRole)) {
            return res.status(403).json({ message: 'Access denied' });
        }
        next();
    };
};

```

=====
File: backend\src\middleware\errorHandler.middleware.ts
=====

```

import { Request, Response, NextFunction } from 'express';
export interface AppError extends Error {
    statusCode?: number;
    isOperational?: boolean;
}
export const errorHandler = (
    error: AppError | Error,
    req: Request,
    res: Response,
    next: NextFunction
) => {
    const appError = error as AppError;
    const statusCode = appError.statusCode || 500;
    const message = appError.message || error.message || 'Internal Server Error';
    // Log error for debugging
    console.error('Error:', {
        message: error.message,
        stack: process.env.NODE_ENV === 'development' ? error.stack : undefined,
        path: req.path,
        method: req.method
    });
    res.status(statusCode).json({
        message,
        ...(process.env.NODE_ENV === 'development' && { stack: error.stack })
    });
}
export const asyncHandler = (fn: Function) => {
    return (req: Request, res: Response, next: NextFunction) => {
        Promise.resolve(fn(req, res, next)).catch(next);
    };
};

```

```
=====
File: backend\src\models\Booking.model.ts
=====
```

```
import mongoose, { Document, Schema } from 'mongoose';
export interface IBooking extends Document {
  // _id is provided by Document as ObjectId
  customerId: mongoose.Types.ObjectId;
  serviceProviderId: mongoose.Types.ObjectId;
  serviceType: string;
  description: string;
  location: {
    address: string;
    city: string;
    state: string;
    zipCode: string;
    coordinates: {
      lat: number;
      lng: number;
    };
  };
  scheduledDate: Date;
  scheduledTime: string;
  status: 'pending' | 'confirmed' | 'in_progress' | 'completed' | 'cancelled';
  isEmergency: boolean;
  estimatedHours: number;
  totalAmount: number;
  paymentStatus: 'pending' | 'paid' | 'refunded';
  createdAt: Date;
  updatedAt: Date;
}
const BookingSchema = new Schema<IBooking>({
  customerId: {
    type: Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  serviceProviderId: {
    type: Schema.Types.ObjectId,
    ref: 'ServiceProvider',
    required: true
  },
  serviceType: {
    type: String,
    required: true
  },
  description: {
    type: String,
    required: true
  },
  location: {
    address: String,
    city: String,
    state: String,
    zipCode: String,
    coordinates: {
      lat: Number,
      lng: Number
    }
  },
  scheduledDate: {
    type: Date,
    required: true
  },
  scheduledTime: {
    type: String,
    required: true
  },
  status: {
```

```

    type: String,
    enum: ['pending', 'confirmed', 'in_progress', 'completed', 'cancelled'],
    default: 'pending'
  },
  isEmergency: {
    type: Boolean,
    default: false
  },
  estimatedHours: {
    type: Number,
    required: true,
    min: 0.5
  },
  totalAmount: {
    type: Number,
    required: true,
    min: 0
  },
  paymentStatus: {
    type: String,
    enum: ['pending', 'paid', 'refunded'],
    default: 'pending'
  }
}, {
  timestamps: true
});
export const Booking = mongoose.model<IBooking>('Booking', BookingSchema);

```

```

=====
File: backend\src\models\Message.model.ts
=====

```

```

import mongoose, { Document, Schema } from 'mongoose';
export interface IMessage extends Document {
  bookingId: mongoose.Types.ObjectId;
  senderId: mongoose.Types.ObjectId;
  content: string;
  createdAt: Date;
}
const MessageSchema = new Schema<IMessage>({
  bookingId: {
    type: Schema.Types.ObjectId,
    ref: 'Booking',
    required: true,
    index: true
  },
  senderId: {
    type: Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  content: {
    type: String,
    required: true
  }
}, {
  timestamps: { createdAt: true, updatedAt: false }
});
export const Message = mongoose.model<IMessage>('Message', MessageSchema);

```

```

=====
File: backend\src\models\Review.model.ts
=====

```

```

import mongoose, { Document, Schema } from 'mongoose';
export interface IReview extends Document {
  bookingId: mongoose.Types.ObjectId;
  customerId: mongoose.Types.ObjectId;
  serviceProviderId: mongoose.Types.ObjectId;
  rating: number;
  comment: string;
  createdAt: Date;
  updatedAt: Date;
}
const ReviewSchema = new Schema<IReview>({
  bookingId: {
    type: Schema.Types.ObjectId,
    ref: 'Booking',
    required: true
  },
  customerId: {
    type: Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  serviceProviderId: {
    type: Schema.Types.ObjectId,
    ref: 'ServiceProvider',
    required: true
  },
  rating: {
    type: Number,
    required: true,
    min: 1,
    max: 5
  },
  comment: {
    type: String,
    trim: true
  }
}, {
  timestamps: true
});
export const Review = mongoose.model<IReview>('Review', ReviewSchema);

```

```

=====
File: backend\src\models\ServiceProvider.model.ts
=====

```

```

import mongoose, { Document, Schema } from 'mongoose';
export interface IServiceProvider extends Document {
  userId: mongoose.Types.ObjectId;
  businessName?: string;
  serviceTypes: string[];
  certifications: Array<{
    name: string;
    issuer: string;
    issueDate: Date;
    expiryDate?: Date;
    documentUrl?: string;
  }>;
  experience: number;
  hourlyRate: number;
  availability: {
    monday: { start: string; end: string; available: boolean };
    tuesday: { start: string; end: string; available: boolean };
    wednesday: { start: string; end: string; available: boolean };
    thursday: { start: string; end: string; available: boolean };
  };
}

```

```

    friday: { start: string; end: string; available: boolean };
    saturday: { start: string; end: string; available: boolean };
    sunday: { start: string; end: string; available: boolean };
  };
  location: {
    address: string;
    city: string;
    state: string;
    zipCode: string;
    coordinates: {
      lat: number;
      lng: number;
    };
  };
  profileImage?: string;
  description: string;
  isVerified: boolean;
  verificationStatus: 'pending' | 'approved' | 'rejected';
  rating: number;
  totalReviews: number;
  createdAt: Date;
  updatedAt: Date;
}
}
const ServiceProviderSchema = new Schema<IServiceProvider>({
  userId: {
    type: Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    unique: true
  },
  businessName: {
    type: String,
    trim: true
  },
  serviceTypes: [{
    type: String,
    enum: ['plumber', 'electrician', 'mechanic', 'carpenter', 'painter', 'handyman',
'other']
  }],
  certifications: [{
    name: String,
    issuer: String,
    issueDate: Date,
    expiryDate: Date,
    documentUrl: String
  }],
  experience: {
    type: Number,
    required: true,
    min: 0
  },
  hourlyRate: {
    type: Number,
    required: true,
    min: 0
  },
  availability: {
    monday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    },
    tuesday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    },
    wednesday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    }
  }
});

```

```

    },
    thursday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    },
    friday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    },
    saturday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: true }
    },
    sunday: {
      start: { type: String, default: '09:00' },
      end: { type: String, default: '17:00' },
      available: { type: Boolean, default: false }
    }
  },
  location: {
    address: String,
    city: String,
    state: String,
    zipCode: String,
    coordinates: {
      lat: Number,
      lng: Number
    }
  },
  profileImage: String,
  description: {
    type: String,
    required: true
  },
  isVerified: {
    type: Boolean,
    default: false
  },
  verificationStatus: {
    type: String,
    enum: ['pending', 'approved', 'rejected'],
    default: 'pending'
  },
  rating: {
    type: Number,
    default: 0,
    min: 0,
    max: 5
  },
  totalReviews: {
    type: Number,
    default: 0
  }
}, {
  timestamps: true
});
export const ServiceProvider = mongoose.model<IServiceProvider>('ServiceProvider',
ServiceProviderSchema);

```

```

=====
File: backend\src\models\Subscription.model.ts
=====

```

```

import mongoose, { Document, Schema } from 'mongoose';

```



```

export interface ISubscription extends Document {
  customerId: mongoose.Types.ObjectId;
  planType: 'basic' | 'premium' | 'enterprise';
  serviceTypes: string[];
  monthlyPrice: number;
  status: 'active' | 'cancelled' | 'expired';
  startDate: Date;
  endDate: Date;
  autoRenew: boolean;
  createdAt: Date;
  updatedAt: Date;
}

const SubscriptionSchema = new Schema<ISubscription>({
  customerId: {
    type: Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  planType: {
    type: String,
    enum: ['basic', 'premium', 'enterprise'],
    required: true
  },
  serviceTypes: [{
    type: String
  }],
  monthlyPrice: {
    type: Number,
    required: true,
    min: 0
  },
  status: {
    type: String,
    enum: ['active', 'cancelled', 'expired'],
    default: 'active'
  },
  startDate: {
    type: Date,
    required: true
  },
  endDate: {
    type: Date,
    required: true
  },
  autoRenew: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true
});

export const Subscription = mongoose.model<ISubscription>('Subscription',
SubscriptionSchema);

```

```

=====
File: backend\src\models\User.model.ts
=====

```

```

import mongoose, { Document, Schema } from 'mongoose';
export interface IUser extends Document {
  email: string;
  password: string;
  firstName: string;
  lastName: string;
  phone: string;
  role: 'customer' | 'service_provider' | 'admin';
}

```

```

    isVerified: boolean;
    profilePhoto?: string;
    isActive: boolean;
    isDeleted: boolean;
    createdAt: Date;
    updatedAt: Date;
  }
}
const UserSchema = new Schema<IUser>({
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true
  },
  firstName: {
    type: String,
    required: true,
    trim: true
  },
  lastName: {
    type: String,
    required: true,
    trim: true
  },
  phone: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  role: {
    type: String,
    enum: ['customer', 'service_provider', 'admin'],
    required: true
  },
  isVerified: {
    type: Boolean,
    default: false
  },
  profilePhoto: {
    type: String,
    default: ''
  },
  isActive: {
    type: Boolean,
    default: true
  },
  isDeleted: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true,
  toJSON: {
    transform: (doc, ret: any) => {
      ret.id = ret._id;
      delete ret._id;
      delete ret.__v;
      delete ret.password;
      return ret;
    }
  },
  toObject: {
    transform: (doc, ret: any) => {
      ret.id = ret._id;
      delete ret._id;
    }
  }
});

```

```

        delete ret.__v;
        delete ret.password;
        return ret;
    }
}
});
export const User = mongoose.model<IUser>('User', UserSchema);

```

```

=====
File: backend\src\routes\admin.routes.ts
=====

```

```

import express from 'express';
import { User } from '../models/User.model.js';
import { Booking } from '../models/Booking.model.js';
import { ServiceProvider } from '../models/ServiceProvider.model.js';
import { Review } from '../models/Review.model.js';
import { authenticate, authorize } from '../middleware/auth.middleware.js';

const router = express.Router();

// Apply admin protection to all routes in this file
router.use(authenticate, authorize('admin'));

// Get admin dashboard stats
router.get('/stats', async (req, res) => {
    try {
        const totalUsers = await User.countDocuments({ isDeleted: false });
        const totalProviders = await ServiceProvider.countDocuments();
        const totalBookings = await Booking.countDocuments();
        const pendingVerifications = await
ServiceProvider.countDocuments({ isVerified: false });
        const activeBookings = await Booking.countDocuments({ status: { $in:
['pending', 'confirmed', 'in_progress'] } });
        // Calculate total revenue
        const completedBookings = await Booking.find({ status: 'completed' });
        const totalRevenue = completedBookings.reduce((sum, b) => sum +
b.totalAmount, 0);

        // Get recent bookings
        const recentBookings = await Booking.find()
            .populate('customerId', 'firstName lastName')
            .populate({
                path: 'serviceProviderId',
                populate: {
                    path: 'userId',
                    select: 'firstName lastName'
                }
            })
            .sort({ createdAt: -1 })
            .limit(5);

        res.json({
            stats: {
                totalUsers,
                totalProviders,
                totalBookings,
                pendingVerifications,
                totalRevenue,
                activeBookings
            },
            recentBookings
        });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});

```

```

    }
    // Get all users
    router.get('/users', async (req, res) => {
      try {
        const users = await User.find({ isDeleted: false }).sort({ createdAt: -1 });
        res.json(users);
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Get all service providers
    router.get('/providers', async (req, res) => {
      try {
        const providers = await ServiceProvider.find()
          .populate('userId', 'firstName lastName email phone profilePhoto isVerified')
          .sort({ createdAt: -1 });
        res.json(providers);
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Get all bookings
    router.get('/bookings', async (req, res) => {
      try {
        const bookings = await Booking.find()
          .populate('customerId', 'firstName lastName email')
          .populate({
            path: 'serviceProviderId',
            populate: {
              path: 'userId',
              select: 'firstName lastName email'
            }
          })
          .sort({ createdAt: -1 });
        res.json(bookings);
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Verify a service provider
    router.post('/providers/:id/verify', async (req, res) => {
      try {
        const provider = await ServiceProvider.findById(req.params.id);
        if (!provider) {
          return res.status(404).json({ message: 'Provider not found' });
        }
        provider.isVerified = true;
        await provider.save();
        // Also update the associated User model verification status
        await User.findByIdAndUpdate(provider.userId, { isVerified: true });
        res.json({ message: 'Provider verified successfully', provider });
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Toggle user active status
    router.post('/users/:id/toggle-active', async (req, res) => {
      try {
        const user = await User.findById(req.params.id);
        if (!user) {
          return res.status(404).json({ message: 'User not found' });
        }
        user.isActive = !user.isActive;

```

```

        await user.save();
    }
    res.json({ message: `User ${user.isActive ? 'activated' : 'deactivated'}
successfully`, user });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});
// Update user details
router.put('/users/:id', async (req, res) => {
    try {
        const { firstName, lastName, email, role } = req.body;
        const user = await User.findByIdAndUpdate(
            req.params.id,
            { firstName, lastName, email, role },
            { new: true }
        );
        if (!user) return res.status(404).json({ message: 'User not found' });
        res.json({ message: 'User updated successfully', user });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});
// Delete user permanently
router.delete('/users/:id', async (req, res) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) return res.status(404).json({ message: 'User not found' });
        // Also remove provider profile if exists
        if (user.role === 'service_provider') {
            await ServiceProvider.findOneAndDelete({ userId: user._id });
        }
        await User.findByIdAndDelete(req.params.id);
        res.json({ message: 'User deleted successfully' });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});
// Reject/Remove service provider
router.delete('/providers/:id', async (req, res) => {
    try {
        const provider = await ServiceProvider.findById(req.params.id);
        if (!provider) return res.status(404).json({ message: 'Provider not
found' });
        // Change user role back to customer if they are removed as provider
        await User.findByIdAndUpdate(provider.userId, { role: 'customer',
isVerified: false });
        await ServiceProvider.findByIdAndDelete(req.params.id);
        res.json({ message: 'Provider removed and role reset to customer' });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});
// Update booking status
router.put('/bookings/:id/status', async (req, res) => {
    try {
        const { status } = req.body;
        const validStatuses = ['pending', 'confirmed', 'in_progress', 'completed',
'cancelled'];
        if (!validStatuses.includes(status)) {
            return res.status(400).json({ message: 'Invalid status' });
        }
    }
}

```

```

const booking = await Booking.findByIdAndUpdate(
  req.params.id,
  { status },
  { new: true }
).populate('customerId', 'firstName lastName email')
  .populate({
    path: 'serviceProviderId',
    populate: { path: 'userId', select: 'firstName lastName email' }
  });

if (!booking) return res.status(404).json({ message: 'Booking not found' });
res.json({ message: 'Booking status updated', booking });
} catch (error: any) {
  res.status(500).json({ message: error.message });
}
});

// Get all reviews
router.get('/reviews', async (req, res) => {
  try {
    const reviews = await Review.find()
      .populate('customerId', 'firstName lastName')
      .populate({
        path: 'serviceProviderId',
        populate: { path: 'userId', select: 'firstName lastName' }
      })
      .sort({ createdAt: -1 });
    res.json(reviews);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

// Delete a review
router.delete('/reviews/:id', async (req, res) => {
  try {
    const review = await Review.findByIdAndDelete(req.params.id);
    if (!review) return res.status(404).json({ message: 'Review not found' });

    // Update provider rating
    const provider = await ServiceProvider.findById(review.serviceProviderId);
    if (provider) {
      const remainingReviews = await Review.find({ serviceProviderId:
provider._id });
      if (remainingReviews.length > 0) {
        const totalRating = remainingReviews.reduce((sum, r) => sum +
r.rating, 0);
        provider.rating = Number((totalRating /
remainingReviews.length).toFixed(1));
        provider.totalReviews = remainingReviews.length;
      } else {
        provider.rating = 0;
        provider.totalReviews = 0;
      }
      await provider.save();
    }

    res.json({ message: 'Review deleted successfully' });
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

export default router;

```

```

=====
File: backend\src\routes\auth.routes.ts
=====

```

```

import express from 'express';

```

```

import { body, validationResult } from 'express-validator';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { User } from '../models/User.model.js';
import { authenticate, AuthRequest } from '../middleware/auth.middleware.js';
import { Request, Response } from 'express';

const router = express.Router();

// Register
router.post('/register',
  [
    body('email').isEmail().normalizeEmail(),
    body('password').isLength({ min: 6 }),
    body('firstName').trim().notEmpty(),
    body('lastName').trim().notEmpty(),
    body('phone').trim().isMobilePhone('any').withMessage('Please enter a valid phone number'),
    body('role').isIn(['customer', 'service_provider'])
  ],
  async (req: Request, res: Response) => {
    try {
      const errors = validationResult(req);
      if (!errors.isEmpty()) {
        return res.status(400).json({ errors: errors.array() });
      }

      const { email, password, firstName, lastName, phone, role } = req.body;

      const existingUser = await User.findOne({ email });
      if (existingUser) {
        return res.status(400).json({ message: 'User already exists' });
      }

      const hashedPassword = await bcrypt.hash(password, 10);
      const user = new User({
        email,
        password: hashedPassword,
        firstName,
        lastName,
        phone,
        role,
        isVerified: role === 'customer' // Customers auto-verified, providers need manual verification
      });

      await user.save();

      const token = jwt.sign(
        { userId: user._id, role: user.role },
        process.env.JWT_SECRET || 'fallback_secret',
        { expiresIn: '7d' }
      );

      res.status(201).json({
        token,
        user: {
          id: user._id,
          email: user.email,
          firstName: user.firstName,
          lastName: user.lastName,
          phone: user.phone,
          role: user.role,
          isVerified: user.isVerified,
          profilePhoto: user.profilePhoto,
          isActive: user.isActive
        }
      });
    } catch (error: any) {
      res.status(500).json({ message: error.message });
    }
  }
);

```

```

    });
    // Login
    router.post('/login',
    [
        body('identifier').notEmpty().withMessage('Email or phone number is required'),
        body('password').notEmpty()
    ],
    async (req: Request, res: Response) => {
        try {
            const errors = validationResult(req);
            if (!errors.isEmpty()) {
                return res.status(400).json({ errors: errors.array() });
            }

            const { identifier, password } = req.body;

            // Find user by email OR phone
            const user = await User.findOne({
                $or: [
                    { email: identifier.toLowerCase() },
                    { phone: identifier }
                ]
            });

            if (!user) {
                return res.status(401).json({ message: 'Invalid credentials' });
            }

            const isMatch = await bcrypt.compare(password, user.password);
            if (!isMatch) {
                return res.status(401).json({ message: 'Invalid credentials' });
            }

            const token = jwt.sign(
                { userId: user._id, role: user.role },
                process.env.JWT_SECRET || 'fallback_secret',
                { expiresIn: '7d' }
            );

            res.json({
                token,
                user: {
                    id: user._id,
                    email: user.email,
                    firstName: user.firstName,
                    lastName: user.lastName,
                    phone: user.phone,
                    role: user.role,
                    isVerified: user.isVerified,
                    profilePhoto: user.profilePhoto,
                    isActive: user.isActive
                }
            });
        } catch (error: any) {
            res.status(500).json({ message: error.message });
        }
    });

    // Get current user
    router.get('/me', authenticate, async (req: AuthRequest, res) => {
        try {
            const user = await User.findById(req.userId).select('-password');
            if (!user) {
                return res.status(404).json({ message: 'User not found' });
            }
            res.json(user);
        } catch (error: any) {
            res.status(500).json({ message: error.message });
        }
    });

```



```

    }
    // Update profile
    router.put('/profile', authenticate,
    [
        body('firstName').optional().trim().notEmpty(),
        body('lastName').optional().trim().notEmpty(),
        body('phone').optional().trim().isMobilePhone('any').withMessage('Please enter a
valid phone number'),
        body('email').optional().isEmail().normalizeEmail(),
        body('profilePhoto').optional().isString()
    ],
    async (req: AuthRequest, res: Response) => {
        try {
            const errors = validationResult(req);
            if (!errors.isEmpty()) {
                return res.status(400).json({ errors: errors.array() });
            }

            const { firstName, lastName, phone, email, profilePhoto } = req.body;
            const user = await User.findById(req.userId);

            if (!user) {
                return res.status(404).json({ message: 'User not found' });
            }

            if (firstName) user.firstName = firstName;
            if (lastName) user.lastName = lastName;
            if (phone) user.phone = phone;
            if (email) {
                const existingUser = await User.findOne({ email, _id: { $ne: req.userId } });
                if (existingUser) {
                    return res.status(400).json({ message: 'Email already in use' });
                }
                user.email = email;
            }
            if (profilePhoto !== undefined) user.profilePhoto = profilePhoto;

            await user.save();

            res.json({
                message: 'Profile updated successfully',
                user: {
                    id: user._id,
                    email: user.email,
                    firstName: user.firstName,
                    lastName: user.lastName,
                    phone: user.phone,
                    role: user.role,
                    isVerified: user.isVerified,
                    profilePhoto: user.profilePhoto,
                    isActive: user.isActive
                }
            });
        } catch (error: any) {
            res.status(500).json({ message: error.message });
        }
    }
);
// Change password
router.put('/change-password', authenticate,
[
    body('currentPassword').notEmpty(),
    body('newPassword').isLength({ min: 6 })
],
async (req: AuthRequest, res: Response) => {
    try {
        const errors = validationResult(req);
        if (!errors.isEmpty()) {
            return res.status(400).json({ errors: errors.array() });
        }
    }
}

```

```

    const { currentPassword, newPassword } = req.body;
    const user = await User.findById(req.userId);

    if (!user) {
        return res.status(404).json({ message: 'User not found' });
    }

    const isMatch = await bcrypt.compare(currentPassword, user.password);
    if (!isMatch) {
        return res.status(400).json({ message: 'Invalid current password' });
    }

    user.password = await bcrypt.hash(newPassword, 10);
    await user.save();

    res.json({ message: 'Password changed successfully' });
} catch (error: any) {
    res.status(500).json({ message: error.message });
}
});

// Deactivate account
router.post('/deactivate', authenticate, async (req: AuthRequest, res: Response) => {
    try {
        const user = await User.findById(req.userId);
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }

        user.isActive = false;
        await user.save();

        res.json({ message: 'Account deactivated successfully' });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});

// Delete account (soft delete)
router.delete('/account', authenticate, async (req: AuthRequest, res: Response) => {
    try {
        const user = await User.findById(req.userId);
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }

        user.isActive = false;
        user.isDeleted = true;
        await user.save();

        res.json({ message: 'Account deleted successfully' });
    } catch (error: any) {
        res.status(500).json({ message: error.message });
    }
});

export default router;

```

```

=====
File: backend\src\routes\booking.routes.ts
=====

```

```

import express from 'express';
import { body, validationResult } from 'express-validator';
import { Booking } from '../models/Booking.model.js';
import { ServiceProvider } from '../models/ServiceProvider.model.js';
import { authenticate, AuthRequest } from '../middleware/auth.middleware.js';
import { compareObjectIds } from '../utils/objectId.js';

```

```

import { Message } from '../models/Message.model.js';
const router = express.Router();
// Create a booking
router.post('/', authenticate, async (req: AuthRequest, res) => {
  try {
    if (req.userRole !== 'customer') {
      return res.status(403).json({ message: 'Only customers can create bookings' });
    }

    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const {
      serviceProviderId,
      serviceType,
      description,
      location,
      scheduledDate,
      scheduledTime,
      isEmergency,
      estimatedHours
    } = req.body;

    // Validation
    if (!serviceProviderId || !serviceType || !description || !location || !
scheduledDate || !scheduledTime || !estimatedHours) {
      return res.status(400).json({ message: 'Missing required fields' });
    }

    if (estimatedHours <= 0) {
      return res.status(400).json({ message: 'Estimated hours must be greater than
0' });
    }

    const provider = await ServiceProvider.findById(serviceProviderId);
    if (!provider) {
      return res.status(404).json({ message: 'Service provider not found' });
    }

    if (!provider.isVerified) {
      return res.status(400).json({ message: 'Service provider is not verified' });
    }

    const totalAmount = provider.hourlyRate * estimatedHours * (isEmergency ? 1.5 :
1);

    const booking = new Booking({
      customerId: req.userId,
      serviceProviderId,
      serviceType,
      description,
      location,
      scheduledDate,
      scheduledTime,
      isEmergency,
      estimatedHours,
      totalAmount,
      status: 'pending'
    });

    await booking.save();
    const populated = await booking.populate([
      { path: 'serviceProviderId', select: 'businessName hourlyRate' },
      { path: 'customerId', select: 'firstName lastName' }
    ]);

    res.status(201).json(populated);
  } catch (error: any) {

```

```

    res.status(500).json({ message: error.message });
  });
  // Get bookings for current user
  router.get('/my-bookings', authenticate, async (req: AuthRequest, res) => {
    try {
      let query: any = {};
      if (req.userRole === 'customer') {
        query.customerId = req.userId;
      } else {
        // Find the ServiceProvider profile for this user
        const provider = await ServiceProvider.findOne({ userId: req.userId });
        if (!provider) {
          return res.json([]); // No profile = no bookings
        }
        query.serviceProviderId = provider._id;
      }
      const bookings = await Booking.find(query)
        .populate('customerId', 'firstName lastName email phone')
        .populate('serviceProviderId', 'businessName hourlyRate')
        .sort({ createdAt: -1 });
      res.json(bookings);
    } catch (error: any) {
      res.status(500).json({ message: error.message });
    }
  });
  // Get single booking
  router.get('/:id', authenticate, async (req: AuthRequest, res) => {
    try {
      const booking = await Booking.findById(req.params.id)
        .populate('customerId', 'firstName lastName email phone')
        .populate('serviceProviderId');
      if (!booking) {
        return res.status(404).json({ message: 'Booking not found' });
      }
      // Check authorization: Must be the customer OR the provider
      // Safely extract IDs whether they are strings, ObjectIds, or populated objects
      const bookingCustomerId = (booking.customerId as any)?._id?.toString() ||
        booking.customerId?.toString();
      const isCustomer = bookingCustomerId === req.userId;
      const providerProfile = booking.serviceProviderId as any;
      const providerUserId = providerProfile?.userId?._id?.toString() ||
        providerProfile?.userId?.toString();
      const isProvider = providerUserId === req.userId;
      if (!isCustomer && !isProvider) {
        return res.status(403).json({ message: 'Unauthorized' });
      }
      res.json(booking);
    } catch (error: any) {
      res.status(500).json({ message: error.message });
    }
  });
  // Update booking status
  router.patch('/:id/status', authenticate, async (req: AuthRequest, res) => {
    try {
      const { status } = req.body;
      const booking = await Booking.findById(req.params.id);
      if (!booking) {
        return res.status(404).json({ message: 'Booking not found' });
      }

```

```

    }
    // Service providers can update to confirmed, in_progress, or completed
    if (req.userRole === 'service_provider') {
      const provider = await ServiceProvider.findOne({ userId: req.userId });
      if (provider && compareObjectIds(booking.serviceProviderId, provider._id)) {
        if (['confirmed', 'in_progress', 'completed'].includes(status)) {
          booking.status = status as any;
          await booking.save();
          return res.json(booking);
        }
      }
    }
  }
}

// Customers can cancel
if (req.userRole === 'customer' &&
  compareObjectIds(booking.customerId, req.userId)) {
  if (status === 'cancelled') {
    booking.status = status;
    await booking.save();
    return res.json(booking);
  }
}

res.status(403).json({ message: 'Unauthorized or invalid status update' });
} catch (error: any) {
  res.status(500).json({ message: error.message });
}
});
}

// Get chat messages for a booking
router.get('/:id/messages', authenticate, async (req: AuthRequest, res) => {
  try {
    const booking = await Booking.findById(req.params.id);
    if (!booking) {
      return res.status(404).json({ message: 'Booking not found' });
    }
  }

  // Authorization: Must be customer or provider
  const isCustomer = compareObjectIds(booking.customerId, req.userId);
  const providerProfile = await
ServiceProvider.findById(booking.serviceProviderId);
  const isProvider = providerProfile && compareObjectIds(providerProfile.userId,
req.userId);

  if (!isCustomer && !isProvider) {
    return res.status(403).json({ message: 'Unauthorized' });
  }

  const messages = await Message.find({ bookingId: req.params.id })
    .sort({ createdAt: 1 });

  res.json(messages);
} catch (error: any) {
  res.status(500).json({ message: error.message });
}
});
}

export default router;
}

```

```

=====
File: backend\src\routes\chatbot.routes.ts
=====

```

```

import express from 'express';
import { body } from 'express-validator';
}
const router = express.Router();
}
// Simple chatbot endpoint

```

```

router.post('/chat', async (req, res) => {
  try {
    const { message } = req.body;

    if (!message) {
      return res.status(400).json({ message: 'Message is required' });
    }

    const lowerMessage = message.toLowerCase();

    // Simple keyword-based responses
    let response = '';

    if (lowerMessage.includes('plumber') || lowerMessage.includes('plumbing')) {
      response = 'I can help you find a plumber! Common services include leak repairs, pipe installation, drain cleaning, and fixture installation. Would you like to browse available plumbers?';
    } else if (lowerMessage.includes('electrician') || lowerMessage.includes('electrical')) {
      response = 'I can help you find an electrician! Services include wiring, panel upgrades, outlet installation, and electrical repairs. Would you like to see available electricians?';
    } else if (lowerMessage.includes('mechanic') || lowerMessage.includes('car') || lowerMessage.includes('vehicle')) {
      response = 'I can help you find a mechanic! Services include oil changes, brake repairs, engine diagnostics, and general maintenance. Would you like to find a mechanic near you?';
    } else if (lowerMessage.includes('carpenter') || lowerMessage.includes('carpentry')) {
      response = 'I can help you find a carpenter! Services include furniture building, repairs, installations, and custom woodwork. Would you like to browse carpenters?';
    } else if (lowerMessage.includes('emergency') || lowerMessage.includes('urgent')) {
      response = 'For emergencies, we have an emergency repair option that prioritizes urgent requests. Would you like to create an emergency service request?';
    } else if (lowerMessage.includes('price') || lowerMessage.includes('cost') || lowerMessage.includes('rate')) {
      response = 'Pricing varies by service provider and is based on hourly rates. Rates typically range from $1500- $15000 per hour depending on the service type and experience level. You can see exact rates when viewing provider profiles.';
    } else if (lowerMessage.includes('verified') || lowerMessage.includes('trust') || lowerMessage.includes('safe')) {
      response = 'All service providers go through a verification process including background checks and certification validation. You can also read reviews from previous customers to make an informed decision.';
    } else {
      response = 'I\'m here to help you find skilled service providers! I can assist with finding plumbers, electricians, mechanics, carpenters, and more. What service do you need today?';
    }

    res.json({ response });
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

export default router;

```

```

=====
File: backend\src\routes\payment.routes.ts
=====

```

```

import express, { Request, Response } from 'express';
import { Booking } from '../models/Booking.model.js';
import { authenticate, AuthRequest } from '../middleware/auth.middleware.js';
import { compareObjectIds } from '../utils/objectId.js';
import Stripe from 'stripe';

```

```

    }
    const stripe = new Stripe(process.env.STRIPE_SECRET_KEY || 'sk_test_fallback', {
      apiVersion: '2023-10-16'
    });
    const router = express.Router();
    // Create payment intent
    router.post('/create-intent', authenticate, async (req: AuthRequest, res: Response) => {
      try {
        const bookingId = (req as Request).body?.bookingId;
        const booking = await Booking.findById(bookingId)
          .populate('customerId');
        if (!booking) {
          return res.status(404).json({ message: 'Booking not found' });
        }
        if (!compareObjectIds(booking.customerId, req.userId)) {
          return res.status(403).json({ message: 'Unauthorized' });
        }
        if (booking.paymentStatus === 'paid') {
          return res.status(400).json({ message: 'Booking already paid' });
        }
        const paymentIntent = await stripe.paymentIntents.create({
          amount: Math.round(booking.totalAmount * 100), // Convert to cents
          currency: 'inr',
          metadata: {
            bookingId: booking._id.toString(),
            customerId: req.userId!
          }
        });
        res.json({
          clientSecret: paymentIntent.client_secret,
          amount: booking.totalAmount
        });
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Confirm payment
    router.post('/confirm', authenticate, async (req: AuthRequest, res: Response) => {
      try {
        const { bookingId, paymentIntentId } = (req as Request).body;
        const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);
        if (paymentIntent.status !== 'succeeded') {
          return res.status(400).json({ message: 'Payment not successful' });
        }
        const booking = await Booking.findById(bookingId);
        if (!booking) {
          return res.status(404).json({ message: 'Booking not found' });
        }
        booking.paymentStatus = 'paid';
        await booking.save();
        res.json({ message: 'Payment confirmed', booking });
      } catch (error: any) {
        res.status(500).json({ message: error.message });
      }
    });
    // Mock confirm for testing environments without real keys

```

```

router.post('/mock-confirm', authenticate, async (req: AuthRequest, res: Response)
=> {
  try {
    const { bookingId } = (req as Request).body;
    const booking = await Booking.findById(bookingId);
    if (!booking) {
      return res.status(404).json({ message: 'Booking not found' });
    }
    booking.paymentStatus = 'paid';
    await booking.save();
    res.json({ message: 'Mock payment successful', booking });
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});
export default router;

```

```

=====
File: backend\src\routes\review.routes.ts
=====

```

```

import express from 'express';
import { body, validationResult } from 'express-validator';
import { Review } from '../models/Review.model.js';
import { Booking } from '../models/Booking.model.js';
import { ServiceProvider } from '../models/ServiceProvider.model.js';
import { authenticate, AuthRequest } from '../middleware/auth.middleware.js';
import { compareObjectIds } from '../utils/objectId.js';

const router = express.Router();

// Create review
router.post('/', authenticate, async (req: AuthRequest, res) => {
  try {
    if (req.userRole !== 'customer') {
      return res.status(403).json({ message: 'Only customers can create reviews' });
    }

    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const { bookingId, rating, comment } = req.body;

    const booking = await Booking.findById(bookingId);
    if (!booking) {
      return res.status(404).json({ message: 'Booking not found' });
    }

    if (!compareObjectIds(booking.customerId, req.userId)) {
      return res.status(403).json({ message: 'Unauthorized' });
    }

    if (booking.status !== 'completed') {
      return res.status(400).json({ message: 'Can only review completed bookings' });
    }

    const existingReview = await Review.findOne({ bookingId });
    if (existingReview) {
      return res.status(400).json({ message: 'Review already exists for this booking' });
    }

    const review = new Review({

```



```

        bookingId,␣
        customerId: req.userId,␣
        serviceProviderId: booking.serviceProviderId,␣
        rating,␣
        comment␣
    });␣
␣
    await review.save();␣
␣
    // Update service provider rating␣
    const provider = await ServiceProvider.findById(booking.serviceProviderId);␣
    if (provider) {␣
        const reviews = await Review.find({ serviceProviderId: provider._id });␣
        const totalRating = reviews.reduce((sum, r) => sum + r.rating, 0);␣
        provider.rating = totalRating / reviews.length;␣
        provider.totalReviews = reviews.length;␣
        await provider.save();␣
    }␣
␣
    res.status(201).json(review);␣
} catch (error: any) {␣
    res.status(500).json({ message: error.message });␣
}␣
});␣
␣
// Get reviews for a service provider␣
router.get('/provider/:providerId', async (req, res) => {␣
    try {␣
        const reviews = await Review.find({ serviceProviderId: req.params.providerId })␣
        .populate('customerId', 'firstName lastName')␣
        .sort({ createdAt: -1 });␣
␣
        res.json(reviews);␣
    } catch (error: any) {␣
        res.status(500).json({ message: error.message });␣
    }␣
});␣
␣
export default router;␣
␣

```

```

=====
File: backend\src\routes\serviceProvider.routes.ts
=====

```

```

import express from 'express';␣
import { body, validationResult } from 'express-validator';␣
import { ServiceProvider } from '../models/ServiceProvider.model.js';␣
import { authenticate, AuthRequest } from '../middleware/auth.middleware.js';␣
␣
const router = express.Router();␣
␣
// Get all service providers (for customers to browse)␣
router.get('/', async (req, res) => {␣
    try {␣
        const { serviceType, city, minRating, verified } = req.query;␣
        let query: any = {};␣
␣
        if (serviceType) {␣
            query.serviceTypes = serviceType;␣
        }␣
        if (city) {␣
            query['location.city'] = new RegExp(city as string, 'i');␣
        }␣
        if (minRating) {␣
            query.rating = { $gte: Number(minRating) };␣
        }␣
        if (verified === 'true') {␣
            query.isVerified = true;␣
        }␣

```

```

    const providers = await ServiceProvider.find(query)
    .populate('userId', 'firstName lastName email phone')
    .select('-availability')
    .sort({ rating: -1 });
    res.json(providers);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

// Get nearby service providers
router.get('/nearby', async (req, res) => {
  try {
    const { lat, lng, serviceType, maxDistance = 50 } = req.query;

    if (!lat || !lng) {
      return res.status(400).json({ message: 'Latitude and longitude required' });
    }

    const providers = await ServiceProvider.find({
      serviceTypes: serviceType || { $exists: true },
      isVerified: true,
      'location.coordinates.lat': {
        $gte: Number(lat) - 0.5,
        $lte: Number(lat) + 0.5
      },
      'location.coordinates.lng': {
        $gte: Number(lng) - 0.5,
        $lte: Number(lng) + 0.5
      }
    })
    .populate('userId', 'firstName lastName email phone')
    .sort({ rating: -1 })
    .limit(20);

    res.json(providers);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

// Get single service provider
router.get('/:id', async (req, res) => {
  try {
    const provider = await ServiceProvider.findById(req.params.id)
    .populate('userId', 'firstName lastName email phone');

    if (!provider) {
      return res.status(404).json({ message: 'Service provider not found' });
    }

    res.json(provider);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});

// Create/Update service provider profile
router.post('/', authenticate, async (req: AuthRequest, res) => {
  try {
    if (req.userRole !== 'service_provider') {
      return res.status(403).json({ message: 'Only service providers can create profiles' });
    }

    const existingProfile = await ServiceProvider.findOne({ userId: req.userId });
    if (existingProfile) {
      return res.status(400).json({ message: 'Profile already exists. Use PUT to update.' });
    }
  }
});

```

```

    const provider = new ServiceProvider({
      ...req.body,
      userId: req.userId
    });
    await provider.save();
    const populated = await provider.populate('userId', 'firstName lastName email phone');
    res.status(201).json(populated);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});
// Update service provider profile
router.put('/:id', authenticate, async (req: AuthRequest, res) => {
  try {
    const provider = await ServiceProvider.findById(req.params.id);
    if (!provider) {
      return res.status(404).json({ message: 'Service provider not found' });
    }
    if (provider.userId.toString() !== req.userId) {
      return res.status(403).json({ message: 'Unauthorized' });
    }
    Object.assign(provider, req.body);
    await provider.save();
    const populated = await provider.populate('userId', 'firstName lastName email phone');
    res.json(populated);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});
// Get my profile (for service provider)
router.get('/profile/me', authenticate, async (req: AuthRequest, res) => {
  try {
    const provider = await ServiceProvider.findOne({ userId: req.userId })
      .populate('userId', 'firstName lastName email phone');
    if (!provider) {
      return res.status(404).json({ message: 'Profile not found' });
    }
    res.json(provider);
  } catch (error: any) {
    res.status(500).json({ message: error.message });
  }
});
export default router;

```

```

=====
File: backend\src\socket\socket.ts
=====

```

```

import { Server, Socket } from 'socket.io';
import jwt from 'jsonwebtoken';
import { Booking } from '../models/Booking.model.js';
import { compareObjectIds } from '../utils/objectId.js';
interface UserSocket extends Socket {
  userId?: string;
}

```

```

export const initializeSocket = (io: Server) => {
  io.use((socket: UserSocket, next) => {
    const token = socket.handshake.auth.token;

    if (!token) {
      return next(new Error('Authentication error'));
    }

    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret');
      as { userId: string; role: string };
      socket.userId = decoded.userId;
      next();
    } catch (error) {
      next(new Error('Authentication error'));
    }
  });

  io.on('connection', (socket: UserSocket) => {
    console.log('User connected:', socket.userId);

    // Join user's personal room
    if (socket.userId) {
      socket.join(`user_${socket.userId}`);
    }

    // Join booking room when user opens a booking
    socket.on('join_booking', async (bookingId: string) => {
      try {
        const booking = await Booking.findById(bookingId);

        if (booking &&
          (compareObjectIds(booking.customerId, socket.userId) ||
            compareObjectIds(booking.serviceProviderId, socket.userId))) {
          socket.join(`booking_${bookingId}`);
        }
      } catch (error) {
        console.error('Error joining booking room:', error);
      }
    });

    // Handle chat messages
    socket.on('send_message', async (data: { bookingId: string; message: string })
=> {
      try {
        const { Message } = await import('../models/Message.model.js');
        const booking = await Booking.findById(data.bookingId);

        if (booking &&
          (compareObjectIds(booking.customerId, socket.userId) ||
            compareObjectIds(booking.serviceProviderId, socket.userId))) {

          // Save to database
          const newMessage = new Message({
            bookingId: data.bookingId,
            senderId: socket.userId,
            content: data.message
          });

          await newMessage.save();

          const messageData = {
            bookingId: data.bookingId,
            senderId: socket.userId,
            message: data.message,
            timestamp: newMessage.createdAt
          };

          // Broadcast to booking room
          io.to(`booking_${data.bookingId}`).emit('new_message', messageData);
        }
      } catch (error) {

```

```

        console.error('Error sending message:', error);
    });
}

// Handle location updates
socket.on('update_location', async (data: { bookingId: string; location: { lat: number; lng: number } }) => {
    try {
        const booking = await Booking.findById(data.bookingId);

        if (booking && compareObjectIds(booking.serviceProviderId, socket.userId)) {
            // Notify customer of service provider location
            io.to(`user_${booking.customerId}`).emit('location_update', {
                bookingId: data.bookingId,
                location: data.location
            });
        }
    } catch (error) {
        console.error('Error updating location:', error);
    }
});

socket.on('disconnect', () => {
    console.log('User disconnected:', socket.userId);
});
};

```

```

=====
File: backend\src\utils\objectId.ts
=====

```

```

import mongoose from 'mongoose';

/**
 * Safely compare two ObjectIds or strings
 * Handles both ObjectId instances and string representations
 */
export const compareObjectIds = (id1: any, id2: any): boolean => {
    if (!id1 || !id2) return false;

    const str1 = id1.toString();
    const str2 = id2.toString();

    return str1 === str2;
};

/**
 * Convert ObjectId or string to string safely
 */
export const toObjectIdString = (id: mongoose.Types.ObjectId | string | undefined): string | null => {
    if (!id) return null;
    return typeof id === 'string' ? id : id.toString();
};

```